

JAVA 17 OCP · YDS STUDY SERIES

Unit 07 · Beyond Classes · Bilingual Notes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu ana kaynak, [OCP_Java_SE17_Chapter1den_Itibaren.pdf](#) dosyasındaki Chapter 7 sayfaları 345–418'i kaynak sırasını bozmadan kapsar. Tekrarlanan running header/footer ve basılı sayfa numaraları içerik sayılmamış; başlıklar, paragraflar, listeler, tablolar, figure/caption metinleri, kodlar, Summary, Exam Essentials ve Review Questions korunmuştur.

[Vocabulary](#) · [Grammar notes](#) · [Teknik hafıza notu](#)

Kaynak kapsam manifesti

- Kaynak: [exam_lecture/OCP_Java_SE17_Chapter1den_Itibaren.pdf](#)
- Chapter: 7 · Beyond Classes
- PDF sayfaları: 345–418
- Beklenen sayfa marker'ı: 74
- Eşleme biçimi: English paragraf → hemen altında Türkçe çeviri → varsa kod

İçindekiler

- 1 [Implementing Interfaces](#)
- 2 [Working with Enums](#)
- 3 [Sealing Classes](#)
- 4 [Encapsulating Data with Records](#)
- 5 [Creating Nested Classes](#)
- 6 [Understanding Polymorphism](#)
- 7 [Summary / Özet](#)
- 8 [Exam Essentials / Sınav İçin Temel Noktalar](#)
- 9 [Review Questions / Gözden Geçirme Soruları](#)

Chapter 7 · Beyond Classes · Eksiksiz çift dilli kaynak

Chapter 7 · Beyond Classes

English: OCP exam objectives covered in this chapter: Utilizing Java Object-Oriented Approach. Declare and instantiate Java objects, including nested class objects, and explain the object life cycle, including creation, reassigning references, and garbage collection. Create classes and records, and define and use instance and static fields and methods, constructors, and instance and static initializers. Understand variable scopes, use local variable type inference, apply encapsulation, and make objects immutable. Implement polymorphism and differentiate object type versus reference type. Perform type casting, identify object types using the `instanceof` operator and pattern matching. Create and use interfaces, identify functional interfaces, and utilize private, static, and default interface methods. Create and use enumerations with fields, methods, and constructors. **Türkçe:** Bu bölümde kapsanan OCP sınav hedefleri: Java'nın Nesne Yönelimli Yaklaşımını kullanmak. Nested class nesneleri dahil Java nesnelerini bildirmek ve oluşturmak; oluşturma, referansları yeniden atama ve garbage collection dahil object life cycle'i açıklamak. Class ve record oluşturmak; instance ve static field/method'ları, constructor'ları, instance/static initializer'ları tanımlamak ve kullanmak. Variable scope'larını anlamak, local variable type inference kullanmak, encapsulation uygulamak ve immutable nesneler oluşturmak. Polymorphism uygulamak ve object type ile reference type ayrımını yapmak. Type casting gerçekleştirmek; `instanceof` operator ve pattern matching ile object type'larını belirlemek. Interface oluşturup kullanmak, functional interface'leri belirlemek ve private, static ve default interface method'larından yararlanmak. Field, method ve constructor içeren enum'lar oluşturmak ve kullanmak.

English: In Chapter 6, “Class Design,” we showed you how to create, initialize, and extend both abstract and concrete classes. In this chapter, we move beyond classes to other types available in Java, including interfaces, enums, sealed classes, and records. Many of the same basic rules you learned about in Chapter 5, “Methods,” still apply, such as access modifiers and static members, although there are additional rules for each type. We also cover encapsulation and how to properly protect instance members. Finally, we conclude this chapter by discussing nested types and polymorphic inheritance. **Türkçe:** Chapter 6, “Class Design” bölümünde hem abstract hem concrete class'ların nasıl oluşturulduğunu, initialize edildiğini ve extend edildiğini gösterdik. Bu bölümde class'ların ötesine geçerek Java'daki interface, enum, sealed class ve record gibi diğer type'lara bakıyoruz. Her type'ın kendine özgü ek kuralları bulunsa da Chapter 5, “Methods” bölümünde öğrendiğiniz access modifier ve static member gibi temel kuralların çoğu hâlâ geçerlidir. Ayrıca encapsulation'ı ve instance member'ların doğru şekilde nasıl korunacağını ele alıyoruz. Bölümü nested type'ları ve polymorphic inheritance'ı tartışarak tamamlıyoruz.

English: For this chapter, remember that a Java file may have at most one public top-level type, and it must match the name of the file. This applies to classes, enums, records, and so on. Also, remember that a top-level type can only be declared with public or package access. **Türkçe:** Bu bölüm boyunca bir Java dosyasının en fazla bir public top-level type içerebileceğini ve bu type'ın adının dosya adıyla eşleşmesi gerektiğini unutmayın. Bu kural class, enum, record ve diğer type'lar için geçerlidir. Ayrıca top-level bir type yalnız public veya package access ile bildirilebilir.

English: Another top-level type available in Java is annotations. Knowing how to create a custom annotation can be a useful skill in practice, although it is not required for the exam. You should still know how to use certain annotations for the exam, such as `@Override`. **Türkçe:** Java'da bulunan başka bir top-level type da annotation'dır. Custom annotation oluşturmayı bilmek pratikte yararlı olsa da sınav için zorunlu değildir. Bununla birlikte `@Override` gibi belirli annotation'ların nasıl kullanılacağını sınav için bilmelisiniz.

Implementing Interfaces

English: In Chapter 6, you learned about abstract classes, specifically how to create and extend one. Since classes can only extend one class, they had limited use for inheritance. On the other hand, a class may implement any number of interfaces. An interface is an abstract data type that declares a list of abstract methods that any class implementing the interface must provide. **Türkçe:** Chapter 6'da abstract class'ları, özellikle bunların nasıl oluşturulup extend edildiğini öğrendiniz. Class'lar yalnız bir class'ı extend edebildiğinden inheritance açısından kullanımları sınırlıdır. Buna karşılık bir class istediği sayıda interface'i implement edebilir. Interface, onu implement eden her class'ın sağlaması gereken abstract method listesini bildiren abstract bir data type'tır.

English: Over time, the precise definition of an interface has changed, as new method types are now supported. In this chapter, we start with a rudimentary definition of an interface and expand it to cover all of the supported members. **Türkçe:** Yeni method türleri desteklenmeye başladıkça interface'in kesin tanımı zaman içinde değişmiştir. Bu bölümde temel bir interface tanımıyla başlayıp tanımı desteklenen bütün member'ları kapsayacak şekilde genişletiyoruz.

Declaring and Using an Interface

English: In Java, an interface is defined with the `interface` keyword, analogous to the `class` keyword used when defining a class. Refer to Figure 7.1 for a proper interface declaration. **Türkçe:** Java'da bir interface, class tanımlarken kullanılan `class` keyword'üne benzer biçimde `interface` keyword'üyle tanımlanır. Doğru bir interface declaration için Figure 7.1'e bakın.

Figure 7.1 · Defining an interface

English — figure callouts: Public or package access; `interface` keyword; interface name; implicit modifier. **Türkçe — şekil çağrıları:** Public veya package access; `interface` keyword'ü; interface adı; implicit modifier.

```
public abstract interface CanBurrow {  
    public static final int MINIMUM_DEPTH = 2;  
    public abstract Float getSpeed(int age);  
}
```

English — figure callouts: Constant variable and implicit modifiers; abstract interface method and implicit modifiers. **Türkçe — şekil çağrıları:** Constant variable ve implicit modifier'ları; abstract interface method ve implicit modifier'ları.

English: In Figure 7.1, our interface declaration includes an abstract method and a constant variable. Interface variables are referred to as constants because they are assumed to be public, static, and final. They are initialized with a constant value when they are declared. Since they are public and static, they can be used outside the interface declaration without requiring an instance of the interface. Figure 7.1 also includes an abstract method that, like an interface variable, is assumed to be public. **Türkçe:** Figure 7.1'deki interface declaration bir abstract method ve bir constant variable içerir. Interface variable'ları public, static ve final kabul edildiğinden constant olarak adlandırılır. Bildirildikleri anda constant bir değerle initialize edilirler. Public ve static oldukları için interface declaration'ın dışında bir interface instance'ı gerektirmeden kullanılabilirler. Figure 7.1 ayrıca interface variable gibi public kabul edilen bir abstract method içerir.

English: For brevity, we often say “an instance of an interface” in this chapter to mean an instance of a class that implements the interface. **Türkçe:** Kısa anlatım için bu bölümde “bir interface instance'ı” derken çoğunlukla interface'i implement eden bir class'ın instance'ını kastediyoruz.

English: What does it mean for a variable or method to be assumed to be something? One aspect of an interface declaration that differs from an abstract class is that it contains implicit modifiers. An implicit modifier is a modifier that the compiler automatically inserts into the code. For example, an interface is always considered to be abstract, even if it is not marked so. We cover rules and examples for implicit modifiers in more detail shortly. **Türkçe:** Bir variable veya method'un belirli bir modifier'a sahip olduğunun “kabul edilmesi” ne demektir? Interface declaration'ı abstract class'tan ayıran özelliklerden biri implicit modifier içermesidir. Implicit modifier, compiler'ın koda otomatik olarak eklediği modifier'dır. Örneğin açıkça işaretlenmese bile bir interface her zaman abstract kabul edilir. Implicit modifier kurallarını ve örneklerini birazdan daha ayrıntılı ele alıyoruz.

English: Let's start with a simple example. Imagine that we have an interface `WalksOnTwoLegs`, defined as follows: **Türkçe:** Basit bir örnek ile başlayalım. Aşağıdaki gibi tanımlanmış bir `WalksOnTwoLegs` interface'imiz olduğunu düşünün:

```
public abstract interface WalksOnTwoLegs {}
```

English: It compiles because interfaces are not required to define any methods. The abstract modifier in this example is optional for interfaces, with the compiler inserting it if it is not provided. Now, consider the following two examples, which do not compile: **Türkçe:** Interface'lerin method tanımlaması zorunlu olmadığı için bu kod derlenir. Örnekteki `abstract` modifier interface için optional'dır; yazılmazsa compiler onu ekler. Şimdi derlenmeyen şu iki örneği inceleyin:

```
public class Biped {
    public static void main(String[] args) {
        var e = new WalksOnTwoLegs(); // DOES NOT COMPILE
    }
}

public final interface WalksOnEightLegs {} // DOES NOT COMPILE
```

English: The first example doesn't compile, as `WalksOnTwoLegs` is an interface and cannot be instantiated. The second example, `WalksOnEightLegs`, doesn't compile because interfaces cannot be marked as final for the same reason that abstract classes cannot be marked as final. In other words, marking an interface final implies no class could ever implement it. **Türkçe:** İlk örnek derlenmez; çünkü `WalksOnTwoLegs` bir interface'tir ve instantiate edilemez. İkinci örnek olan `WalksOnEightLegs` da derlenmez; çünkü abstract class'ların final olamamasıyla aynı nedenle interface'ler final işaretlenemez. Başka bir deyişle interface'i final yapmak, hiçbir class'ın onu implement edemeyeceği anlamına gelir.

English: How do you use an interface? Let's say we have an interface `Climb`, defined as follows:
Türkçe: Bir interface nasıl kullanılır? Aşağıdaki gibi tanımlanmış bir `Climb` interface'imiz olduğunu varsayalım:

```
public interface Climb {  
    Number getSpeed(int age);  
}
```

English: Next, we have a concrete class `FieldMouse` that invokes the `Climb` interface by using the `implements` keyword in its class declaration, as shown in Figure 7.2. **Türkçe:** Ardından Figure 7.2'de gösterildiği gibi class declaration'ında `implements` keyword'ünü kullanarak `Climb` interface'ini uygulayan concrete `FieldMouse` class'ı vardır.

Figure 7.2 · Implementing an interface

English — figure callouts: `public` keyword required; class name; `implements` keyword required; interface name(s) separated by commas. **Türkçe — şekil çağrıları:** Zorunlu `public` keyword'ü; class adı; zorunlu `implements` keyword'ü; virgülle ayrılmış interface adları.

```
public class FieldMouse implements Climb, CanBurrow {  
    public Float getSpeed(int age) {  
        return 11f;  
    }  
}
```

English — figure callouts: Method signature matching the interface method; covariant return type.
Türkçe — şekil çağrıları: Interface method'uyla eşleşen method signature; covariant return type.

English: The `FieldMouse` class declares that it implements the `Climb` interface and includes an overridden version of `getSpeed()` inherited from the `Climb` interface. The method signature of `getSpeed()` matches exactly, and the return type is covariant, since a `Float` can be implicitly cast to a `Number`. The access modifier of the interface method is implicitly public in `Climb`, although the concrete class `FieldMouse` must explicitly declare it. **Türkçe:** `FieldMouse` class'ı `Climb` interface'ini implement ettiğini bildirir ve `Climb` interface'inden inherited `getSpeed()` method'unun overridden bir sürümünü içerir. `getSpeed()` method signature'ı tam olarak eşleşir; ayrıca `Float` implicit olarak `Number`'a cast edilebildiği için return type covariant'tır. Interface method'unun access modifier'ı `Climb` içinde implicit public'tir; ancak concrete `FieldMouse` class'ı bunu explicit olarak bildirmelidir.

English: As shown in Figure 7.2, a class can implement multiple interfaces, each separated by a comma. If any of the interfaces define abstract methods, then the concrete class is required to override them. In this case, `FieldMouse` implements the `CanBurrow` interface that we saw in Figure 7.1. In this manner, the class overrides two abstract methods at the same time with one method declaration. You learn more about duplicate and compatible interface methods in this chapter. **Türkçe:** Figure 7.2'de gösterildiği gibi bir class, adları virgülle ayrılan birden fazla interface'i implement edebilir. Interface'lerden herhangi biri abstract method tanımlıyorsa concrete class bunları override etmek zorundadır. Burada `FieldMouse`, Figure 7.1'de gördüğümüz `CanBurrow` interface'ini de implement eder. Böylece class tek bir method declaration ile iki abstract method'u aynı anda override eder. Duplicate ve compatible interface method'larını bu bölümün ilerleyen kısmında daha ayrıntılı öğreneceksiniz.

Extending an Interface

English: Like a class, an interface can extend another interface using the `extends` keyword. **Türkçe:** Bir class gibi interface de `extends` keyword'ünü kullanarak başka bir interface'i extend edebilir.

```
public interface Nocturnal {}
public interface HasBigEyes extends Nocturnal {}
```

English: Unlike a class, which can extend only one class, an interface can extend multiple interfaces. **Türkçe:** Yalnız bir class'ı extend edebilen class'ın aksine bir interface birden fazla interface'i extend edebilir.

```
public interface Nocturnal {
    public int hunt();
}

public interface CanFly {
    public void flap();
}

public interface HasBigEyes extends Nocturnal, CanFly {}

public class Owl implements HasBigEyes {
    public int hunt() { return 5; }
    public void flap() { System.out.println("Flap!"); }
}
```

English: In this example, the `Owl` class implements the `HasBigEyes` interface and must implement the `hunt()` and `flap()` methods. Extending two interfaces is permitted because interfaces are not initialized as part of a class hierarchy. Unlike abstract classes, they do not contain constructors and are not part of instance initialization. Interfaces simply define a set of rules and methods that a class implementing them must follow. **Türkçe:** Bu örnekte `Owl` class'ı `HasBigEyes` interface'ini implement eder ve `hunt()` ile `flap()` method'larını uygulamak zorundadır. İki interface'i extend etmeye izin verilir; çünkü interface'ler class hierarchy'nin bir parçası olarak initialize edilmez. Abstract class'ların aksine constructor içermez ve instance initialization'ın parçası değildir. Interface'ler yalnızca onları implement eden class'ın uyması gereken kurallar ve method'lar kümesini tanımlar.

Inheriting an Interface

English: Like an abstract class, when a concrete class inherits an interface, all of the inherited abstract methods must be implemented. We illustrate this principle in Figure 7.3. How many abstract methods does the concrete `Swan` class inherit? **Türkçe:** Abstract class'ta olduğu gibi concrete bir class bir interface'i inherit ettiğinde, inherited abstract method'ların tamamını implement etmek zorundadır. Bu ilkeyi Figure 7.3'te gösteriyoruz. Concrete `Swan` class'ı kaç abstract method inherit eder?

Figure 7.3 · Interface inheritance

```
abstract class Animal: abstract int getType()
    ↓ extends
abstract class Bird: abstract boolean canSwoop()
    ↓ extends
class Swan: ???

interface Fly: void fly() — implements — Bird
interface Swim: void swim() — implements — Swan
```

Türkçe şekil açıklaması: `Swan`, class zincirinden `getType()` ve `canSwoop()`; interface'lerden `fly()` ve `swim()` sözleşmelerini alır.

English: Give up? The concrete `Swan` class inherits four abstract methods that it must implement: `getType()`, `canSwoop()`, `fly()`, and `swim()`. Let's take a look at another example involving an abstract class that implements an interface: **Türkçe:** Pes mi ettiniz? Concrete `Swan` class'ı implement etmek zorunda olduğu dört abstract method inherit eder: `getType()`, `canSwoop()`, `fly()` ve `swim()`. Şimdi bir interface'i implement eden abstract class içeren başka bir örneğe bakalım:

```
public interface HasTail {
    public int getTailLength();
}

public interface HasWhiskers {
    public int getNumberOfWhiskers();
}

public abstract class HarborSeal implements HasTail, HasWhiskers {}
public class CommonSeal extends HarborSeal {} // DOES NOT COMPILE
```

English: The `HarborSeal` class compiles because it is abstract and not required to implement any of the abstract methods it inherits. The concrete `CommonSeal` class, though, must override all inherited abstract methods. **Türkçe:** `HarborSeal` class'ı abstract olduğu ve inherit ettiği abstract method'ları implement etmek zorunda olmadığı için derlenir. Buna karşılık concrete `CommonSeal` class'ı inherited abstract method'ların tamamını override etmek zorundadır.

Mixing Class and Interface Keywords

English: The exam creators are fond of questions that mix class and interface terminology. Although a class can implement an interface, a class cannot extend an interface. Likewise, while an interface can extend another interface, an interface cannot implement another interface. The following examples illustrate these principles: **Türkçe:** Sınav hazırlayanlar class ve interface terminolojisini karıştıran soruları sever. Bir class interface'i implement edebilse de interface'i extend edemez. Benzer biçimde bir interface başka bir interface'i extend edebilir, fakat başka bir interface'i implement edemez. Aşağıdaki örnekler bu ilkeleri gösterir:

```
public interface CanRun {}
public class Cheetah extends CanRun {} // DOES NOT COMPILE

public class Hyena {}
public interface HasFur extends Hyena {} // DOES NOT COMPILE
```

English: The first example shows a class trying to extend an interface and doesn't compile. The second example shows an interface trying to extend a class, which also doesn't compile. Be wary of examples on the exam that mix class and interface declarations. **Türkçe:** İlk örnek bir interface'i extend etmeye çalışan class gösterir ve derlenmez. İkinci örnek bir class'ı extend etmeye çalışan interface gösterir; o da derlenmez. Sınavda class ve interface declaration'larını karıştıran örneklere karşı dikkatli olun.

Inheriting Duplicate Abstract Methods

English: Java supports inheriting two abstract methods that have compatible method declarations.

Türkçe: Java, compatible method declaration'a sahip iki abstract method'un birlikte inherit edilmesini destekler.

```
public interface Herbivore { public void eatPlants(); }
public interface Omnivore { public void eatPlants(); }

public class Bear implements Herbivore, Omnivore {
    public void eatPlants() {
        System.out.println("Eating plants");
    }
}
```

English: By compatible, we mean a method can be written that properly overrides both inherited methods: for example, by using covariant return types that you learned about in Chapter 6. The following is an example of an incompatible declaration: **Türkçe:** Compatible derken iki inherited method'u da doğru biçimde override eden tek bir method yazılabilmesini kastediyoruz; örneğin Chapter 6'da öğrendiğiniz covariant return type'lar kullanılabilir. Aşağıdaki declaration incompatible bir örnektir:

```
public interface Herbivore { public void eatPlants(); }
public interface Omnivore { public int eatPlants(); }

public class Tiger implements Herbivore, Omnivore { // DOES NOT COMPILE
    // ...
}
```

English: It's impossible to write a version of `Tiger` that satisfies both inherited abstract methods. The code does not compile, regardless of what is declared inside the `Tiger` class. **Türkçe:** İki inherited abstract method'u birden karşılayan bir `Tiger` sürümü yazmak imkânsızdır. `Tiger` class'ının içinde ne bildirilirse bildirilsin kod derlenmez.

Inserting Implicit Modifiers

English: As mentioned earlier, an implicit modifier is one that the compiler will automatically insert. It's reminiscent of the compiler inserting a default no-argument constructor if you do not define a constructor, which you learned about in Chapter 6. You can choose to insert these implicit modifiers yourself or let the compiler insert them for you. **Türkçe:** Daha önce belirtildiği gibi implicit modifier, compiler'ın otomatik olarak eklediği modifier'dır. Bu durum Chapter 6'da öğrendiğiniz, constructor tanımlamadığınızda compiler'ın default no-argument constructor eklemesini hatırlatır. Bu implicit modifier'ları kendiniz yazabilir veya compiler'ın sizin için eklemesine izin verebilirsiniz.

English: The following list includes the implicit modifiers for interfaces that you need to know for the exam: Interfaces are implicitly abstract. Interface variables are implicitly public, static, and final. Interface methods without a body are implicitly abstract. Interface methods without the private modifier are implicitly public. The last rule applies to abstract, default, and static interface methods, which we cover in the next section. **Türkçe:** Sınav için bilmeniz gereken interface implicit modifier'ları şunlardır: Interface'ler implicit olarak abstract'tır. Interface variable'ları implicit olarak public, static ve final'dır. Body'si olmayan interface method'ları implicit olarak abstract'tır. Private modifier'ı bulunmayan interface method'ları implicit olarak public'tir. Son kural, bir sonraki bölümde ele alınan abstract, default ve static interface method'ları için geçerlidir.

English: Let's take a look at an example. The following two interface definitions are equivalent, as the compiler will convert them both to the second declaration: **Türkçe:** Bir örneğe bakalım. Compiler her ikisini de ikinci declaration'a dönüştüreceği için aşağıdaki iki interface tanımı eşdeğerdir:

```
public interface Soar {
    int MAX_HEIGHT = 10;
    final static boolean UNDERWATER = true;
    void fly(int speed);
    abstract void takeoff();
    public abstract double dive();
}
```

```
public abstract interface Soar {
    public static final int MAX_HEIGHT = 10;
    public final static boolean UNDERWATER = true;
    public abstract void fly(int speed);
    public abstract void takeoff();
    public abstract double dive();
}
```

English: In this example, we've marked in bold the implicit modifiers that the compiler automatically inserts. First, the **abstract** keyword is added to the interface declaration. Next, the **public**, **static**, and **final** keywords are added to the interface variables if they do not exist. Finally, each abstract method is prepended with the **abstract** and **public** keywords if it does not contain them already.

Türkçe: Bu örnekte compiler'ın otomatik eklediği implicit modifier'lar kaynakta bold gösterilmiştir. Önce interface declaration'a **abstract** keyword'ü eklenir. Ardından interface variable'larında yoksa **public**, **static** ve **final** keyword'leri eklenir. Son olarak her abstract method'da henüz bulunmuyorsa başına **abstract** ve **public** keyword'leri getirilir.

Conflicting Modifiers

English: What happens if a developer marks a method or variable with a modifier that conflicts with an implicit modifier? For example, if an abstract method is implicitly public, can it be explicitly marked protected or private?

Türkçe: Bir geliştirici method veya variable'ı implicit modifier ile çakışan bir modifier'la işaretlerse ne olur? Örneğin bir abstract method implicit public ise explicit olarak protected veya private yapılabilir mi?

```
public interface Dance {
    private int count = 4;           // DOES NOT COMPILE
    protected void step();          // DOES NOT COMPILE
}
```

English: Neither of these interface member declarations compiles, as the compiler will apply the public modifier to both, resulting in a conflict. **Türkçe:** Compiler her ikisine de public modifier uygulayarak çakışmaya yol açacağı için bu iki interface member declaration da derlenmez.

Differences between Interfaces and Abstract Classes

English: Even though abstract classes and interfaces are both considered abstract types, only interfaces make use of implicit modifiers. How do the **play()** methods differ in the following two definitions?

Türkçe: Abstract class ve interface'in ikisi de abstract type kabul edilse de implicit modifier'ları yalnız interface'ler kullanır. Aşağıdaki iki tanımdaki **play()** method'ları nasıl farklıdır?

```
abstract class Husky {           // abstract required in class declaration
    abstract void play();         // abstract required in method declaration
}

interface Poodle {               // abstract optional in interface declaration
    void play();                 // abstract optional in method declaration
}
```

English: Both of these method definitions are considered abstract. That said, the **Husky** class will not compile if the **play()** method is not marked abstract, whereas the method in the **Poodle** interface will compile with or without the abstract modifier. **Türkçe:** Bu iki method tanımı da abstract kabul edilir. Bununla birlikte **Husky** class'ında **play()** method'u abstract işaretlenmezse class derlenmez; **Poodle** interface'indeki method ise abstract modifier yazılsa da yazılmasa da derlenir.

English: What about the access level of the `play()` method? Can you spot anything wrong with the following class definitions that use our abstract types? **Türkçe:** `play()` method'unun access level'i hakkında ne söylenebilir? Abstract type'larımızı kullanan aşağıdaki class tanımlarındaki hatayı görebiliyor musunuz?

```
public class Webby extends Husky {
    void play() {} // OK - play() is declared with package access in Husky
}

public class Georgette implements Poodle {
    void play() {} // DOES NOT COMPILE: play() is public in Poodle
}
```

English: The `Webby` class compiles, but the `Georgette` class does not. Even though the two method implementations are identical, the method in the `Georgette` class reduces the access modifier on the method from public to package access. **Türkçe:** `Webby` class'ı derlenir, fakat `Georgette` derlenmez. İki method implementation aynı olsa da `Georgette` içindeki method access modifier'ı public'ten package access'e daraltır.

Declaring Concrete Interface Methods

English: While interfaces started with abstract methods and constants, they've grown to include a lot more. Table 7.1 lists the six interface member types that you need to know for the exam. We've already covered abstract methods and constants, so we focus on the remaining four concrete methods in this section. **Türkçe:** Interface'ler abstract method ve constant ile başlamış olsa da zamanla çok daha fazlasını içerecek şekilde genişlemiştir. Table 7.1 sınav için bilmeniz gereken altı interface member type'ını listeler. Abstract method ve constant'ları zaten ele aldığımızdan bu bölümde kalan dört concrete method'a odaklanıyoruz.

Table 7.1 · Interface member types

Member type	Membership	Required modifiers	Implicit modifiers	Value/body?
Constant variable	Class	—	<code>public static final</code>	Yes
Abstract method	Instance	—	<code>public abstract</code>	No
Default method	Instance	<code>default</code>	<code>public</code>	Yes
Static method	Class	<code>static</code>	<code>public</code>	Yes
Private method	Instance	<code>private</code>	—	Yes
Private static method	Class	<code>private static</code>	—	Yes

Türkçe tablo özeti: Constant variable class üyeliğine; abstract, default ve private method'lar instance üyeliğine; static ve private static method'lar class üyeliğine sahiptir. Required ve implicit modifier'lar tabloda ayrı gösterilir; yalnız abstract method body taşımaz.

English: In Table 7.1, the membership type determines how it is able to be accessed. A method with a membership type of class is shared among all instances of the interface, whereas a method with a membership type of instance is associated with a particular instance of the interface. **Türkçe:** Table 7.1'de membership type member'a nasıl erişilebileceğini belirler. Class membership type'ına sahip method interface'in bütün instance'ları arasında paylaşılır; instance membership type'ına sahip method ise interface'in belirli bir instance'ıyla ilişkilidir.

What About protected or Package Interface Members?

English: Alongside public methods, interfaces now support private methods. They do not support protected access, though, as a class cannot extend an interface. They also do not support package access, although more likely for syntax reasons and backward compatibility. Since interface methods without an access modifier have been considered implicitly public, changing this behavior to package access would break many existing programs! **Türkçe:** Interface'ler public method'ların yanında artık private method'ları da destekler. Ancak bir class interface'i extend edemediği için protected access desteklenmez. Muhtemelen syntax ve backward compatibility nedenleriyle package access de desteklenmez. Access modifier içermeyen interface method'ları eskiden beri implicit public kabul edildiğinden bu davranışı package access'e çevirmek mevcut birçok programı bozardı.

Writing a default Interface Method

English: The first type of concrete method you should be familiar with for the exam is a default method. A default method is a method defined in an interface with the `default` keyword and includes a method body. It may be optionally overridden by a class implementing the interface. **Türkçe:** Sınav için bilmeniz gereken ilk concrete method türü default method'dur. Default method, interface içinde `default` keyword'üyle tanımlanan ve body içeren method'dur. Interface'i implement eden class isterse onu override edebilir.

English: One use of default methods is for backward compatibility. You can add a new default method to an interface without the need to modify all of the existing classes that implement the interface. The older classes will just use the default implementation of the method defined in the interface. This is where the name default method comes from! **Türkçe:** Default method'ların kullanım amaçlarından biri backward compatibility'dir. Interface'e yeni bir default method eklerken onu implement eden mevcut class'ların tamamını değiştirmeniz gerekmez. Eski class'lar interface'te tanımlanan default implementation'ı kullanır. "Default method" adı buradan gelir.

English: The following is an example of a default method defined in an interface: **Türkçe:** Aşağıda interface içinde tanımlanmış bir default method örneği vardır:

```
public interface IsColdBlooded {  
    boolean hasScales();  
  
    default double getTemperature() {  
        return 10.0;  
    }  
}
```

English: This example defines two interface methods, one abstract and one default. The following `Snake` class, which implements `IsColdBlooded`, must implement `hasScales()`. It may rely on the default implementation of `getTemperature()` or override the method with its own version: **Türkçe:** Bu örnek biri abstract, diğeri default olan iki interface method tanımlar. `IsColdBlooded` interface'ini implement eden aşağıdaki `Snake` class'ı `hasScales()` method'unu implement etmek zorundadır. `getTemperature()` için default implementation'a güvenebilir veya method'u kendi sürümüyle override edebilir:

```
public class Snake implements IsColdBlooded {
    public boolean hasScales() { return true; } // Required override

    // Source continues on the next page.
```

```
    public double getTemperature() { return 12; } // Optional override
}
```

English: Note that the default interface method modifier is not the same as the `default` label used in a switch statement or expression. Likewise, even though package access is sometimes referred to as default access, that feature is implemented by omitting an access modifier. Sorry if this is confusing! We agree Java has overused the word default over the years! **Türkçe:** Default interface method modifier'in switch statement veya expression içindeki `default` label ile aynı olmadığını unutmayın. Benzer şekilde package access bazen default access olarak adlandırılrsa da bu özellik access modifier yazılmayarak uygulanır. Kafa karıştırıyorsa üzgünüz; Java'nın yıllar boyunca "default" kelimesini fazla kullandığı konusunda biz de hemfikiriz.

English: For the exam, you should be familiar with various rules for declaring default methods. **Türkçe:** Sınav için default method bildirmeye ilişkin çeşitli kurallara aşina olmalısınız.

Default Interface Method Definition Rules

English: 1. A default method may be declared only within an interface. 2. A default method must be marked with the `default` keyword and include a method body. 3. A default method is implicitly public. 4. A default method cannot be marked abstract, final, or static. 5. A default method may be overridden by a class that implements the interface. 6. If a class inherits two or more default methods with the same method signature, then the class must override the method. **Türkçe:** 1. Default method yalnız interface içinde bildirilebilir. 2. Default method `default` keyword'üyle işaretlenmeli ve method body içermelidir. 3. Default method implicit public'tir. 4. Default method abstract, final veya static işaretlenemez. 5. Interface'i implement eden class default method'u override edebilir. 6. Bir class aynı method signature'a sahip iki veya daha fazla default method inherit ederse method'u override etmek zorundadır.

English: The first rule should give you some comfort in that you'll only see default methods in interfaces. If you see them in a class or enum on the exam, something is wrong. The second rule just denotes syntax, as default methods must use the `default` keyword. For example, the following code snippets will not compile because they mix up concrete and abstract interface methods: **Türkçe:** İlk kural, default method'ları yalnız interface'lerde göreceğinizi söyler. Sınavda class veya enum içinde görürseniz bir hata vardır. İkinci kural syntax'ı belirtir; default method `default` keyword'ünü kullanmalıdır. Örneğin aşağıdaki kodlar concrete ve abstract interface method'larını karıştırdıkları için derlenmez:

```
public interface Carnivore {
    public default void eatMeat(); // DOES NOT COMPILE

    public int getRequiredFoodAmount() { // DOES NOT COMPILE
        return 13;
    }
}
```

English: The next three rules for default methods follow from the relationship with abstract interface methods. Like abstract interface methods, default methods are implicitly public. Unlike abstract methods, though, default interface methods cannot be marked abstract since they provide a body. They also cannot be marked as final, because they are designed so that they can be overridden in classes implementing the interface, just like abstract methods. Finally, they cannot be marked static since they are associated with the instance of the class implementing the interface. **Türkçe:** Default method'lara ait sonraki üç kural abstract interface method'larıyla ilişkiden doğar. Abstract interface method'ları gibi default method'lar da implicit public'tir. Ancak body sağladıkları için abstract işaretlenemezler. Abstract method'lar gibi interface'i implement eden class'larda override edilmek üzere tasarlandıklarından final da olamazlar. Son olarak interface'i implement eden class instance'ıyla ilişkili oldukları için static işaretlenemezler.

Inheriting Duplicate default Methods

English: The last rule for creating a default interface method requires some explanation. For example, what value would the following code output? **Türkçe:** Default interface method oluşturmaya ilişkin son kural biraz açıklama gerektirir. Örneğin aşağıdaki kod hangi değeri yazdırır?

```
public interface Walk {
    public default int getSpeed() { return 5; }
}

public interface Run {
    public default int getSpeed() { return 10; }
}

public class Cat implements Walk, Run {} // DOES NOT COMPILE
```

English: In this example, `Cat` inherits the two default methods for `getSpeed()`, so which does it use? Since `Walk` and `Run` are considered siblings in terms of how they are used in the `Cat` class, it is not clear whether the code should output 5 or 10. In this case, the compiler throws up its hands and says, “Too hard, I give up!” and fails. **Türkçe:** Bu örnekte `Cat`, `getSpeed()` için iki default method inherit eder; peki hangisini kullanacaktır? `Walk` ve `Run`, `Cat` içindeki kullanımları bakımından sibling kabul edildiğinden kodun 5 mi yoksa 10 mu yazdırması gerektiği belirsizdir. Bu durumda compiler “Çok zor, pes ediyorum!” diyerek derlemeyi başarısız kılar.

English: All is not lost, though. If the class implementing the interfaces overrides the duplicate default method, the code will compile without issue. By overriding the conflicting method, the ambiguity about which version of the method to call has been removed. For example, the following modified implementation of `Cat` will compile: **Türkçe:** Yine de her şey kaybedilmiş değildir. Interface'leri implement eden class duplicate default method'u override ederse kod sorunsuz derlenir. Çakışan method override edilerek hangi sürümün çağrılacağına ilişkin ambiguity kaldırılır. Örneğin aşağıdaki değiştirilmiş `Cat` implementation derlenir:

```
public class Cat implements Walk, Run {  
    public int getSpeed() { return 1; }  
}
```

Calling a Hidden default Method

English: In the last section, we showed how our `Cat` class could override a pair of conflicting default methods, but what if the `Cat` class wanted to access the version of `getSpeed()` in `Walk` or `Run`? Is it still accessible? Yes, but it requires some special syntax. **Türkçe:** Önceki bölümde `Cat` class'ının çakışan iki default method'u nasıl override edebileceğini gösterdik. Peki `Cat`, `Walk` veya `Run` içindeki `getSpeed()` sürümüne erişmek isterse ne olur? Hâlâ erişilebilir mi? Evet, fakat özel bir syntax gerekir.

```
public class Cat implements Walk, Run {  
    public int getSpeed() {  
        return 1;  
    }  
  
    public int getWalkSpeed() {  
        return Walk.super.getSpeed();  
    }  
}
```

English: This is an area where a default method exhibits properties of both a static and instance method. We use the interface name to indicate which method we want to call, but we use the `super` keyword to show that we are following instance inheritance, not class inheritance. Note that calling `Walk.getSpeed()` or `Walk.this.getSpeed()` would not have worked. A bit confusing, we know, but you need to be familiar with this syntax for the exam. **Türkçe:** Burada default method hem static hem instance method özelliği gösterir. Hangi method'u çağırmak istediğimizi belirtmek için interface adını, class inheritance değil instance inheritance izlediğimizi göstermek için `super` keyword'ünü kullanırız. `Walk.getSpeed()` veya `Walk.this.getSpeed()` çağrılarının çalışmayacağını unutmayın. Biraz kafa karıştırıcıdır; ancak sınav için bu syntax'a aşina olmalısınız.

Declaring static Interface Methods

English: Interfaces are also declared with static methods. These methods are defined explicitly with the `static` keyword and, for the most part, behave just like static methods defined in classes. **Türkçe:** Interface'lerde static method da bildirilebilir. Bu method'lar `static` keyword'üyle explicit tanımlanır ve büyük ölçüde class'larda tanımlanan static method'lar gibi davranır.

Static Interface Method Definition Rules

English: 1. A static method must be marked with the `static` keyword and include a method body. 2. A static method without an access modifier is implicitly public. 3. A static method cannot be marked abstract or final. 4. A static method is not inherited and cannot be accessed in a class implementing the interface without a reference to the interface name. **Türkçe:** 1. Static method `static` keyword'üyle işaretlenmeli ve method body içermelidir. 2. Access modifier içermeyen static method implicit public'tir. 3. Static method abstract veya final işaretlenemez. 4. Static method inherit edilmez ve interface'i implement eden class içinde interface adına referans vermeden erişilemez.

English: These rules should follow from what you know so far of classes, interfaces, and static methods. For example, you can't declare static methods without a body in classes, either. Like default and abstract interface methods, static interface methods are implicitly public if they are declared without an access modifier. As you see shortly, you can use the private access modifier with static methods. **Türkçe:** Bu kurallar class, interface ve static method'lar hakkında şimdiye kadar öğrendiklerinizden çıkarılabilir. Örneğin class'larda da body'siz static method bildiremezsiniz. Default ve abstract interface method'ları gibi static interface method'ları da access modifier olmadan bildirilirse implicit public'tir. Birazdan göreceğiniz gibi static method'larda private access modifier kullanılabilir.

English: Let's take a look at a static interface method: **Türkçe:** Bir static interface method'una bakalım:

```
public interface Hop {
    static int getJumpHeight() {
        return 8;
    }
}
```

English: Since the method is defined without an access modifier, the compiler will automatically insert the public access modifier. The method `getJumpHeight()` works just like a static method as defined in a class. In other words, it can be accessed without an instance of a class. **Türkçe:** Method access modifier olmadan tanımlandığı için compiler public access modifier'ı otomatik ekler. `getJumpHeight()` method'u bir class'ta tanımlanan static method gibi çalışır; başka bir deyişle class instance'ı olmadan erişilebilir.

```
public class Skip {
    public int skip() {
        return Hop.getJumpHeight();
    }
}
```

English: The last rule about inheritance might be a little confusing, so let's look at an example. The following is an example of a class `Bunny` that implements `Hop` and does not compile: **Türkçe:** Inheritance hakkındaki son kural biraz kafa karıştırıcı olabilir; bu nedenle bir örneğe bakalım. Aşağıda `Hop` interface'ini implement eden ve derlenmeyen bir `Bunny` class'ı vardır:

```
public class Bunny implements Hop {
    public void printDetails() {
        System.out.println(getJumpHeight()); // DOES NOT COMPILE
    }
}
```

English: Without an explicit reference to the name of the interface, the code will not compile, even though `Bunny` implements `Hop`. This can be easily fixed by using the interface name: **Türkçe:** `Bunny`, `Hop` interface'ini implement etse bile interface adına explicit referans olmadan kod derlenmez. Interface adı kullanılarak kolayca düzeltilebilir:

```
public class Bunny implements Hop {
    public void printDetails() {
        System.out.println(Hop.getJumpHeight());
    }
}
```

English: Notice we don't have the same problem we did when we inherited two default interface methods with the same signature. Java "solved" the multiple inheritance problem of static interface methods by not allowing them to be inherited! **Türkçe:** Aynı signature'a sahip iki default interface method inherit ettiğimizde yaşadığımız sorunun burada bulunmadığına dikkat edin. Java, static interface method'larının multiple inheritance sorununu bunların inherit edilmesine izin vermeyerek "çözmüştür."

Reusing Code with private Interface Methods

English: The last two types of concrete methods that can be added to interfaces are private and private static interface methods. Because both types of methods are private, they can only be used in the interface declaration in which they are declared. For this reason, they were added primarily to reduce code duplication. For example, consider the following code sample: **Türkçe:** Interface'lere eklenebilen son iki concrete method türü private ve private static interface method'larıdır. Her iki tür de private olduğundan yalnız bildirildikleri interface declaration içinde kullanılabilir. Bu nedenle öncelikle code duplication'ı azaltmak için eklenmişlerdir. Aşağıdaki kodu inceleyin:

```
public interface Schedule {
    default void wakeUp() { checkTime(7); }
    private void haveBreakfast() { checkTime(9); }
    static void workOut() { checkTime(18); }

    private static void checkTime(int hour) {
        if (hour > 17) {
            System.out.println("You're late!");
        } else {
            System.out.println("You have " + (17 - hour) + " hours left "
                               + "to make the appointment");
        }
    }
}
```

English: You could write this interface without using a private method by copying the contents of the `checkTime()` method into the places it is used. It's a lot shorter and easier to read if you don't. Since the authors of Java were nice enough to add this feature for our convenience, we might as well use it! **Türkçe:** `checkTime()` method'unun içeriğini kullanıldığı yerlere kopyalayarak bu interface'i private method olmadan yazabilirsiniz. Ancak kopyalamazsanız kod çok daha kısa ve okunabilir olur. Java'nın geliştiricileri bu özelliği kolaylık için eklediğine göre onu kullanabiliriz.

English: We could have also declared `checkTime()` as public in the previous example, but this would expose the method to use outside the interface. One important tenet of encapsulation is to not expose the internal workings of a class or interface when not required. We cover encapsulation later in this chapter. **Türkçe:** Önceki örnekte `checkTime()` public de bildirilebilirdi; fakat bu, method'u interface dışında kullanıma açardı. Encapsulation'ın önemli ilkelerinden biri gerekmediğinde class veya interface'in iç işleyişini dışarı açmamaktır. Encapsulation'ı bu bölümün ilerleyen kısmında ele alacağız.

English: The difference between a non-static private method and a static one is analogous to the difference between an instance and static method declared within a class. In particular, it's all about what methods each can be called from. **Türkçe:** Non-static private method ile static private method arasındaki fark, class içinde bildirilen instance ve static method arasındaki farka benzer. Özellikle her birinin hangi method'lardan çağrılacağı önemlidir.

Private Interface Method Definition Rules

English: 1. A private interface method must be marked with the `private` modifier and include a method body. 2. A private static interface method may be called by any method within the interface definition. 3. A private interface method may be called only by default and other private non-static methods within the interface definition. **Türkçe:** 1. Private interface method `private` modifier'la işaretlenmeli ve method body içermelidir. 2. Private static interface method, interface definition içindeki herhangi bir method tarafından çağrılabilir. 3. Private interface method yalnız interface definition içindeki default ve diğer private non-static method'lar tarafından çağrılabilir.

English: Another way to think of it is that a private interface method is accessible only to non-static methods defined within the interface. A private static interface method, on the other hand, can be accessed by any method in the interface. For both types of private methods, a class inheriting the interface cannot directly invoke them. **Türkçe:** Başka bir düşünme biçimiyle private interface method'a yalnız interface içinde tanımlanan non-static method'lar erişebilir. Private static interface method'a ise interface'teki herhangi bir method erişebilir. Her iki private method türü de interface'i inherit eden class tarafından doğrudan çağrılmaz.

Calling Abstract Methods

English: We've talked a lot about the newer types of interface methods, but what about abstract methods? It turns out default and private non-static methods can access abstract methods declared in the interface. This is the primary reason we associate these methods with instance membership. When they are invoked, there is an instance of the interface. **Türkçe:** Yeni interface method türleri hakkında çok konuştuk; peki abstract method'lar ne olacak? Default ve private non-static method'lar interface'te bildirilen abstract method'lara erişebilir. Bu method'ları instance membership ile ilişkilendirmemizin temel nedeni budur: çağrıldıklarında bir interface instance'ı vardır.

```
public interface ZooRenovation {
    public String projectName();
    abstract String status();

    default void printStatus() {
        System.out.print("The " + projectName() + " project " + status());
    }
}
```

English: In this example, both `projectName()` and `status()` have the same modifiers (`abstract` and `public` are implicit) and can be called by the default method `printStatus()`. **Türkçe:** Bu örnekte `projectName()` ile `status()` aynı modifier'lara sahiptir (`abstract` ve `public` implicit'tir) ve default `printStatus()` method'u tarafından çağrılabilir.

Reviewing Interface Members

English: We conclude our discussion of interface members with Table 7.2, which shows the access rules for members within and outside an interface. **Türkçe:** Interface member'ları tartışmasını, interface içindeki ve dışındaki member access kurallarını gösteren Table 7.2 ile tamamlıyoruz.

Table 7.2 · Interface member access

Member	Accessible from default and private methods within the interface?	Accessible from static methods within the interface?	Accessible from methods in classes inheriting the interface?	Accessible without an instance of the interface?
Constant variable	Yes	Yes	Yes	Yes
Abstract method	Yes	No	Yes	No
Default method	Yes	No	Yes	No
Static method	Yes	Yes	Yes, interface name required	Yes, interface name required
Private method	Yes	No	No	No
Private static method	Yes	Yes	No	No

Türkçe tablo açıklaması: Constant variable'a her dört bağlamda da erişilebilir. Abstract ve default method için instance gerekir. Static method interface içinde her bağlamdan ve dışarıda interface adıyla erişilebilir. Private method türleri yalnız interface declaration içinde görünür; private non-static method static bağlamdan çağrılmaz.

English: While Table 7.2 might seem like a lot to remember, here are some quick tips for the exam: Treat abstract, default, and non-static private methods as belonging to an instance of the interface. Treat static methods and variables as belonging to the interface class object. All private interface method types are only accessible within the interface declaration. **Türkçe:** Table 7.2 ezberlenecek çok şey varmış gibi görünse de sınav için şu kısa ipuçlarını kullanın: Abstract, default ve non-static private method'ları interface instance'ına ait kabul edin. Static method ve variable'ları interface class object'ine ait kabul edin. Bütün private interface method türleri yalnız interface declaration içinde erişilebilirdir.

English: Using these rules, which of the following methods do not compile? **Türkçe:** Bu kuralları kullanarak aşağıdaki method'lardan hangilerinin derlenmediğini belirleyin.

```
public interface ZooTrainTour {
    abstract int getTrainName();
    private static void ride() {}
    default void playHorn() { getTrainName(); ride(); }
    public static void slowDown() { playHorn(); }
    static void speedUp() { ride(); }
}
```

English: The `ride()` method is private and static, so it can be accessed by any default or static method within the interface declaration. The `getTrainName()` method is abstract, so it can be accessed by a default method associated with the instance. The `slowDown()` method is static, though, and cannot call a default or private method, such as `playHorn()`, without an explicit reference object. Therefore, the `slowDown()` method does not compile. **Türkçe:** `ride()` private ve static olduğundan interface declaration içindeki herhangi bir default veya static method tarafından erişilebilir. `getTrainName()` abstract olduğundan instance ile ilişkili default method tarafından erişilebilir. Ancak `slowDown()` static'tir ve explicit bir reference object olmadan `playHorn()` gibi default veya private instance method'u çağırılmaz. Bu nedenle `slowDown()` derlenmez.

English: Give yourself a pat on the back! You just learned a lot about interfaces, probably more than you thought possible. Now take a deep breath. Ready? The next type we are going to cover is enums. **Türkçe:** Kendinizi tebrik edin! Interface'ler hakkında, muhtemelen mümkün olduğunu düşündüğünüzden de fazlasını öğrendiniz. Şimdi derin bir nefes alın. Hazır mısınız? Ele alacağımız sonraki type enum'dur.

Working with Enums

English: In programming, it is common to have a type that can have only a finite set of values, such as days of the week, seasons of the year, primary colors, and so on. An enumeration, or enum for short, is like a fixed set of constants. **Türkçe:** Programlamada haftanın günleri, yılın mevsimleri veya ana renkler gibi yalnız sonlu bir değer kümesine sahip olabilen type'lar yaygındır. Enumeration, kısaca enum, sabit bir constant kümesine benzer.

English: Using an enum is much better than using a bunch of constants because it provides type-safe checking. With numeric or `String` constants, you can pass an invalid value and not find out until runtime. With enums, it is impossible to create an invalid enum value without introducing a compiler error. **Türkçe:** Enum kullanmak, type-safe kontrol sağladığı için çok sayıda constant kullanmaktan daha iyidir. Numeric veya `String` constant'larda geçersiz bir değer gönderebilir ve bunu runtime'a kadar fark etmeyebilirsiniz. Enum'larda compiler error oluşturmadan geçersiz enum değeri üretmek mümkün değildir.

English: Enumerations show up whenever you have a set of items whose types are known at compile time. Common examples include the compass directions, the months of the year, the planets in the solar system, and the cards in a deck—well, maybe not the planets in a solar system, given that Pluto had its planetary status revoked. **Türkçe:** Type'ları compile time'da bilinen bir öge kümeniz olduğunda enum karşınıza çıkar. Pusula yönleri, yılın ayları, güneş sistemindeki gezegenler ve bir destedeki kartlar yaygın örneklerdir; gerçi Plüton'un gezegen statüsü geri alındığı için belki güneş sistemi gezegenleri o kadar iyi bir örnek değildir.

Creating Simple Enums

English: To create an enum, declare a type with the `enum` keyword, a name, and a list of values, as shown in Figure 7.4. **Türkçe:** Enum oluşturmak için Figure 7.4'te gösterildiği gibi `enum` keyword'ü, bir ad ve değer listesiyle type bildirin.

Figure 7.4 · Defining a simple enum

English — figure callouts: Public or package access; `enum` keyword; enum name. **Türkçe — şekil çağrıları:** Public veya package access; `enum` keyword'ü; enum adı.

```
public enum Season {  
    WINTER, SPRING, SUMMER, FALL;  
}
```

English — figure callouts: Enum values (comma separated); semicolon optional for simple enums. **Türkçe — şekil çağrıları:** Virgülle ayrılan enum değerleri; simple enum'larda optional semicolon.

English: We refer to an enum that contains only a list of values as a simple enum. When working with simple enums, the semicolon at the end of the list is optional. Keep the `Season` enum handy, as we use it throughout this section. **Türkçe:** Yalnız değer listesi içeren enum'a simple enum deriz. Simple enum ile çalışırken listenin sonundaki semicolon optional'dır. Bu bölüm boyunca kullanacağımız için `Season` enum'unu elinizin altında tutun.

English: Enum values are considered constants and are commonly written using snake case. For example, an enum declaring a list of ice cream flavors might include values like `VANILLA`, `ROCKY_ROAD`, `MINT_CHOCOLATE_CHIP`, and so on. **Türkçe:** Enum değerleri constant kabul edilir ve genellikle snake case ile yazılır. Örneğin dondurma aromaları listesi bildiren bir enum `VANILLA`, `ROCKY_ROAD`, `MINT_CHOCOLATE_CHIP` gibi değerler içerebilir.

English: Using an enum is super easy. **Türkçe:** Enum kullanmak son derece kolaydır.

```
var s = Season.SUMMER;  
System.out.println(Season.SUMMER); // SUMMER  
System.out.println(s == Season.SUMMER); // true
```

English: As you can see, enums print the name of the enum when `toString()` is called. They can be compared using `==` because they are like static final constants. In other words, you can use `equals()` or `==` to compare enums, since each enum value is initialized only once in the Java Virtual Machine (JVM). **Türkçe:** Görüldüğü gibi `toString()` çağrıldığında enum'lar enum değerinin adını yazdırır. Static final constant gibi olduklarından `==` ile karşılaştırılabilirler. Her enum değeri JVM'de yalnız bir kez initialize edildiği için enum karşılaştırmasında `equals()` veya `==` kullanılabilir.

English: One thing that you can't do is extend an enum. **Türkçe:** Yapamayacağınız şeylerden biri enum'u extend etmektir.

```
public enum ExtendedSeason extends Season {} // DOES NOT COMPILE
```

English: The values in an enum are fixed. You cannot add more by extending the enum. **Türkçe:** Enum içindeki değerler sabittir; enum'u extend ederek yeni değer ekleyemezsiniz.

Calling the values(), name(), and ordinal() Methods

English: An enum provides a `values()` method to get an array of all of the values. You can use this like any normal array, including in a for-each loop: **Türkçe:** Enum bütün değerlerin array'ini almak için `values()` method'unu sağlar. Bunu for-each loop dahil normal bir array gibi kullanabilirsiniz:

```
for (var season : Season.values()) {  
    System.out.println(season.name() + " " + season.ordinal());  
}
```

English: The output shows that each enum value has a corresponding `int` value, and the values are listed in the order in which they are declared: **Türkçe:** Çıktı her enum değerine karşılık gelen bir `int` bulunduğunu ve değerlerin declaration sırasıyla listelendiğini gösterir:

```
WINTER 0  
SPRING 1  
SUMMER 2  
FALL 3
```

English: The `int` value will remain the same during your program, but the program is easier to read if you stick to the human-readable enum value. You can't compare an `int` and an enum value directly anyway, since an enum is a type, like a Java class, and not a primitive `int`. **Türkçe:** `int` değer program boyunca aynı kalır; fakat okunabilir enum değerini kullanırsanız program daha kolay okunur. Zaten enum bir Java class'ı gibi type olduğu ve primitive `int` olmadığı için `int` ile enum değerini doğrudan karşılaştıramazsınız.

```
if (Season.SUMMER == 2) {} // DOES NOT COMPILE
```

Calling the valueOf() Method

English: Another useful feature is retrieving an enum value from a `String` using the `valueOf()` method. This is helpful when working with older code or parsing user input. The `String` passed in must match the enum value exactly, though. **Türkçe:** Başka bir yararlı özellik `valueOf()` method'uyla `String`'den enum değeri elde etmektir. Bu, eski kodla çalışırken veya kullanıcı girdisini parse ederken yararlıdır. Ancak verilen `String`, enum değeriyle tam eşleşmelidir.

```
Season s = Season.valueOf("SUMMER"); // SUMMER  
Season t = Season.valueOf("summer"); // IllegalArgumentException
```

English: The first statement works and assigns the proper enum value to `s`. Note that this line is not creating an enum value, at least not directly. Each enum value is created once when the enum is first loaded. Once the enum has been loaded, it retrieves the single enum value with the matching name.

Türkçe: İlk statement çalışır ve uygun enum değerini `s`'ye atar. Bu satırın en azından doğrudan yeni enum değeri oluşturmadığına dikkat edin. Her enum değeri enum ilk yüklendiğinde bir kez oluşturulur. Enum yüklendikten sonra eşleşen ada sahip tek enum değeri alınır.

English: The second statement encounters a problem. There is no enum value with the lowercase name `summer`. Java throws up its hands in defeat and throws an `IllegalArgumentException`. **Türkçe:** İkinci statement sorunla karşılaşır. Küçük harfli `summer` adına sahip enum değeri yoktur. Java pes eder ve `IllegalArgumentException` fırlatır.

```
Exception in thread "main" java.lang.IllegalArgumentException:
No enum constant enums.Season.summer
```

Using Enums in switch Statements

English: Enums can be used in switch statements and expressions. Pay attention to the case values in this code: **Türkçe:** Enum'lar switch statement ve expression içinde kullanılabilir. Aşağıdaki koddaki case değerlerine dikkat edin:

```
Season summer = Season.SUMMER;
switch (summer) {
    case WINTER:
        System.out.print("Get out the sled!");
        break;
    case SUMMER:
        System.out.print("Time for the pool!");
        break;
    default:
        System.out.print("Is it summer yet?");
}
```

English: The code prints `Time for the pool!` since it matches `SUMMER`. In each case statement, we typed the value of the enum rather than writing `Season.WINTER`. After all, the compiler already knows that the only possible matches can be enum values. Java treats the enum type as implicit. In fact, if you were to type `case Season.WINTER`, it would not compile. Don't believe us? Take a look at this equivalent example using a switch expression: **Türkçe:** Kod `SUMMER` ile eşleştiği için `Time for the pool!` yazdırır. Her case statement'ta `Season.WINTER` yerine yalnız enum değerini yazdık. Compiler olası eşleşmelerin yalnız enum değerleri olabileceğini zaten bilir; Java enum type'ını implicit kabul eder. Gerçekten de `case Season.WINTER` yazılsaydı derlenmezdi. Aşağıdaki eşdeğer switch expression örneğine bakın:

```
Season summer = Season.SUMMER;
var message = switch (summer) {
    case Season.WINTER -> "Get out the sled!"; // DOES NOT COMPILE
```

```
    case 0 -> "Time for the pool!"; // DOES NOT COMPILE
    default -> "Is it summer yet?";
};
System.out.print(message);
```

English: The first case statement does not compile because `Season` is used in the case value. If we changed `Season.FALL` to just `FALL`, then the line would compile. What about the second case statement? Just as earlier we said that you can't compare enums with int values, you cannot use them in a switch statement with int values. On the exam, pay special attention when working with enums that they are used only as enums. **Türkçe:** İlk case statement, case değerinde `Season` kullanıldığı için derlenmez. Kaynak metne göre `Season.FALL` yalnız `FALL` olarak değiştirilirse satır derlenir. Peki ikinci case statement ne olacak? Daha önce enum'ları `int` değerlerle karşılaştıramayacağınızı söylediğimiz gibi onları `int` değerlerle aynı switch statement içinde de kullanamazsınız. Sınavda enum'larla çalışırken bunların yalnızca enum olarak kullanılmasına özellikle dikkat edin.

OCP teknik düzeltme: Kaynak paragraftaki `Season.FALL`, hemen üstteki code snippet ile uyuşmayan bir yazım hatasıdır. Kodda bulunan ve düzeltilmesi gereken label `case Season.WINTER` olduğundan doğru değişiklik `case WINTER`'dir.

Adding Constructors, Fields, and Methods

English: While a simple enum is composed of just a list of values, we can define a complex enum with additional elements. Let's say our zoo wants to keep track of traffic patterns to determine which seasons get the most visitors. **Türkçe:** Simple enum yalnız değer listesinden oluşurken ek element'lar içeren complex enum da tanımlayabiliriz. Hayvanat bahçemizin hangi mevsimlerin en çok ziyaretçi aldığını belirlemek için trafik örüntülerini izlemek istediğini varsayalım.

```
public enum Season {
    WINTER("Low"), SPRING("Medium"), SUMMER("High"), FALL("Medium");

    private final String expectedVisitors;

    private Season(String expectedVisitors) {
        this.expectedVisitors = expectedVisitors;
    }

    public void printExpectedVisitors() {
        System.out.println(expectedVisitors);
    }
}
```

English: There are a few things to notice here. On line 2, the list of enum values ends with a semicolon. While this is optional when our enum is composed solely of a list of values, it is required if there is anything in the enum besides the values. **Türkçe:** Burada dikkat edilecek birkaç nokta vardır. Line 2'de enum değer listesi semicolon ile biter. Enum yalnız değer listesinden oluşuyorsa bu optional'dır; fakat enum içinde değerlerden başka bir şey varsa zorunludur.

English: Lines 3–9 are regular Java code. We have an instance variable, a constructor, and a method. We mark the instance variable private and final on line 3 so that our enum properties cannot be modified. **Türkçe:** Lines 3–9 normal Java kodudur. Bir instance variable, constructor ve method vardır. Enum property'lerinin değiştirilememesi için line 3'te instance variable'ı private ve final işaretleriz.

English: Although it is possible to create an enum with instance variables that can be modified, it is a very poor practice to do so since they are shared within the JVM. When designing an enum, the values should be immutable. **Türkçe:** Değiştirilebilen instance variable'lar içeren bir enum oluşturmak mümkün olsa da bunlar JVM içinde paylaşıldığından bunu yapmak son derece kötü bir pratiktir. Enum tasarlarken değerler immutable olmalıdır.

English: All enum constructors are implicitly private, with the modifier being optional. This is reasonable since you can't extend an enum and the constructors can be called only within the enum itself. In fact, an enum constructor will not compile if it contains a public or protected modifier. **Türkçe:** Bütün enum constructor'ları implicit private'tır ve modifier optional'dır. Enum extend edilemediği ve constructor yalnız enum içinden çağrılabilirdiği için bu mantıklıdır. Enum constructor public veya protected modifier içerirse derlenmez.

English: What about the parentheses on line 2? Those are constructor calls, but without the `new` keyword normally used for objects. The first time we ask for any of the enum values, Java constructs all of the enum values. After that, Java just returns the already constructed enum values. Given that explanation, you can see why this calls the constructor only once: **Türkçe:** Line 2'deki parentheses nedir? Bunlar constructor call'dur; ancak object'lerde normalde kullanılan `new` keyword'ü yoktur. Enum değerlerinden herhangi biri ilk istendiğinde Java bütün enum değerlerini oluşturur. Daha sonra yalnız önceden oluşturulmuş enum değerlerini döndürür. Bu nedenle aşağıdaki örnek constructor'ı yalnız bir kez çağırır:

```
public enum OnlyOne {
    ONCE(true);

    private OnlyOne(boolean b) {
        System.out.print("constructing,");
    }
}

public class PrintTheOne {
    public static void main(String[] args) {
        System.out.print("begin,");
        OnlyOne firstCall = OnlyOne.ONCE; // Prints constructing,
        OnlyOne secondCall = OnlyOne.ONCE; // Doesn't print anything
        System.out.print("end");
    }
}
```

English: This class prints the following: **Türkçe:** Bu class aşağıdakini yazdırır:

```
begin,constructing,end
```

English: If the `OnlyOne` enum was used earlier in the program, and therefore initialized sooner, then the line that declares the `firstCall` variable would not print anything. **Türkçe:** `OnlyOne` enum programda daha önce kullanılmış ve bu nedenle daha önce initialize edilmişse `firstCall` variable'ını bildiren satır hiçbir şey yazdırmazdı.

English: How do we call an enum method? That's easy, too: we just use the enum value followed by the method call. **Türkçe:** Enum method'u nasıl çağırılır? Bu da kolaydır: enum değerinin ardından method call yazılır.

```
Season.SUMMER.printExpectedVisitors();
```

English: Sometimes you want to define different methods for each enum. For example, our zoo has different seasonal hours. It is cold and gets dark early in the winter. We can keep track of the hours through instance variables, or we can let each enum value manage hours itself. **Türkçe:** Bazen her enum için farklı method tanımlamak istersiniz. Örneğin hayvanat bahçemizin mevsime göre farklı saatleri vardır; kışın hava soğuk ve erken karanlık olur. Saatleri instance variable ile izleyebilir veya her enum değerinin kendi saatini yönetmesini sağlayabiliriz.

```
public enum Season {  
    WINTER {  
        public String getHours() { return "10am-3pm"; }  
    },  
    SPRING {  
        public String getHours() { return "9am-5pm"; }  
    },  
    SUMMER {  
        public String getHours() { return "9am-7pm"; }  
    },  

```

```
    FALL {  
        public String getHours() { return "9am-5pm"; }  
    };  
  
    public abstract String getHours();  
}
```

English: What's going on here? It looks like we created an abstract class and a bunch of tiny subclasses. In a way, we did. The enum itself has an abstract method. This means that each and every enum value is required to implement this method. If we forget to implement the method for one of the values, we get a compiler error: 'The enum constant WINTER must implement the abstract method getHours()'. **Türkçe:** Burada ne oluyor? Abstract class ve çok sayıda küçük subclass oluşturmuşuz gibi görünür; bir bakıma öyledir. Enum'un kendisinde abstract method vardır. Bu, her enum değerinin bu method'u implement etmesini zorunlu kılar. Değerlerden biri için method'u unutursak şu compiler error alınır: `The enum constant WINTER must implement the abstract method getHours()`.

English: But what if we don't want each and every enum value to have a method? No problem. We can create an implementation for all values and override it only for the special cases. **Türkçe:** Her enum değerinin ayrı method'da sahip olmasını istemiyorsak sorun yoktur. Bütün değerler için ortak implementation oluşturup yalnız özel durumlarda override edebiliriz.

```
public enum Season {
    WINTER {
        public String getHours() { return "10am-3pm"; }
    },
    SUMMER {
        public String getHours() { return "9am-7pm"; }
    },
    SPRING, FALL;

    public String getHours() { return "9am-5pm"; }
}
```

English: This looks better. We only code the special cases and let the others use the enum-provided implementation. **Türkçe:** Bu daha iyi görünür. Yalnız özel durumları kodlar, diğerlerinin enum tarafından sağlanan implementation'ı kullanmasına izin veririz.

English: An enum can even implement an interface, as this just requires overriding the abstract methods: **Türkçe:** Yalnız abstract method'ları override etmeyi gerektirdiğinden enum bir interface'i bile implement edebilir:

```
public interface Weather {
    int getAverageTemperature();
}

public enum Season implements Weather {
    WINTER, SPRING, SUMMER, FALL;

    public int getAverageTemperature() { return 30; }
}
```

English: Just because an enum can have lots of methods doesn't mean that it should. Try to keep your enums simple. If your enum is more than a page or two, it is probably too long. When enums get too long or too complex, they are hard to read. **Türkçe:** Enum çok sayıda method içerebiliyor diye içermesi gerekmez. Enum'larınızı basit tutmaya çalışın. Enum bir iki sayfadan uzunsa muhtemelen fazlasıyla uzundur. Enum'lar uzadıkça veya karmaşıktıkça okunmaları zorlaşır.

English: You might have noticed that in each of these enum examples, the list of values came first. This was not an accident. Whether the enum is simple or complex, the list of values always comes first.

Türkçe: Bu enum örneklerinin hepsinde değer listesinin önce geldiğini fark etmiş olabilirsiniz. Bu tesadüf değildir. Enum simple veya complex olsun, değer listesi her zaman önce gelir.

Sealing Classes

English: An enum with many constructors, fields, and methods may start to resemble a full-featured class. What if we could create a class but limit the direct subclasses to a fixed set of classes? Enter sealed classes! A sealed class is a class that restricts which other classes may directly extend it. These are brand new to Java 17, so expect to see at least one question about them on the exam. **Türkçe:** Çok sayıda constructor, field ve method içeren enum tam özellikli class'a benzemeye başlayabilir. Bir class oluşturup direct subclass'larını sabit bir class kümesiyle sınırlayabilsek nasıl olurdu? Sealed class'lar tam bunu sağlar. Sealed class, hangi diğer class'ların onu doğrudan extend edebileceğini sınırlar. Bunlar Java 17 için yenidir; dolayısıyla sınavda en az bir sealed class sorusu bekleyin.

English: Did you happen to notice that we said directly extend in the definition of a sealed class? As you see shortly, there is a way for a class not named in the sealed class declaration to extend it indirectly. Unless we say otherwise, though, assume that we're referring to subclasses that directly extend the sealed class. **Türkçe:** Sealed class tanımında "doğrudan extend" dediğimizi fark ettiniz mi? Birazdan sealed class declaration'da adı bulunmayan bir class'ın onu dolaylı extend edebildiğini göreceksiniz. Aksi belirtilmedikçe sealed class'ı doğrudan extend eden subclass'lardan söz ettiğimizi varsayın.

Declaring a Sealed Class

English: Let's start with a simple example. A sealed class declares a list of classes that can extend it, while the subclasses declare that they extend the sealed class. Figure 7.5 declares a sealed class with two direct subclasses. **Türkçe:** Basit bir örnekle başlayalım. Sealed class kendisini extend edebilen class'ların listesini bildirirken subclass'lar sealed class'ı extend ettiklerini bildirir. Figure 7.5 iki direct subclass'a sahip sealed class gösterir.

Figure 7.5 · Defining a sealed class

English — figure callouts: `sealed` keyword; list of permitted classes; `class` keyword; `permits` keyword; `final`, `sealed`, or `non-sealed` subclass modifier. **Türkçe — şekil çağrılar:** `sealed` keyword'ü; permitted class listesi; `class` keyword'ü; `permits` keyword'ü; subclass modifier olarak `final`, `sealed` veya `non-sealed`.

```
public sealed class Bear permits Kodiak, Panda {}
public final class Kodiak extends Bear {}
public non-sealed class Panda extends Bear {}
```

English — figure callout: Extends sealed class. **Türkçe — şekil çağrısı:** Sealed class'ı extend eder.

English: Notice anything new? Java 17 includes three new keywords that you should be familiar with for the exam. We often use `final` with sealed subclasses, but we get into each of these after we cover the basics. **Türkçe:** Yeni bir şey fark ettiniz mi? Java 17, sınav için aşına olmanız gereken üç yeni keyword içerir. Sealed subclass'larda sık sık `final` kullanırız; ancak temellerden sonra her birini ayrı ayrı ele alacağız.

Sealed Class Keywords

English: `sealed`: Indicates that a class or interface may be extended or implemented only by named classes or interfaces. **Türkçe:** `sealed`: Bir class veya interface'in yalnız adı belirtilen class veya interface'ler tarafından extend/implement edilebileceğini gösterir.

English: `permits`: Used with the `sealed` keyword to list the classes and interfaces allowed. `non-sealed`: Applied to a class or interface that extends a sealed class, indicating that it can be extended by unspecified classes. **Türkçe:** `permits`: İzin verilen class ve interface'leri listelemek için `sealed` keyword'üyle birlikte kullanılır. `non-sealed`: Sealed class'ı extend eden class veya interface'e uygulanır ve adı belirtilmeyen class'lar tarafından da extend edilebileceğini gösterir.

English: Pretty easy so far, right? The exam is just as likely to test you on what sealed classes cannot be used for. For example, can you see why each of these sets of declarations does not compile? **Türkçe:** Şimdiye kadar oldukça kolay, değil mi? Sınav sealed class'ların hangi durumlarda kullanılamayacağını da aynı olasılıkla sorabilir. Örneğin aşağıdaki declaration kümelerinin neden derlenmediğini görebiliyor musunuz?

```
public class sealed Frog permits GlassFrog {} // DOES NOT COMPILE
public final class GlassFrog extends Frog {}

public abstract sealed class Wolf permits Timber {}
public final class Timber extends Wolf {}
public final class MyWolf extends Wolf {} // DOES NOT COMPILE
```

English: The first example does not compile because the `class` and `sealed` modifiers are in the wrong order. The modifier has to be before the class type. The second example does not compile because `MyWolf` isn't listed in the declaration of `Wolf`. **Türkçe:** İlk örnek `class` ile `sealed` yanlış sırada olduğu için derlenmez; modifier class type'tan önce gelmelidir. İkinci örnek `MyWolf`, `Wolf` declaration'ında listelenmediği için derlenmez.

English: Sealed classes are commonly declared with the `abstract` modifier, although this is certainly not required. **Türkçe:** Zorunlu olmasa da sealed class'lar yaygın olarak `abstract` modifier ile bildirilir.

English: Declaring a sealed class with the `sealed` modifier is the easy part. Most of the time, if you see a question on the exam about sealed classes, they are testing your knowledge of whether the subclass extends the sealed class properly. There are a number of important rules you need to know for the exam, so read the next sections carefully. **Türkçe:** Sealed class'ı `sealed` modifier ile bildirmek kolay kısımdır. Sınavdaki sealed class soruları çoğunlukla subclass'ın sealed class'ı doğru biçimde extend edip etmediğini ölçer. Bilmeniz gereken çok sayıda önemli kural vardır; sonraki bölümleri dikkatle okuyun.

Compiling Sealed Classes

English: Let's say we create a `Penguin` class and compile it in a new package without any other source code. With that in mind, does the following compile? **Türkçe:** Başka source code bulunmayan yeni bir package'ta `Penguin` class'ı oluşturup derlediğimizi varsayalım. Aşağıdaki kod derlenir mi?

```
// Penguin.java
package zoo;
public sealed class Penguin permits Emperor {}
```

English: No, it does not! Why? The answer is that a sealed class needs to be declared and compiled in the same package as its direct subclasses. But what about the subclasses themselves? They must each extend the sealed class. For example, the following does not compile: **Türkçe:** Hayır, derlenmez. Sealed class direct subclass'larıyla aynı package'ta bildirilmeli ve derlenmelidir. Subclass'ların her biri de sealed class'ı extend etmelidir. Örneğin aşağıdaki kod derlenmez:

```
// Penguin.java
package zoo;
public sealed class Penguin permits Emperor {} // DOES NOT COMPILE
```

```
// Emperor.java
package zoo;
public final class Emperor {}
```

English: Even though the `Emperor` class is declared, it does not extend the `Penguin` class. **Türkçe:** `Emperor` class'ı bildirilmiş olsa da `Penguin` class'ını extend etmez.

English: But wait, there's more! In Chapter 12, "Modules," you learn about named modules, which allow sealed classes and their direct subclasses in different packages, provided they are in the same named module. **Türkçe:** Dahası da vardır. Chapter 12, "Modules" bölümünde, aynı named module içinde olmaları koşuluyla sealed class ve direct subclass'larının farklı package'larda bulunmasına izin veren named module'ları öğreneceksiniz.

Specifying the Subclass Modifier

English: While some types, like interfaces, have a certain number of implicit modifiers, sealed classes do not. Every class that directly extends a sealed class must specify exactly one of the following three modifiers: `final`, `sealed`, or `non-sealed`. Remember this rule for the exam! **Türkçe:** Interface gibi bazı type'ların implicit modifier'ları vardır; sealed class'larda ise yoktur. Sealed class'ı doğrudan extend eden her class şu üç modifier'dan tam birini belirtmelidir: `final`, `sealed` veya `non-sealed`. Bu kuralı sınav için unutmayın.

A final Subclass

English: The first modifier we're going to look at that can be applied to a direct subclass of a sealed class is the `final` modifier. **Türkçe:** Sealed class'ın direct subclass'ına uygulanabilen ilk modifier `final` modifier'dır.

```
public sealed class Antelope permits Gazelle {}
public final class Gazelle extends Antelope {}
public class George extends Gazelle {} // DOES NOT COMPILE
```

English: Just as with a regular class, the final modifier prevents the subclass `Gazelle` from being extended further. **Türkçe:** Normal class'ta olduğu gibi final modifier `Gazelle` subclass'ının daha fazla extend edilmesini engeller.

A sealed Subclass

English: Next, let's look at an example using the sealed modifier. **Türkçe:** Şimdi sealed modifier kullanan bir örneğe bakalım:

```
public sealed class Mammal permits Equine {}  
public sealed class Equine extends Mammal permits Zebra {}  
public final class Zebra extends Equine {}
```

English: The sealed modifier applied to the subclass `Equine` means the same kind of rules that we applied to the parent class `Mammal` must be present. Namely, `Equine` defines its own list of permitted subclasses. Notice in this example that `Zebra` is an indirect subclass of `Mammal` but is not named in the `Mammal` class. **Türkçe:** `Equine` subclass'ına uygulanan sealed modifier, parent `Mammal` class'ına uygulanan kuralların aynısının bulunması gerektiğini gösterir. `Equine` kendi permitted subclass listesini tanımlar. `Zebra` bu örnekte `Mammal`'ın indirect subclass'ıdır, fakat `Mammal` class'ında adı geçmez.

English: Despite allowing indirect subclasses not named in `Mammal`, the list of classes that can inherit `Mammal` is still fixed. If you have a reference to a `Mammal` object, it must be a `Mammal`, `Equine`, or `Zebra`. **Türkçe:** `Mammal` içinde adı geçmeyen indirect subclass'lara izin verilse de `Mammal`'ı inherit edebilen class listesi hâlâ sabittir. Bir `Mammal` object reference'nız varsa object `Mammal`, `Equine` veya `Zebra` olmalıdır.

A non-sealed Subclass

English: The `non-sealed` modifier is used to open a sealed parent class to potentially unknown subclasses. Let's modify our earlier example to allow `MyWolf` to compile without modifying the declaration of `Wolf`. **Türkçe:** `non-sealed` modifier, sealed parent class'ı potansiyel olarak bilinmeyen subclass'lara açmak için kullanılır. Önceki örneği `Wolf` declaration'ını değiştirmeden `MyWolf` derlenecek şekilde değiştirelim:

```
public sealed class Wolf permits Timber {}  
public non-sealed class Timber extends Wolf {}  
public class MyWolf extends Timber {}
```

English: In this example, we are able to create an indirect subclass of `Wolf`, called `MyWolf`, not named in the declaration of `Wolf`. Also notice that `MyWolf` is not final, so it may be extended by any subclass, such as `MyFurryWolf`. **Türkçe:** Bu örnekte `Wolf` declaration'ında adı geçmeyen `MyWolf` adlı indirect subclass oluşturabiliriz. `MyWolf` final olmadığı için `MyFurryWolf` gibi herhangi bir subclass tarafından extend edilebilir.

```
public class MyFurryWolf extends MyWolf {}
```

English: At first glance, this might seem counterintuitive. After all, we were able to create subclasses of `Wolf` that were not declared in `Wolf`. So is `Wolf` still sealed? Yes, but that's thanks to polymorphism. Any instance of `MyWolf` or `MyFurryWolf` is also an instance of `Timber`, which is named in the `Wolf` declaration. We discuss polymorphism more toward the end of this chapter. **Türkçe:** İlk bakışta bu ters gelebilir; çünkü `Wolf` içinde bildirilmeyen subclass'lar oluşturduk. Peki `Wolf` hâlâ sealed mıdır? Evet; polymorphism sayesinde `MyWolf` veya `MyFurryWolf` instance'larının her biri, `Wolf` declaration'ında adı geçen bir `Timber` instance'dır. Polymorphism'i bölümün sonuna doğru daha ayrıntılı ele alacağız.

English: If you're still worried about opening a sealed class too much with a non-sealed subclass, remember that the person writing the sealed class can see the declaration of all direct subclasses at compile time. They can decide whether to allow the non-sealed subclass to be supported. **Türkçe:** Non-sealed subclass ile sealed class'ı fazla açmaktan endişe ediyorsanız sealed class'ı yazan kişinin bütün direct subclass declaration'larını compile time'da görebildiğini unutmayın. Non-sealed subclass'a izin verip vermemeye karar verebilir.

Omitting the permits Clause

English: Up until now, all of the examples you've seen have required a permits clause when declaring a sealed class, but this is not always the case. Imagine that you have a `Snake.java` file with two top-level classes defined inside it: **Türkçe:** Şimdiye kadarki örneklerin tamamında sealed class declaration için permits clause gerekiyordu; ancak her zaman böyle değildir. İçinde iki top-level class tanımlanan bir `Snake.java` dosyası düşünün:

```
// Snake.java
public sealed class Snake permits Cobra {}
final class Cobra extends Snake {}
```

English: In this case, the permits clause is optional and can be omitted. The `extends` keyword is still required in the subclass, though: **Türkçe:** Bu durumda permits clause optional'dır ve çıkarılabilir. Ancak subclass'ta `extends` keyword'ü hâlâ zorunludur:

```
// Snake.java
public sealed class Snake {}
final class Cobra extends Snake {}
```

English: If these classes were in separate files, this code would not compile! This rule also applies to sealed classes with nested subclasses. **Türkçe:** Bu class'lar ayrı dosyalarda olsaydı kod derlenmezdi. Aynı kural nested subclass içeren sealed class'lara da uygulanır.

```
// Snake.java
public sealed class Snake {
    final class Cobra extends Snake {}
}
```

Referencing Nested Subclasses

English: While it makes the code easier to read if you omit the permits clause for nested subclasses, you are welcome to name them. However, the syntax might be different than you expect. **Türkçe:** Nested subclass'larda permits clause'u çıkarmak kodu daha okunur kılsa da bunların adını yazabilirsiniz. Ancak syntax beklediğinizden farklı olabilir.

```
public sealed class Snake permits Cobra { // DOES NOT COMPILE
    final class Cobra extends Snake {}
}
```

English: This code does not compile because `Cobra` requires a reference to the `Snake` namespace. The following fixes this issue: **Türkçe:** `Cobra`, `Snake` namespace'ine referans gerektirdiği için bu kod derlenmez. Aşağıdaki kod sorunu düzeltir:

```
public sealed class Snake permits Snake.Cobra {
    final class Cobra extends Snake {}
}
```

English: When all of your subclasses are nested, we strongly recommend omitting the permits clause. We cover nested classes shortly. For now, you just need to know that a nested class is a class defined inside another class and that the omit rule also applies to nested classes. Table 7.3 is a handy reference to these cases. **Türkçe:** Bütün subclass'lar nested ise permits clause'u çıkarmanızı güçlü biçimde öneririz. Nested class'ları birazdan ele alacağız. Şimdilik nested class'ın başka bir class içinde tanımlanan class olduğunu ve omission kuralının nested class'lara da uygulandığını bilmeniz yeterlidir. Table 7.3 bu durumlar için kullanışlı bir referanstır.

Table 7.3 · Usage of the permits clause in sealed classes

Location of direct subclasses	permits clause
In a different file from the sealed class	Required
In the same file as the sealed class	Permitted, but not required
Nested inside the sealed class	Permitted, but not required

Türkçe tablo açıklaması: Direct subclass sealed class'tan farklı dosyada ise permits clause zorunludur. Aynı dosyada veya sealed class içinde nested ise kullanılabilir fakat zorunlu değildir.

Sealing Interfaces

English: Besides classes, interfaces can also be sealed. The idea is analogous to classes, and many of the same rules apply. For example, the sealed interface must appear in the same package or named module as the classes or interfaces that directly extend or implement it. **Türkçe:** Class'ların yanında interface'ler de sealed olabilir. Düşünce class'lara benzer ve aynı kuralların çoğu uygulanır. Örneğin sealed interface, onu doğrudan extend veya implement eden class/interface'lerle aynı package veya named module içinde bulunmalıdır.

English: One distinct feature of a sealed interface is that the permits list can apply to a class that implements the interface or an interface that extends the interface. **Türkçe:** Sealed interface'in ayırt edici özelliği permits listesinin, interface'i implement eden class'a veya interface'i extend eden başka bir interface'e uygulanabilmesidir.

```
// Sealed interface
public sealed interface Swims permits Duck, Swan, Floats {}

// Classes permitted to implement sealed interface
public final class Duck implements Swims {}
public final class Swan implements Swims {}

// Interface permitted to extend sealed interface
public non-sealed interface Floats extends Swims {}
```

English: What about the modifier applied to interfaces that extend the sealed interface? Remember that interfaces are implicitly abstract and cannot be marked final. For this reason, interfaces that extend a sealed interface can be marked only sealed or non-sealed. They cannot be marked final. **Türkçe:** Sealed interface'i extend eden interface'lere hangi modifier uygulanır? Interface'lerin implicit abstract olduğunu ve final işaretlenemediğini unutmayın. Bu nedenle sealed interface'i extend eden interface yalnız sealed veya non-sealed olabilir; final olamaz.

Reviewing Sealed Class Rules

English: Any time you see a sealed class on the exam, pay close attention to the subclass declaration and modifiers. **Türkçe:** Sınavda sealed class gördüğünüz her zaman subclass declaration ve modifier'lara dikkatle bakın.

Sealed Class Rules

English: Sealed classes are declared with the sealed and permits modifiers. Sealed classes must be declared in the same package or named module as their direct subclasses. **Türkçe:** Sealed class'lar sealed ve permits modifier'larıyla bildirilir. Direct subclass'larıyla aynı package veya named module içinde bulunmalıdır.

English: Direct subclasses of sealed classes must be marked final, sealed, or non-sealed. The permits clause is optional if the sealed class and its direct subclasses are declared within the same file or the subclasses are nested within the sealed class. Interfaces can be sealed to limit the classes that implement them or the interfaces that extend them. **Türkçe:** Sealed class'ların direct subclass'ları final, sealed veya non-sealed işaretlenmelidir. Sealed class ile direct subclass'ları aynı dosyada bildirilmişse veya subclass'lar sealed class içinde nested ise permits clause optional'dır. Interface'ler de onları implement eden class'ları veya extend eden interface'leri sınırlamak için sealed yapılabilir.

Why Have Sealed Classes?

English: In Chapter 3, “Making Decisions,” you learned about switch expressions and pattern matching. Imagine if we could treat a sealed class like an enum in a switch expression by applying pattern matching. Given a sealed class `Fish` with two direct subclasses, it might look something like this: **Türkçe:** Chapter 3, “Making Decisions” bölümünde switch expression ve pattern matching öğrendiniz. Pattern matching uygulayarak sealed class'ı switch expression içinde enum gibi ele alabildiğimizi düşünün. İki direct subclass'a sahip sealed `Fish` class'ı şöyle görünebilir:

```
public void printName(Fish fish) {
    System.out.println(switch (fish) {
        case Trout t -> t.getTroutName();
        case Bass b -> b.getBassName();
    });
}
```

English: If `Fish` wasn't sealed, the switch expression would require a default branch, or the code would not compile. Since it's sealed, the compiler knows all the options. The good news is that this feature is on the way, but the bad news is that it's still in Preview in Java 17 and not officially released. We just wanted to give you an idea of where some of these new features were heading. **Türkçe:** `Fish` sealed olmasaydı switch expression default branch gerektirirdi; aksi halde kod derlenmezdi. Sealed olduğu için compiler bütün seçenekleri bilir. İyi haber bu özelliğin gelmekte olmasıdır; kötü haber ise Java 17'de hâlâ Preview olması ve resmen yayımlanmamış olmasıdır. Burada yalnız yeni özelliklerin hangi yöne ilerlediğine ilişkin fikir vermek amaçlanmıştır.

OCP/Java 17 notu: Bu pattern `switch` örneği Java 17'nin standart, preview'siz derlemesinde kullanılamaz; kaynak da bunu açıkça Preview olarak işaretler.

Encapsulating Data with Records

English: We saved the best new Java type for last! If you've heard anything about the new features in Java, you have probably heard about records. Records are exciting because they remove a ton of boilerplate code. Before we get into records, it helps to have some context for why they were added to the language, so we start with encapsulation. **Türkçe:** En iyi yeni Java type'ını sona sakladık. Java'nın yeni özellikleri hakkında bir şey duyduysanız muhtemelen record'ları duymuşsunuzdur. Record'lar çok miktarda boilerplate code'u kaldırdıkları için heyecan vericidir. Neden dile eklendiklerini anlamak yararlı olacağından önce encapsulation ile başlıyoruz.

Understanding Encapsulation

English: A POJO, which stands for Plain Old Java Object, is a class used to model and pass data around, often with few or no complex methods—hence the “plain” part of the definition. You might also have heard of a JavaBean, which is a POJO that has some additional rules applied. **Türkçe:** Plain Old Java Object'ın kısaltması olan POJO, çoğunlukla az sayıda complex method içeren veya hiç içermeyen, veriyi modellemek ve taşımak için kullanılan class'tır; tanımdaki “plain” buradan gelir. Ek kurallar uygulanan bir POJO olan JavaBean'i de duymuş olabilirsiniz.

English: Let's create a simple POJO with two fields: **Türkçe:** İki field içeren basit bir POJO oluşturalım:

```
public class Crane {  
    int numberEggs;  
    String name;  
  
    public Crane(int numberEggs, String name) {  
        this.numberEggs = numberEggs;  
        this.name = name;  
    }  
}
```

English: Uh oh, the fields are package access. Why do we care? That means someone outside the class in the same package could change these values and create invalid data such as this: **Türkçe:** Field'lar package access; bu neden önemlidir? Aynı package'ta fakat class dışında bulunan biri bu değerleri değiştirerek aşağıdaki gibi geçersiz veri oluşturabilir:

```
public class Poacher {  
    public void badActor() {  
        var mother = new Crane(5, "Cathy");  
        mother.numberEggs = -100;  
    }  
}
```

English: This is clearly no good. We do not want the mother *Crane* to have a negative number of eggs! Encapsulation to the rescue. Encapsulation is a way to protect class members by restricting access to them. In Java, it is commonly implemented by declaring all instance variables private. Callers are required to use methods to retrieve or modify instance variables. **Türkçe:** Bu açıkça iyi değildir; anne *Crane*'in negatif sayıda yumurtası olmasını istemeyiz. Encapsulation imdada yetişir. Encapsulation, class member'larına erişimi sınırlayarak onları koruma yöntemidir. Java'da genellikle bütün instance variable'lar private bildirilerek uygulanır. Caller'lar instance variable'ları almak veya değiştirmek için method kullanmak zorundadır.

English: Encapsulation is about protecting a class from unexpected use. It also allows us to modify the methods and behavior of the class later without someone already having direct access to an instance variable within the class. For example, we can change the data type of an instance variable but maintain the same method signatures. In this manner, we maintain full control over the internal workings of a class. **Türkçe:** Encapsulation class'ı beklenmeyen kullanımdan korur. Ayrıca birinin class içindeki instance variable'a doğrudan erişimi olmadan, daha sonra class'ın method ve davranışlarını değiştirmemizi sağlar. Örneğin aynı method signature'ları koruyup instance variable'ın data type'ını değiştirebiliriz. Böylece class'ın iç işleyişi üzerinde tam kontrolü sürdürürüz.

English: Let's take a look at the newly encapsulated and immutable *Crane* class: **Türkçe:** Yeni encapsulated ve immutable *Crane* class'ına bakalım:

```
public final class Crane {
    private final int numberEggs;
    private final String name;

    public Crane(int numberEggs, String name) {
        if (numberEggs >= 0)
            this.numberEggs = numberEggs; // guard condition
        else
            throw new IllegalArgumentException();
        this.name = name;
    }

    public int getNumberEggs() { // getter
        return numberEggs;
    }

    public String getName() { // getter
        return name;
    }
}
```

English: Note that the instance variables are now private on lines 2 and 3. This means only code within the class can read or write their values. Since we wrote the class, we know better than to set a negative number of eggs. We added a method on lines 9–11 to read the value, which is called an accessor method or a getter. **Türkçe:** Instance variable'ların artık lines 2 ve 3'te private olduğuna dikkat edin. Bu, değerleri yalnız class içindeki kodun okuyup yazabileceği anlamına gelir. Class'ı biz yazdığımız için negatif yumurta sayısı atamamamız gerektiğini biliriz. Değeri okumak için lines 9–11'e accessor method, diğer adıyla getter, ekledik.

English: You might have noticed that we marked the class and its instance variables final, and we don't have any mutator methods, or setters, to modify the value of the instance variables. That's because we want our class to be immutable in addition to being well encapsulated. As you saw in Chapter 6, the immutable objects pattern is an object-oriented design pattern in which an object cannot be modified after it is created. Instead of modifying an immutable object, you create a new object that contains any properties from the original object you want copied over. **Türkçe:** Class ve instance variable'ları final işaretlediğimizi, instance variable değerlerini değiştirecek mutator method, yani setter bulunmadığını fark etmiş olabilirsiniz. Çünkü class'ın iyi encapsulated olmasının yanında immutable da olmasını istiyoruz. Chapter 6'da gördüğümüz immutable objects pattern, object oluşturulduktan sonra değiştirilemediği object-oriented design pattern'dir. Immutable object'i değiştirmek yerine original object'ten kopyalamak istediğiniz property'leri içeren yeni object oluşturursunuz.

English: To review, remember that data—an instance variable—is private and getters/setters are public for encapsulation. You don't even have to provide getters and setters. As long as the instance variables are private, you are good. For example, the following class is well encapsulated, although it is not terribly useful since it doesn't declare any non-private methods: **Türkçe:** Tekrar için encapsulation'da data'nın, yani instance variable'ın private; getter/setter'ların public olduğunu hatırlayın. Getter ve setter sağlamak zorunda bile değilsiniz; instance variable'lar private olduğu sürece encapsulation sağlanır. Örneğin aşağıdaki class non-private method bildirmediğinden pek yararlı olmasa da iyi encapsulated'dır:

```
public class Vet {
    private String name = "Dr Rogers";
    private int yearsExperience = 25;
}
```

English: You must omit the setters for a class to be immutable. Review Chapter 6 for the additional rules on creating immutable objects. **Türkçe:** Class'ın immutable olması için setter'ları çıkarmalısınız. Immutable object oluşturmaya ilişkin ek kurallar için Chapter 6'yı tekrar edin.

Applying Records

English: Our `Crane` class was 15 lines long. We can write that much more succinctly, as shown in Figure 7.6. Putting aside the guard clause on `numberEggs` in the constructor for a moment, this record is equivalent and immutable! **Türkçe:** `Crane` class'ımız 15 satırdı. Figure 7.6'da gösterildiği gibi çok daha kısa yazabiliriz. Constructor'daki `numberEggs` guard clause'u şimdilik bir kenara bırakırsak bu record eşdeğer ve immutable'dır.

Figure 7.6 · Defining a record

English — figure callouts: `record` keyword; record name; list of fields surrounded by parentheses. **Türkçe — şekil çağrılar:** `record` keyword'ü; record adı; parentheses ile çevrili field listesi.

```
public record Crane(int numberEggs, String name) {}
```

English — figure callout: May declare optional constructors, methods, and constants. **Türkçe — şekil çağırısı:** Optional constructor, method ve constant bildirebilir.

English: Wow! It's only one line long! A record is a special type of data-oriented class in which the compiler inserts boilerplate code for you. **Türkçe:** Yalnız bir satır. Record, compiler'ın boilerplate code'u sizin için eklediği özel bir data-oriented class türüdür.

English: In fact, the compiler inserts much more than the 14 lines we wrote earlier. As a bonus, the compiler inserts useful implementations of the `Object` methods `equals()`, `hashCode()`, and `toString()`. We've covered a lot in one line of code! **Türkçe:** Compiler aslında daha önce yazdığımız 14 satırdan çok daha fazlasını ekler. Ek olarak `Object` class'ının `equals()`, `hashCode()` ve `toString()` method'larının kullanışlı implementation'larını ekler. Tek satır kodla çok şey elde ettik.

English: Now imagine that we had 10 data fields instead of 2. That's a lot of methods we are saved from writing. And we haven't even talked about constructors! Worse yet, any time someone changes a field, dozens of lines of related code may need to be updated. For example, `name` may be used in the constructor, `toString()`, `equals()` method, and so on. If we have an application with hundreds of POJOs, a record can save us valuable time. **Türkçe:** İki yerine on data field bulunduğunu düşünün; yazmaktan kurtulduğumuz çok sayıda method olur. Üstelik constructor'lardan henüz söz etmedik. Bir field değiştiğinde ilişkili düzinelerce kod satırının güncellenmesi gerekebilir. Örneğin `name`; constructor, `toString()`, `equals()` ve başka yerlerde kullanılabilir. Yüzlerce POJO içeren application'da record önemli ölçüde zaman kazandırabilir.

English: Creating an instance of a `Crane` and printing some fields is easy. **Türkçe:** `Crane` instance'ı oluşturmak ve bazı field'ları yazdırmak kolaydır:

```
var mommy = new Crane(4, "Cammy");
System.out.println(mommy.numberEggs()); // 4
System.out.println(mommy.name());      // Cammy
```

English: A few things should stand out here. First, we never defined any constructors or methods in our `Crane` declaration. How does the compiler know what to do? Behind the scenes, it creates a constructor for you with the parameters in the same order in which they appear in the record declaration. Omitting or changing the type order will lead to compiler errors. **Türkçe:** Burada birkaç nokta öne çıkmalıdır. `Crane` declaration içinde hiç constructor veya method tanımlamadık. Compiler ne yapacağını nasıl bilir? Arka planda record declaration'daki sırayla aynı parameter'lara sahip constructor oluşturur. Parameter çıkarmak veya type sırasını değiştirmek compiler error üretir:

```
var mommy1 = new Crane("Cammy", 4); // DOES NOT COMPILE
var mommy2 = new Crane("Cammy");    // DOES NOT COMPILE
```

English: For each field, it also creates an accessor as the field name plus a set of parentheses. Unlike traditional POJOs or JavaBeans, the methods don't have the prefix `get` or `is`. Just a few more characters that records save you from having to type! Finally, records override a number of methods in `Object` for you. **Türkçe:** Her field için field adı ve parentheses'ten oluşan accessor da oluşturur. Geleneksel POJO veya JavaBean'lerin aksine method'larda `get` ya da `is` prefix'i bulunmaz. Record birkaç karakter daha yazmaktan kurtarır. Son olarak `Object` içindeki bazı method'ları sizin için override eder.

Members Automatically Added to Records

English: Constructor: A constructor with the parameters in the same order as the record declaration. Accessor method: One accessor for each field. **Türkçe:** Constructor: Record declaration ile aynı sırada parameter içeren constructor. Accessor method: Her field için bir accessor.

English: `equals()`: A method to compare two elements that returns true if each field is equal in terms of `equals()`. `hashCode()`: A consistent `hashCode()` method using all of the fields. `toString()`: A `toString()` implementation that prints each field of the record in a convenient, easy-to-read format. **Türkçe:** `equals()`: İki element'i karşılaştırır ve her field `equals()` bakımından eşitse true döndürür. `hashCode()`: Bütün field'ları kullanan tutarlı `hashCode()` method'u. `toString()`: Record'un her field'ını kolay okunur biçimde yazdıran `toString()` implementation'ı.

English: The following shows examples of the new methods. Remember that the `println()` method will call the `toString()` method automatically on any object passed to it. **Türkçe:** Aşağıdaki kod yeni method'ların örneklerini gösterir. `println()` method'unun kendisine verilen her object'te `toString()` method'unu otomatik çağırdığını unutmayın.

```
var father = new Crane(0, "Craig");
System.out.println(father); // Crane[numberEggs=0, name=Craig]

var copy = new Crane(0, "Craig");
System.out.println(copy); // Crane[numberEggs=0, name=Craig]
System.out.println(father.equals(copy)); // true
System.out.println(father.hashCode() + ", " + copy.hashCode()); // 1007, 1007
```

English: That's the basics of records. We say “basics” because there's a lot more you can do with them, as you see in the next sections. **Türkçe:** Bunlar record'ların temelleridir. “Temeller” diyoruz; çünkü sonraki bölümlerde göreceğiniz gibi record'larla çok daha fazlası yapılabilir.

English: Given our one-line declaration of `Crane`, imagine how much code and work would be required to write an equivalent class. It could easily take more than 40 lines! It might be a fun exercise to try to write all the methods that records supply. **Türkçe:** Tek satırlık `Crane` declaration'a eşdeğer bir class yazmak için gereken kodu ve emeği düşünün. Kolayca 40 satırı aşabilir. Record'un sağladığı bütün method'ları yazmayı denemek eğlenceli bir alıştırmaya olabilir.

English: Fun fact: it is legal to have a record without any fields. It is simply declared with the record keyword and parentheses: **Türkçe:** İlginç bilgi: Hiç field içermeyen record geçerlidir. Yalnız record keyword'ü ve parentheses ile bildirilir:

```
public record Crane() {}
```

English: Not the kind of thing you'd use in your own code, but it could come up on the exam. **Türkçe:** Kendi kodunuzda kullanacağınız türden değildir; ancak sınavda karşınıza çıkabilir.

Understanding Record Immutability

English: As you saw, records don't have setters. Every field is inherently final and cannot be modified after it has been written in the constructor. In order to “modify” a record, you have to make a new object and copy all of the data you want to preserve. **Türkçe:** Gördüğümüz gibi record'ların setter'ı yoktur. Her field doğası gereği final'dır ve constructor içinde yazıldıktan sonra değiştirilemez. Record'u “değiştirmek” için yeni object oluşturup korumak istediğiniz bütün veriyi kopyalamanız gerekir.

```
var cousin = new Crane(3, "Jenny");
var friend = new Crane(cousin.numberEggs(), "Janeice");
```

English: Just as interfaces are implicitly abstract, records are also implicitly final. The final modifier is optional but assumed. **Türkçe:** Interface'lerin implicit abstract olması gibi record'lar da implicit final'dır. Final modifier optional'dır fakat var kabul edilir.

```
public final record Crane(int numberEggs, String name) {}
```

English: Like enums, that means you can't extend or inherit a record. **Türkçe:** Enum'larda olduğu gibi record'u extend veya inherit edemezsiniz.

```
public record BlueCrane() extends Crane {} // DOES NOT COMPILE
```

English: Also like enums, a record can implement a regular or sealed interface, provided it implements all of the abstract methods. **Türkçe:** Yine enum'larda olduğu gibi record, bütün abstract method'ları implement etmesi koşuluyla normal veya sealed interface'i implement edebilir.

```
public interface Bird {}  
public record Crane(int numberEggs, String name) implements Bird {}
```

English: Although well beyond the scope of this book, there are some good reasons to make data-oriented classes immutable. Doing so can lead to less error-prone code, as a new object is established any time the data is modified. It also makes them inherently thread-safe and usable in concurrent frameworks. **Türkçe:** Kitabın kapsamının ötesinde olsa da data-oriented class'ları immutable yapmanın iyi nedenleri vardır. Veri her değiştirildiğinde yeni object oluşturulduğu için daha az hata eğilimli kod sağlayabilir. Ayrıca bunları doğası gereği thread-safe ve concurrent framework'lerde kullanılabilir yapar.

Teknik sınır: Record component reference'ı final'dır; gösterdiği mutable object otomatik olarak immutable olmaz. Bu nedenle record immutability'si shallow'dur ve gerekirse defensive copy gerekir.

Declaring Constructors

English: What if you need to declare a record with some guards as we did earlier? In this section, we cover two ways we can accomplish this with records. **Türkçe:** Daha önceki gibi guard içeren record bildirmeniz gerekirse ne olur? Bu bölümde bunu record'larla yapmanın iki yolunu ele alıyoruz.

The Long Constructor

English: First, we can just declare the constructor the compiler normally inserts automatically, which we refer to as the long constructor. **Türkçe:** Önce compiler'ın normalde otomatik eklediği constructor'ı kendimiz bildirebiliriz; buna long constructor diyoruz.

```
public record Crane(int numberEggs, String name) {  
    public Crane(int numberEggs, String name) {  
        if (numberEggs < 0) throw new IllegalArgumentException();  
        this.numberEggs = numberEggs;  
        this.name = name;  
    }  
}
```

English: The compiler will not insert a constructor if you define one with the same list of parameters in the same order. Since each field is final, the constructor must set every field. For example, this record does not compile: **Türkçe:** Aynı parameter listesi ve sırasıyla constructor tanımlarsanız compiler başka constructor eklemeyiz. Her field final olduğundan constructor bütün field'ları atamalıdır. Örneğin aşağıdaki record derlenmez:

```
public record Crane(int numberEggs, String name) {  
    public Crane(int numberEggs, String name) {} // DOES NOT COMPILE  
}
```

English: While being able to declare a constructor is a nice feature of records, it's also problematic. If we have 20 fields, we'll need to declare assignments for every one, introducing the boilerplate we sought to remove. Oh, bother! **Türkçe:** Constructor bildirebilmek record'ların güzel bir özelliği olsa da sorun da yaratır. Yirmi field varsa her biri için assignment yazmak gerekir; böylece kaldırmaya çalıştığımız boilerplate geri gelir.

Compact Constructors

English: Luckily, the authors of Java added the ability to define a compact constructor for records. A compact constructor is a special type of constructor used for records to process validation and transformations succinctly. It takes no parameters and implicitly sets all fields. Figure 7.7 shows an example of a compact constructor. **Türkçe:** Neyse ki Java'nın geliştiricileri record'larda compact constructor tanımlama olanağını ekledi. Compact constructor, validation ve transformation işlemlerini kısa biçimde yapmak için record'larda kullanılan özel constructor türüdür. Parameter almaz ve bütün field'ları implicit olarak atar. Figure 7.7 bir compact constructor örneği gösterir.

Figure 7.7 · Declaring a compact constructor

English — figure callouts: No parentheses or constructor parameters; compact constructor; custom validation; refers to input parameters (not instance members); long constructor implicitly called at end of compact constructor. **Türkçe — şekil çağrıları:** Parentheses veya constructor parameter'ı yok; compact constructor; custom validation; instance member'lara değil input parameter'larına referans verir; long constructor compact constructor'ın sonunda implicit olarak çağrılır.

```
public record Crane(int numberEggs, String name) {  
    public Crane { // no parentheses or constructor parameters  
        if (numberEggs < 0) throw new IllegalArgumentException();  
        name = name.toUpperCase(); // input parameter, not instance member  
    }  
    // Long constructor assignments happen implicitly after the compact body.  
}
```

English: Great! Now we can check the values we want, and we don't have to list all the constructor parameters and trivial assignments. Java will execute the full constructor after the compact constructor. You should also remember that a compact constructor is declared without parentheses, as the exam might try to trick you on this. As shown in Figure 7.7, we can even transform constructor parameters, as we discuss more in the next section. **Türkçe:** Artık istediğimiz değerleri kontrol ederken bütün constructor parameter'larını ve basit assignment'ları listelememiz gerekmez. Java compact constructor body'den sonra full constructor assignment'larını çalıştırır. Sınav sizi yanıltmaya çalışabileceği için compact constructor'ın parentheses olmadan bildirildiğini unutmayın. Figure 7.7'deki gibi constructor parameter'larını dönüştürebiliriz; sonraki bölümde bunu genişletiyoruz.

English: You might think that you need custom methods for every field in the record, like the negative check we did with `setNumberEggs()`. In practice, many POJOs are created for general-purpose use with little validation. **Türkçe:** `setNumberEggs()` ile yaptığımız negatif kontrol gibi record'daki her field için custom method gerektiğini düşünebilirsiniz. Pratikte birçok POJO az validation ile general-purpose kullanım için oluşturulur.

Transforming Parameters

English: Compact constructors give you the opportunity to apply transformations to any of the input values. See if you can figure out what the following compact constructor does: **Türkçe:** Compact constructor'lar input değerlerine transformation uygulama fırsatı verir. Aşağıdaki compact constructor'ın ne yaptığını bulun:

```
public record Crane(int numberEggs, String name) {
    public Crane {
        if (name == null || name.length() < 1)
            throw new IllegalArgumentException();
        name = name.substring(0, 1).toUpperCase()
            + name.substring(1).toLowerCase();
    }
}
```

English: Give up? It validates the string, then formats it such that only the first letter is capitalized. As before, Java calls the full constructor after the compact constructor but with the modified constructor parameters. **Türkçe:** String'i validate eder, ardından yalnız ilk harf uppercase olacak biçimde formatlar. Daha önce olduğu gibi Java compact constructor'dan sonra, fakat değiştirilmiş constructor parameter'larıyla full constructor assignment'larını gerçekleştirir.

English: While compact constructors can modify the constructor parameters, they cannot modify the fields of the record. For example, this does not compile: **Türkçe:** Compact constructor constructor parameter'larını değiştirebilir, fakat record field'larını değiştiremez. Örneğin aşağıdaki kod derlenmez:

```
public record Crane(int numberEggs, String name) {
    public Crane {
        this.numberEggs = 10; // DOES NOT COMPILE
    }
}
```

English: Removing the `this` reference allows the code to compile, as the constructor parameter is modified instead. **Türkçe:** `this` reference çıkarılırsa record field yerine constructor parameter değiştirildiği için kod derlenir.

English: Although we covered both the long and compact forms of record constructors in this section, it is highly recommended that you stick with the compact form unless you have a good reason not to. **Türkçe:** Bu bölümde long ve compact record constructor biçimlerini ele alsak da aksini gerektiren iyi bir neden yoksa compact form kullanmanız güçlü biçimde önerilir.

Overloaded Constructors

English: You can also create overloaded constructors that take a completely different list of parameters. They are more closely related to the long-form constructor and don't use any of the syntactical features of compact constructors. **Türkçe:** Tamamen farklı parameter listesi alan overloaded constructor da oluşturabilirsiniz. Bunlar long-form constructor'la daha yakından ilişkilidir ve compact constructor'ın syntax özelliklerini kullanmaz.

```
public record Crane(int numberEggs, String name) {  
    public Crane(String firstName, String lastName) {  
        this(0, firstName + " " + lastName);  
    }  
}
```

English: The first line of an overloaded constructor must be an explicit call to another constructor via `this()`. If there are no other constructors, the long constructor must be called. Contrast this with what you learned about in Chapter 6, where calling `super()` or `this()` was often optional in constructor declarations. Also, unlike compact constructors, you can only transform the data on the first line. After the first line, all fields will already be assigned, and the object is immutable. **Türkçe:** Overloaded constructor'ın ilk satırı `this()` ile başka constructor'a explicit call olmalıdır. Başka constructor yoksa long constructor çağrılmalıdır. Bu durumu constructor declaration'da `super()` veya `this()` çağrısının çoğunlukla optional olduğu Chapter 6 ile karşılaştırın. Compact constructor'ın aksine data yalnız ilk satırda dönüştürülebilir. İlk satırdan sonra bütün field'lar atanmıştır ve object immutable'dır.

```
public record Crane(int numberEggs, String name) {  
    public Crane(int numberEggs, String firstName, String lastName) {  
        this(numberEggs + 1, firstName + " " + lastName);  
        numberEggs = 10;           // NO EFFECT: parameter, not instance field  
        this.numberEggs = 20;      // DOES NOT COMPILE  
    }  
}
```

English: As you saw in Chapter 6, you also can't declare two record constructors that call each other infinitely or as a cycle. **Türkçe:** Chapter 6'da gördüğümüz gibi birbirini sonsuza kadar veya cycle biçiminde çağıran iki record constructor da bildirilemez.

```
public record Crane(int numberEggs, String name) {  
    public Crane(String name) {  
        this(1); // DOES NOT COMPILE  
    }  
  
    public Crane(int numberEggs) {  
        this(""); // DOES NOT COMPILE  
    }  
}
```

Customizing Records

English: Since records are data-oriented, we've focused on the features of records you are likely to use. Records actually support many of the same features as a class. Here are some of the members that records can include and that you should be familiar with for the exam: overloaded and compact constructors; instance methods, including overriding any provided methods—accessors, `equals()`, `hashCode()`, and `toString()`; nested classes, interfaces, annotations, enums, and records. **Türkçe:** Record'lar data-oriented olduğundan muhtemelen kullanacağınız özelliklerine odaklandık. Aslında class'larla aynı birçok özelliği destekler. Sınav için record'ların şu member'ları içerebildiğini bilin: overloaded ve compact constructor'lar; sağlanan accessor, `equals()`, `hashCode()` ve `toString()` dahil instance method override'ları; nested class, interface, annotation, enum ve record'lar.

English: As an illustrative example, the following overrides two instance methods using the optional `@Override` annotation: **Türkçe:** Açıklayıcı bir örnek olarak aşağıdaki kod optional `@Override` annotation ile iki instance method'u override eder:

```
public record Crane(int numberEggs, String name) {
    @Override public int numberEggs() { return 10; }
    @Override public String toString() { return name; }
}
```

English: While you can add methods, static fields, and other data types, you cannot add instance fields outside the record declaration, even if they are private. Doing so defeats the purpose of using a record and could break immutability! **Türkçe:** Method, static field ve başka data type'lar ekleyebilirsiniz; ancak private olsalar bile record declaration dışına instance field ekleyemezsiniz. Bunu yapmak record kullanım amacını bozar ve immutability'yi kırabilir.

```
public record Crane(int numberEggs, String name) {
    private static int type = 10;
    public int size;           // DOES NOT COMPILE
    private boolean friendly;  // DOES NOT COMPILE
}
```

English: Records also do not support instance initializers. All initialization for the fields of a record must happen in a constructor. **Türkçe:** Record'lar instance initializer da desteklemez. Record field'larının bütün initialization işlemi constructor içinde gerçekleşmelidir.

English: While it's a useful feature that records support many of the same members as a class, try to keep them simple. Like the POJOs and JavaBeans they were born out of, the more complicated they get, the less usable they become. **Türkçe:** Record'ların class'larla aynı birçok member'ı desteklemesi yararlı olsa da onları basit tutmaya çalışın. Köken aldıkları POJO ve JavaBean'ler gibi karmaşıklıklaştıkça daha az kullanılabilir olurlar.

English: This is the second time we've mentioned nested types, the first being with sealed classes and now records. Don't worry; we're covering them next! **Türkçe:** Nested type'lardan ikinci kez söz ettik; ilki sealed class'larda, şimdi de record'larda. Endişelenmeyin, sırada onları ele alıyoruz.

Creating Nested Classes

English: A nested class is a class that is defined within another class. A nested class can come in one of four flavors: inner class, a non-static type defined at the member level of a class; static nested class, a static type defined at the member level of a class; local class, a class defined within a method body; anonymous class, a special case of a local class that does not have a name. **Türkçe:** Nested class başka bir class içinde tanımlanan class'tır ve dört türü vardır: Inner class, class'ın member level'ında tanımlanan non-static type; static nested class, class'ın member level'ında tanımlanan static type; local class, method body içinde tanımlanan class; anonymous class, adı olmayan özel bir local class türü.

English: There are many benefits of using nested classes. They can define helper classes and restrict them to the containing class, thereby improving encapsulation. They can make it easy to create a class that will be used in only one place. They can even make the code cleaner and easier to read. **Türkçe:** Nested class kullanımının birçok yararı vardır. Helper class'ları tanımlayıp containing class ile sınırlayarak encapsulation'ı iyileştirebilir. Yalnız bir yerde kullanılacak class oluşturmayı kolaylaştırabilir ve kodu daha temiz, daha okunur hâle getirebilir.

English: When used improperly, though, nested classes can sometimes make the code harder to read. They also tend to tightly couple the enclosing and inner class, but there may be cases where you want to use the inner class by itself. In this case, you should move the inner class out into a separate top-level class. **Türkçe:** Yanlış kullanıldığında nested class kodun okunmasını zorlaştırabilir. Enclosing ve inner class'ı sıkı biçimde couple etme eğilimindedir; oysa inner class'ı tek başına kullanmak istediğiniz durumlar olabilir. Böyle bir durumda inner class'ı ayrı top-level class'a taşımalısınız.

English: Unfortunately, the exam tests edge cases where programmers wouldn't typically use a nested class. This tends to create code that is difficult to read, so please never do this in practice! **Türkçe:** Ne yazık ki sınav programcılarının normalde nested class kullanmayacağı edge case'leri test eder. Bu, okunması zor kod üretir; pratikte böyle kod yazmayın.

English: By convention, and throughout this chapter, we often use the term nested class to refer to all nested types, including nested interfaces, enums, records, and annotations. You might even come across literature that refers to all of them as inner classes. We agree that this can be confusing! **Türkçe:** Convention gereği bu bölüm boyunca nested interface, enum, record ve annotation dahil bütün nested type'lar için sık sık “nested class” terimini kullanıyoruz. Hepsini inner class olarak adlandıran kaynaklarla da karşılaşabilirsiniz; bunun kafa karıştırıcı olabileceğini kabul ediyoruz.

Declaring an Inner Class

English: An inner class, also called a member inner class, is a non-static type defined at the member level of a class—the same level as methods, instance variables, and constructors. Because they are not top-level types, they can use any of the four access levels, not just public and package access. **Türkçe:** Member inner class da denilen inner class; method, instance variable ve constructor'larla aynı member level'da tanımlanan non-static type'tır. Top-level type olmadığından yalnız public ve package değil, dört access level'ın herhangi birini kullanabilir.

English: Inner classes have the following properties: They can be declared public, protected, package, or private. They can extend a class and implement interfaces. They can be marked abstract or final. They can access members of the outer class, including private members. **Türkçe:** Inner class'ların özellikleri: Public, protected, package veya private bildirilebilirler. Bir class'ı extend ve interface'leri implement edebilirler. Abstract veya final işaretlenebilirler. Private member'lar dahil outer class member'larına erişebilirler.

English: The last property is pretty cool. It means that the inner class can access variables in the outer class without doing anything special. Ready for a complicated way to print `Hi` three times? **Türkçe:** Son özellik oldukça kullanışlıdır. Inner class'ın özel bir işlem yapmadan outer class variable'larına erişebilmesi anlamına gelir. `Hi` değerini üç kez yazdırmanın karmaşık bir yoluna hazır mısınız?

```
1: public class Home {
2:     private String greeting = "Hi"; // Outer class instance variable
3:
4:     protected class Room {          // Inner class declaration
5:         public int repeat = 3;
6:         public void enter() {
7:             for (int i = 0; i < repeat; i++) greet(greeting);
8:         }
9:         private static void greet(String message) {
10:             System.out.println(message);
11:         }
12:     }
13:
14:     public void enterRoom() {        // Instance method in outer class
15:         var room = new Room();       // Create the inner class instance
16:         room.enter();
17:     }
18:     public static void main(String[] args) {
19:         var home = new Home();       // Create the outer class instance
20:         home.enterRoom();
21:     } }
```

English: An inner class declaration looks just like a stand-alone class declaration except that it happens to be located inside another class. Line 7 shows that the inner class refers to `greeting` as if it were available in the `Room` class. This works because it is, in fact, available. Even though the variable is private, it is accessed within that same class. **Türkçe:** Inner class declaration başka class içinde bulunması dışında stand-alone class declaration'a benzer. Line 7 inner class'ın `greeting` variable'ına `Room` içinde mevcutmuş gibi referans verdiğini gösterir; çünkü gerçekten erişilebilirdir. Variable private olsa da aynı enclosing class yapısı içinden erişilir.

English: Since an inner class is not static, it has to be called using an instance of the outer class. That means you have to create two objects. Line 19 creates the outer `Home` object, while line 15 creates the inner `Room` object. It's important to notice that line 15 doesn't require an explicit instance of `Home` because it is an instance method within `Home`. This works because `enterRoom()` is an instance method within the `Home` class. Both `Room` and `enterRoom()` are members of `Home`. **Türkçe:** Inner class static olmadığından outer class instance'ı kullanılarak çağırılmalıdır; iki object oluşturmak gerekir. Line 19 outer `Home`, line 15 inner `Room` object'ini oluşturur. Line 15'in explicit `Home` instance'ı gerektirmediğine dikkat edin; çünkü `Home` içindeki bir instance method kapsamındadır. Bu işe yarar; çünkü `enterRoom()`, `Home` class'ındaki bir instance method'dur. `Room` ile `enterRoom()` ikisi de `Home` member'ıdır.

Nested Classes Can Now Have static Members

English: Eagle-eyed readers may have noticed that we included a static method in our inner `Room` class on line 9. In Java 11, this would have resulted in a compiler error. Previously, only static nested classes were allowed to include static methods. With the introduction of records in Java 16, the existing rule that prevented an inner class from having any static members other than static constants was removed. All four types of nested classes can now define static variables and methods! **Türkçe:** Dikkatli okuyucular line 9'da inner `Room` class'ına static method eklediğimizi fark etmiş olabilir. Bu, Java 11'de compiler error üretti. Önceden yalnız static nested class'ların static method içermesine izin veriliyordu. Java 16'da record'ların gelmesiyle inner class'ın static constant dışındaki static member'lara sahip olmasını engelleyen kural kaldırıldı. Artık dört nested class türünün tamamı static variable ve method tanımlayabilir.

Instantiating an Instance of an Inner Class

English: There is another way to instantiate `Room` that looks odd at first. Okay, maybe not just at first. This syntax isn't used often enough to get used to it. **Türkçe:** `Room` instantiate etmenin ilk bakışta tuhaf görünen başka bir yolu vardır; belki yalnız ilk bakışta değil. Bu syntax alışıla gelen kadar sık kullanılmaz:

```
public static void main(String[] args) {
    var home = new Home();
    Room room = home.new Room(); // Create inner class instance
    room.enter();
}
```

English: Let's take a closer look at lines 21 and 22. We need an instance of `Home` to create a `Room`. We can't just call `new Room()` inside the static `main()` method, because Java won't know which instance of `Home` it is associated with. Java solves this by calling `new` as if it were a method on the `home` variable. We can shorten lines 21–23 to a single line: **Türkçe:** Lines 21 ve 22'ye yakından bakalım. `Room` oluşturmak için `Home` instance'ı gerekir. Static `main()` içinde yalnız `new Room()` çağrılmaz; çünkü Java bunun hangi `Home` instance'ıyla ilişkili olduğunu bilemez. Java, `new` ifadesini `home` variable'ı üzerinde method gibi çağırarak bu sorunu çözer. Lines 21–23 tek satıra indirilebilir:

```
new Home().new Room().enter(); // Sorry, it looks ugly to us too!
```

Creating .class Files for Inner Classes

English: Compiling the `Home.java` class with which we have been working creates two class files. You should be expecting the `Home.class` file. For the inner class, the compiler creates `Home$Room.class`. You don't need to know this syntax for the exam. We mention it so that you aren't surprised to see files with `$` appearing in your directories. You do need to understand that multiple class files are created from a single `.java` file. **Türkçe:** Üzerinde çalıştığımız `Home.java` class'ını derlemek iki class file oluşturur. `Home.class` beklenen dosyadır; compiler inner class için `Home$Room.class` oluşturur. Bu syntax'ı sınav için bilmeniz gerekmez; yalnız dizinlerinizde `$` içeren dosyalar görünce şaşırmanız için belirtilmiştir. Tek bir `.java` dosyasından birden fazla class file üretilebildiğini anlamamız gerekir.

Referencing Members of an Inner Class

English: Inner classes can have the same variable names as outer classes, making scope a little tricky. There is a special way of calling `this` to say which variable you want to access. This is something you might see on the exam but, ideally, not in the real world. **Türkçe:** Inner class'lar outer class'larla aynı variable adlarına sahip olabilir; bu da scope'u zorlaştırır. Hangi variable'a erişmek istediğinizi belirtmek için `this` çağrısının özel bir biçimi vardır. Bunu sınavda görebilirsiniz, ideal olarak gerçek kodda görmezsiniz.

English: In fact, you aren't limited to just one inner class. While the following is common on the exam, please never do this in code you write. Here is how to nest multiple classes and access a variable with the same name in each: **Türkçe:** Tek bir inner class ile sınırlı değilsiniz. Aşağıdaki yapı sınavda yaygın olsa da yazdığınız gerçek kodda kullanmayın. Birden fazla class'ı nested yapmak ve her birindeki aynı adlı variable'a erişmek şöyle gerçekleştirilir:

```
public class A {
    private int x = 10;

    class B {
        private int x = 20;

        class C {
            private int x = 30;

            public void allTheX() {
                System.out.println(x);           // 30
                System.out.println(this.x);       // 30
                System.out.println(B.this.x);     // 20
                System.out.println(A.this.x);     // 10
            }
        }
    }

    public static void main(String[] args) {
        A a = new A();
        A.B b = a.new B();
        A.B.C c = b.new C();
        c.allTheX();
    }
}
```

English: Yes, this code makes us cringe too. It has two nested classes. Line 14 instantiates the outermost one. Line 15 uses the awkward syntax to instantiate a `B`. Notice that the type is `A.B`. We could have written `B` as the type because that is available at the member level of `A`; Java knows where to look. On line 16, we instantiate a `C`. This time, the `A.B.C` type is necessary to specify. `C` is too deep for Java to know where to look. Then line 17 calls a method on the instance variable `c`. **Türkçe:** Bu kod bizi de rahatsız ediyor. İki nested class içerir. Line 14 en dıştakini instantiate eder. Line 15 tuhaf syntax ile `B` oluşturur; type'ın `A.B` olduğuna dikkat edin. `B`, `A`'nın member level'ında erişilebilir olduğundan type olarak yalnız `B` de yazılabilirdi; Java nereye bakacağını bilir. Line 16'da `C` instantiate edilir. Bu kez `A.B.C` type'ını belirtmek gerekir; `C`, Java'nın tek başına yerini bulamayacağı kadar derindedir. Line 17 instance variable `c` üzerinde method çağırır.

English: Lines 8 and 9 are the type of code we are used to seeing. They refer to the instance variable on the current class—the one declared on line 6, to be precise. Line 10 uses `this` in a special way. We still want an instance variable, but this time we want the one on class `B`, declared on line 4. Line 11 does the same thing for class `A`, getting the variable from line 2. **Türkçe:** Lines 8 ve 9 alışık olduğumuz koddur; tam olarak söylersek current class'ta line 6'da bildirilen instance variable'a referans verir. Line 10 `this`'i özel biçimde kullanır. Yine instance variable isteriz, fakat bu kez class `B` üzerinde line 4'te bildirilen variable'ı seçeriz. Line 11 aynı işlemi class `A` için yaparak line 2'deki variable'ı alır.

Inner Classes Require an Instance

English: Take a look at the following and see whether you can figure out why two of the three constructor calls do not compile. **Türkçe:** Aşağıdaki örneğe bakarak üç constructor call'dan ikisinin neden derlenmediğini bulmaya çalışın:

```
public class Fox {
    private class Den {}

    public void goHome() {
        new Den();
    }

    public static void visitFriend() {
        new Den(); // DOES NOT COMPILE
    }
}

public class Squirrel {
    public void visitFox() {
        new Den(); // DOES NOT COMPILE
    }
}
```

English: The first constructor call compiles because `goHome()` is an instance method, and therefore the call is associated with the `this` instance. The second call does not compile because it is called inside a static method. You can still call the constructor, but you have to explicitly give it a reference to a `Fox` instance. **Türkçe:** İlk constructor call derlenir; çünkü `goHome()` instance method'dur ve çağrı `this` instance'ıyla ilişkilidir. İkinci çağrı static method içinde yapıldığı için derlenmez. Constructor yine çağrılabilir; ancak explicit bir `Fox` instance reference verilmelidir.

English: The last constructor call does not compile for two reasons. Even though it is an instance method, it is not an instance method inside the `Fox` class. Adding a `Fox` reference would not fix the problem entirely, though. `Den` is private and not accessible in the `Squirrel` class. **Türkçe:** Son constructor call iki nedenle derlenmez. Instance method içinde olsa da bu method `Fox` class'ında değildir. `Fox` reference eklemek de sorunu bütünüyle çözmez; `Den` private'tır ve `Squirrel` class'ından erişilemez.

Creating a static Nested Class

English: A static nested class is a static type defined at the member level. Unlike an inner class, a static nested class can be instantiated without an instance of the enclosing class. The tradeoff, though, is that it can't access instance variables or methods declared in the outer class. **Türkçe:** Static nested class member level'da tanımlanan static type'tır. Inner class'ın aksine enclosing class instance'ı olmadan instantiate edilebilir. Bunun karşılığında outer class'ta bildirilen instance variable veya method'lara erişemez.

English: In other words, it is like a top-level class except for the following: The nesting creates a namespace because the enclosing class name must be used to refer to it. It can additionally be marked private or protected. The enclosing class can refer to the fields and methods of the static nested class. **Türkçe:** Başka bir deyişle şu farklar dışında top-level class gibidir: Nesting bir namespace oluşturur; referans verirken enclosing class adı kullanılmalıdır. Ek olarak private veya protected işaretlenebilir. Enclosing class static nested class'ın field ve method'larına erişebilir.

English: Let's take a look at an example: **Türkçe:** Bir örneğe bakalım:

```
1: public class Park {  
2:     static class Ride {  
3:         private int price = 6;  
4:     }
```

```
5:     public static void main(String[] args) {  
6:         var ride = new Ride();  
7:         System.out.println(ride.price);  
8:     } }
```

English: Line 6 instantiates the nested class. Since the class is static, you do not need an instance of `Park` to use it. You are allowed to access private instance variables, as shown on line 7. **Türkçe:** Line 6 nested class'ı instantiate eder. Class static olduğundan onu kullanmak için `Park` instance'ı gerekmez. Line 7'de gösterildiği gibi private instance variable'a erişmeye izin verilir.

Writing a Local Class

English: A local class is a nested class defined within a method. Like local variables, a local class declaration does not exist until the method is invoked, and it goes out of scope when the method returns. This means you can create instances only from within the method. Those instances can still be returned from the method. This is just how local variables work. **Türkçe:** Local class method içinde tanımlanan nested class'tır. Local variable'lar gibi local class declaration method çağrılana kadar var olmaz ve method döndüğünde scope dışına çıkar. Bu nedenle instance yalnız method içinden oluşturulabilir; yine de bu instance method'dan döndürülebilir. Local variable'ların çalışma biçimi de budur.

English: Local classes are not limited to being declared only inside methods. For example, they can be declared inside constructors and initializers. For simplicity, we limit our discussion to methods in this chapter. **Türkçe:** Local class yalnız method içinde bildirilmekle sınırlı değildir; constructor ve initializer içinde de bildirilebilir. Basitlik için bu bölümde tartışmayı method'larla sınırlandırıyoruz.

English: Local classes have the following properties: They do not have an access modifier. They can be declared final or abstract. They have access to all fields and methods of the enclosing class when defined in an instance method. They can access final and effectively final local variables. **Türkçe:** Local class özellikleri: Access modifier taşımaz. Final veya abstract bildirilebilir. Instance method içinde tanımlandığında enclosing class'ın bütün field ve method'larına erişebilir. Final ve effectively final local variable'lara erişebilir.

English: Remember when we presented effectively final in Chapter 5? We said it would come in handy later, and it's later! If you need a refresher on final and effectively final, turn back to Chapter 5 now. Don't worry; we'll wait! **Türkçe:** Chapter 5'te effectively final'ı anlatırken daha sonra işe yarayacağını söylemiştik; o zaman geldi. Final ve effectively final tekrarına gereksiniminiz varsa Chapter 5'e dönün; bekleriz.

English: Ready for an example? Here's a complicated way to multiply two numbers: **Türkçe:** Bir örneğe hazır mısınız? İki sayıyı çarpmanın karmaşık bir yolu:

```
1: public class PrintNumbers {
2:     private int length = 5;
3:     public void calculate() {
4:         final int width = 20;
5:         class Calculator {
6:             public void multiply() {
7:                 System.out.print(length * width);
8:             }
9:         }
```

```
10:         var calculator = new Calculator();
11:         calculator.multiply();
12:     }
13:     public static void main(String[] args) {
14:         var printer = new PrintNumbers();
15:         printer.calculate(); // 100
16:     }
17: }
```

English: Lines 5–9 are the local class. That class's scope ends on line 12, where the method ends. Line 7 refers to an instance variable and a final local variable, so both variable references are allowed from within the local class. **Türkçe:** Lines 5–9 local class'tır. Class'ın scope'u method'un bittiği line 12'de sona erer. Line 7 bir instance variable ile final local variable'a referans verir; bu nedenle iki reference da local class içinden kullanılabilir.

English: Earlier, we made the statement that local variable references are allowed if they are final or effectively final. As an illustrative example, consider the following: **Türkçe:** Daha önce local variable reference'ların final veya effectively final olmaları durumunda kullanılabildiğini söyledik. Aşağıdaki açıklayıcı örneği inceleyin:

```
public void processData() {  
    final int length = 5;  
    int width = 10;  
    int height = 2;  
  
    class VolumeCalculator {  
        public int multiply() {  
            return length * width * height; // DOES NOT COMPILE  
        }  
    }  
  
    width = 2;  
}
```

English: The `length` and `height` variables are final and effectively final, respectively, so neither causes a compilation issue. On the other hand, the `width` variable is reassigned during the method, so it cannot be effectively final. For this reason, the local class declaration does not compile. **Türkçe:** `length` final, `height` effectively final olduğundan ikisi de compilation sorunu oluşturmaz. `width` ise method sırasında yeniden atandığından effectively final olamaz. Bu nedenle local class declaration derlenmez.

Why Can Local Classes Only Access final or Effectively Final Variables?

English: Earlier, we mentioned that the compiler generates a separate `.class` file for each inner class. A separate class has no way to refer to a local variable. However, if the local variable is final or effectively final, Java can handle it by passing a copy of the value or reference variable to the constructor of the local class. If it weren't final or effectively final, these tricks wouldn't work because the value could change after the copy was made. **Türkçe:** Compiler'ın her inner class için ayrı `.class` file ürettiğini belirtmiştik. Ayrı class'ın local variable'a referans verme yolu yoktur. Ancak local variable final veya effectively final ise Java value ya da reference variable'ın kopyasını local class constructor'ına geçirerek bunu çözebilir. Final/effectively final olmasaydı kopya oluşturulduktan sonra value değişebileceğinden bu yöntem çalışmazdı.

Defining an Anonymous Class

English: An anonymous class is a specialized form of a local class that does not have a name. It is declared and instantiated all in one statement using the `new` keyword, a type name with parentheses, and a set of braces `{ }`. Anonymous classes must extend an existing class or implement an existing interface. They are useful when you have a short implementation that will not be used anywhere else. Here's an example: **Türkçe:** Anonymous class adı olmayan özel bir local class biçimidir. `new` keyword'ü, parentheses içeren type adı ve `{ }` braces kullanılarak tek statement'ta hem bildirilir hem instantiate edilir. Anonymous class mevcut bir class'ı extend veya mevcut interface'i implement etmelidir. Başka yerde kullanılmayacak kısa implementation'lar için yararlıdır.

```

1: public class ZooGiftShop {
2:     abstract class SaleTodayOnly {
3:         abstract int dollarsOff();
4:     }
5:     public int admission(int basePrice) {
6:         SaleTodayOnly sale = new SaleTodayOnly() {
7:             int dollarsOff() { return 3; }
8:         }; // Don't forget the semicolon!
9:         return basePrice - sale.dollarsOff();
10:    } }

```

English: Lines 2–4 define an abstract class. Lines 6–8 define the anonymous class. Notice how this anonymous class does not have a name. The code says to instantiate a new `SaleTodayOnly` object. But wait: `SaleTodayOnly` is abstract. This is okay because we provide the class body right there—anonously. In this example, writing an anonymous class is equivalent to writing a local class with an unspecified name that extends `SaleTodayOnly` and immediately uses it. **Türkçe:** Lines 2–4 abstract class, lines 6–8 anonymous class tanımlar. Anonymous class'ın adı olmadığına dikkat edin. Kod yeni `SaleTodayOnly` object'i instantiate eder; oysa `SaleTodayOnly` abstract'tır. Class body'yi aynı yerde anonymous biçimde sağladığımız için bu geçerlidir. Bu örnekte anonymous class yazmak, adı belirtilmeyen ve `SaleTodayOnly` class'ını extend edip hemen kullanan local class yazmaya eşdeğerdir.

English: Pay special attention to the semicolon on line 8. We are declaring a local variable on these lines. Local variable declarations are required to end with semicolons, just like other Java statements—even if they are long and happen to contain an anonymous class. **Türkçe:** Line 8'deki semicolon'a özellikle dikkat edin. Bu satırlarda local variable bildiriyoruz. Local variable declaration, uzun olup anonymous class içerse bile diğer Java statement'ları gibi semicolon ile bitmek zorundadır.

English: Now we convert this same example to implement an interface instead of extending an abstract class: **Türkçe:** Aynı örneği abstract class extend etmek yerine interface implement edecek biçime dönüştürelim:

```

1: public class ZooGiftShop {
2:     interface SaleTodayOnly {
3:         int dollarsOff();
4:     }
5:     public int admission(int basePrice) {
6:         SaleTodayOnly sale = new SaleTodayOnly() {
7:             public int dollarsOff() { return 3; }
8:         };
9:         return basePrice - sale.dollarsOff();
10:    } }

```

English: The most interesting thing here is how little has changed. Lines 2–4 declare an interface instead of an abstract class. Line 7 is public instead of using default access since interfaces require public methods. And that is it. The anonymous class is the same whether you implement an interface or extend a class! Java figures out which one you want automatically. Just remember that in this second example, an instance of a class is created on line 6, not an interface. **Türkçe:** En ilginç nokta çok az şeyin değişmiş olmasıdır. Lines 2–4 abstract class yerine interface bildirir. Interface'ler public method gerektirdiği için line 7 default access yerine public'tir. Hepsi budur. Interface implement ederken de class extend ederken de anonymous class aynıdır; Java hangisini istediğinizi otomatik belirler. İkinci örnekte line 6'da interface değil class instance'ı oluşturulduğunu unutmayın.

English: But what if we want both to implement an interface and extend a class? You can't do so with an anonymous class unless the class to extend is `java.lang.Object`. The `Object` class doesn't count in the rule. Remember that an anonymous class is just an unnamed local class. You can write a local class and give it a name if you have this problem. Then you can extend a class and implement as many interfaces as you like. If your code is this complex, a local class probably isn't the most readable option anyway. **Türkçe:** Hem interface implement edip hem class extend etmek istersek ne olur? Extend edilen class `java.lang.Object` olmadığı sürece bunu anonymous class ile yapamazsınız; `Object` bu kuralda sayılmaz. Anonymous class'ın adsız local class olduğunu unutmayın. Bu sorun varsa adlı local class yazabilir, ardından bir class'ı extend edip istediğiniz sayıda interface'i implement edebilirsiniz. Kod bu kadar karmaşıksa local class da muhtemelen en okunur seçenek değildir.

English: You can even define anonymous classes outside a method body. The following may look like we are instantiating an interface as an instance variable, but the `{ }` after the interface name indicates that this is an anonymous class implementing the interface: **Türkçe:** Anonymous class method body dışında bile tanımlanabilir. Aşağıdaki kod interface'i instance variable olarak instantiate ediyor gibi görünür; ancak interface adından sonraki `{ }`, interface'i implement eden anonymous class olduğunu gösterir:

```
public class Gorilla {
    interface Climb {}
    Climb climbing = new Climb() {};
}
```

Anonymous Classes and Lambda Expressions

English: Prior to Java 8, anonymous classes were frequently used for asynchronous tasks and event handlers. For example, the following shows an anonymous class used as an event handler in a JavaFX application: **Türkçe:** Java 8'den önce anonymous class'lar asynchronous task ve event handler'larda sık kullanılırdı. Örneğin aşağıdaki kod JavaFX application'da event handler olarak anonymous class kullanır:

```
var redButton = new Button();
redButton.setOnAction(new EventHandler<ActionEvent>() {
    public void handle(ActionEvent e) {
        System.out.println("Red button pressed!");
    }
});
```

English: Since the introduction of lambda expressions, anonymous classes are now often replaced with much shorter implementations: **Türkçe:** Lambda expression'ların gelmesiyle anonymous class'ların yerini çoğunlukla daha kısa implementation'lar alır:

```
Button redButton = new Button();
redButton.setOnAction(e -> System.out.println("Red button pressed!"));
```

English: We cover lambda expressions in detail in the next chapter. **Türkçe:** Lambda expression'ları sonraki bölümde ayrıntılı ele alıyoruz.

Reviewing Nested Classes

English: For the exam, make sure that you know the information in Table 7.4 about which syntax rules are permitted in Java. **Türkçe:** Sınav için Java'da hangi syntax kurallarına izin verildiğini gösteren Table 7.4 bilgisini bildiğinizden emin olun.

Table 7.4 · Modifiers in nested classes

Permitted modifier	Inner class	Static nested class	Local class	Anonymous class
Access modifiers	All	All	None	None
<code>abstract</code>	Yes	Yes	Yes	No
<code>final</code>	Yes	Yes	Yes	No

Türkçe tablo açıklaması: Inner ve static nested class bütün access modifier'ları ile `abstract`/`final` olabilir. Local class access modifier alamaz, fakat `abstract` veya `final` olabilir. Anonymous class bu modifier'ları explicit olarak bildirmez.

English: You should also know the information in Table 7.5 about types of access. For example, the exam might try to trick you by having a static class access an outer class instance variable without a reference to the outer class. **Türkçe:** Access türlerini gösteren Table 7.5'i de bilmelisiniz. Örneğin sınav static class'ın outer class reference olmadan outer instance variable'a erişmesini göstererek sizi yanıltmaya çalışabilir.

Table 7.5 · Nested class access rules

Rule	Inner class	Static nested class	Local class	Anonymous class
Can extend a class or implement any number of interfaces?	Yes	Yes	Yes	No—must have exactly one superclass or one interface
Can access instance members of enclosing class?	Yes	No	Yes, if declared in an instance method	Yes, if declared in an instance method
Can access local variables of enclosing method?	N/A	N/A	Yes, if final or effectively final	Yes, if final or effectively final

Türkçe tablo açıklaması: Inner, static nested ve local class bir class'ı extend ve interface'leri implement edebilir. Anonymous class yalnız tek bir superclass veya interface hedefler. Enclosing instance member erişimi static nested class'ta yoktur; local/anonymous class'ta instance method bağlamına bağlıdır. Enclosing method local variable'ları local ve anonymous class için final veya effectively final olmalıdır.

Understanding Polymorphism

English: We conclude this chapter with a discussion of polymorphism, the property of an object to take on many different forms. To put this more precisely, a Java object may be accessed using a reference with the same type as the object, a reference that is a superclass of the object, or a reference that defines an interface the object implements or inherits. **Türkçe:** Bu bölümü bir object'in birçok farklı biçim alabilme özelliği olan polymorphism tartışmasıyla tamamlıyoruz. Daha kesin ifadeyle Java object'ine; object ile aynı type'ta reference, object'in superclass'ı olan reference veya object'in implement/inherit ettiği interface'i tanımlayan reference üzerinden erişilebilir.

English: Furthermore, a cast is not required if the object is being reassigned to a supertype or interface of the object. Phew, that's a lot! Don't worry; it'll make sense shortly. **Türkçe:** Ayrıca object kendi supertype'ına veya interface'ine yeniden atanıyorsa cast gerekmez. Çok şey varmış gibi görünse de birazdan anlamlı hâle gelecektir.

English: Let's illustrate this polymorphism property with the following example: **Türkçe:** Polymorphism özelliğini aşağıdaki örnekle gösterelim:

```
public class Primate {
    public boolean hasHair() {
        return true;
    }
}

public interface HasTail {
    public abstract boolean isTailStriped();
}

public class Lemur extends Primate implements HasTail {
    public boolean isTailStriped() {
        return false;
    }

    public int age = 10;

    public static void main(String[] args) {
        Lemur lemur = new Lemur();
        System.out.println(lemur.age);

        HasTail hasTail = lemur;
        System.out.println(hasTail.isTailStriped());

        Primate primate = lemur;
        System.out.println(primate.hasHair());
    }
}
```

English: This code compiles and prints the following output: **Türkçe:** Bu kod derlenir ve aşağıdaki çıktıyı üretir:

```
10
false
true
```

English: The most important thing to note about this example is that only one object, `Lemur`, is created. Polymorphism enables an instance of `Lemur` to be reassigned or passed to a method using one of its supertypes, such as `Primate` or `HasTail`. **Türkçe:** Bu örnekteki en önemli nokta yalnız bir `Lemur` object'i oluşturulmasıdır. Polymorphism, `Lemur` instance'ının `Primate` veya `HasTail` gibi supertype'larından biri kullanılarak yeniden atanmasını ya da method'a geçirilmesini sağlar.

English: Once the object has been assigned to a new reference type, only the methods and variables available to that reference type are callable on the object without an explicit cast. For example, the following snippets do not compile: **Türkçe:** Object yeni reference type'a atandıktan sonra explicit cast olmadan yalnız bu reference type'ta bulunan method ve variable'lar object üzerinde çağrılabilir. Örneğin aşağıdaki kodlar derlenmez:

```
HasTail hasTail = new Lemur();
System.out.println(hasTail.age); // DOES NOT COMPILE

Primate primate = new Lemur();
System.out.println(primate.isTailStriped()); // DOES NOT COMPILE
```

English: In this example, the reference `hasTail` has direct access only to methods defined with the `HasTail` interface; therefore, it doesn't know that the variable `age` is part of the object. Likewise, the reference `primate` has access only to methods defined in the `Primate` class, and it doesn't have direct access to the `isTailStriped()` method. **Türkçe:** Bu örnekte `hasTail` reference yalnız `HasTail` interface'inde tanımlanan method'lara doğrudan erişir; dolayısıyla `age` variable'ının object'in parçası olduğunu bilmez. Benzer şekilde `primate` yalnız `Primate` class'ında tanımlanan method'lara erişir ve `isTailStriped()` method'una doğrudan erişemez.

Object vs. Reference

English: In Java, all objects are accessed by reference, so as a developer you never have direct access to the object itself. Conceptually, though, you should consider the object as the entity that exists in memory, allocated by the Java Runtime Environment. Regardless of the type of reference you have for the object in memory, the object itself doesn't change. For example, since all objects inherit `java.lang.Object`, they can all be reassigned to `java.lang.Object`, as shown in the following example: **Türkçe:** Java'da bütün object'lere reference ile erişilir; geliştirici olarak object'in kendisine doğrudan erişemezsiniz. Kavramsal olarak object'i Java Runtime Environment tarafından memory'de allocate edilen entity kabul edin. Memory'deki object için kullandığınız reference type ne olursa olsun object'in kendisi değişmez. Örneğin bütün object'ler `java.lang.Object` inherit ettiği için hepsi ona yeniden atanabilir:

```
Lemur lemur = new Lemur();
Object lemurAsObject = lemur;
```

English: Even though the `Lemur` object has been assigned to a reference with a different type, the object itself has not changed and still exists as a `Lemur` object in memory. What has changed, then, is our ability to access methods within the `Lemur` class with the `lemurAsObject` reference. Without an explicit cast back to `Lemur`, as you see in the next section, we no longer have access to the `Lemur` properties of the object. **Türkçe:** `Lemur` object farklı type'ta bir reference'a atanmış olsa da object'in kendisi değişmemiştir ve memory'de hâlâ bir `Lemur` object olarak bulunur. O halde değişen şey, `lemurAsObject` reference'ı ile `Lemur` class'ındaki method'lara erişme yeteneğimizdir. Sonraki bölümde göreceğiniz gibi tekrar `Lemur`'a explicit cast yapmadan object'in `Lemur` property'lerine artık erişemeyiz.

English: We can summarize this principle with the following two rules: 1. The type of the object determines which properties exist within the object in memory. 2. The type of the reference to the object determines which methods and variables are accessible to the Java program. **Türkçe:** İlke iki kuralla özetlenebilir: 1. Object type, memory'deki object içinde hangi property'lerin bulunduğunu belirler. 2. Object'e ait reference type, Java programının hangi method ve variable'lara erişebildiğini belirler.

English: It therefore follows that successfully changing a reference of an object to a new reference type may give you access to new properties of the object; but remember, those properties existed before the reference change occurred. **Türkçe:** Buna göre object reference'ını başarıyla yeni reference type'a dönüştürmek object'in yeni property'lerine erişim sağlayabilir; fakat bu property'lerin reference değişmeden önce de var olduğunu unutmayın.

English: Using the `Lemur` example, we illustrate this property in Figure 7.8. **Türkçe:** Bu özelliği `Lemur` örneğiyle Figure 7.8'de gösteriyoruz.

Figure 7.8 · Object vs. reference

English — figure labels in source order: Reference of interface `HasTail`; `hasTail`; `Lemur` object in memory; reference of class `Lemur`; `age 10`; `lemur`; `hasHair()`; `isTailStriped()`; reference of class `Primate`; `primate`. **Türkçe — kaynak sırasıyla şekil etiketleri:** `HasTail` interface reference'ı; `hasTail`; memory'deki `Lemur` object'i; `Lemur` class reference'ı; `age 10`; `lemur`; `hasHair()`; `isTailStriped()`; `Primate` class reference'ı; `primate`.

```
HasTail reference: hasTail ┌
Lemur reference:  lemur  └─┬─┐ Lemur object in memory
Primate reference: primate └─┴─┘ age = 10
                             hasHair()
                             isTailStriped()
```

English: As you can see in the figure, the same object exists in memory regardless of which reference is pointing to it. Depending on the type of the reference, we may only have access to certain methods. For example, the `hasTail` reference has access to the method `isTailStriped()` but doesn't have access to the variable `age` defined in the `Lemur` class. As you learn in the next section, it is possible to reclaim access to the variable `age` by explicitly casting the `hasTail` reference to a reference of type `Lemur`.

Türkçe: Şekilde görebileceğiniz gibi hangi reference gösterirse gösterebilirsin memory'de aynı object bulunur. Reference'ın type'ına bağlı olarak yalnızca belirli method'lara erişebiliriz. Örneğin `hasTail` reference'ı `isTailStriped()` method'una erişebilir; ancak `Lemur` class'ında tanımlanan `age` variable'ına erişemez. Sonraki bölümde öğreneceğiniz gibi `hasTail` reference'ını explicit olarak `Lemur` type'ında bir reference'a cast ederek `age` variable'ına yeniden erişmek mümkündür.

Using Interface References

English: When working with a group of objects that implement a common interface, it is considered good coding practice to use an interface as the reference type. This is especially common with collections, which you learn about in Chapter 9, “Collections and Generics.” Consider the following method: **Türkçe:** Ortak interface'i implement eden object grubuyla çalışırken reference type olarak interface kullanmak iyi coding practice kabul edilir. Bu özellikle Chapter 9, “Collections and Generics” bölümünde öğreneceğiniz collection'larda yaygındır. Aşağıdaki method'u inceleyin:

```
public void sortAndPrintZooAnimals(List<String> animals) {
    Collections.sort(animals);
    for (String a : animals) System.out.println(a);
}
```

English: This method sorts and prints animals in alphabetical order. At no point is this class interested in what the actual underlying object for `animals` is. It might be an `ArrayList` or another type. The point is, our code works on any of these types because we used the interface reference type rather than a class type. **Türkçe:** Bu method hayvanları alfabetik sıraya dize ve yazdırır. Bu class, `animals` için gerçek underlying object'in ne olduğuyla hiçbir noktada ilgilenmez. Bu object bir `ArrayList` ya da başka bir type olabilir. Önemli nokta şudur: Class type yerine interface reference type kullandığımız için kodumuz bu type'ların herhangi biriyle çalışır.

Casting Objects

English: In the previous example, we created a single instance of a `Lemur` object and accessed it via superclass and interface references. Once we changed the reference type, though, we lost access to more specific members defined in the subclass that still exist within the object. We can reclaim those references by casting the object back to the specific subclass it came from: **Türkçe:** Önceki örnekte tek `Lemur` object instance'ı oluşturup superclass ve interface reference'larıyla eriştik. Reference type değişince object içinde var olmayı sürdüren fakat subclass'ta tanımlanan daha specific member'lara erişimi kaybettik. Object'i geldiği specific subclass'a cast ederek bu erişimi geri alabiliriz:

```
Lemur lemur = new Lemur();
Primate primate = lemur;           // Implicit Cast to supertype
Lemur lemur2 = (Lemur) primate;    // Explicit Cast to subtype
Lemur lemur3 = primate;            // DOES NOT COMPILE (missing cast)
```

English: In this example, we first create a `Lemur` object and implicitly cast it to a `Primate` reference. Since `Lemur` is a subtype of `Primate`, this can be done without a cast operator. We then cast it back to a `Lemur` object using an explicit cast, gaining access to all of the methods and fields in the `Lemur` class. The last line does not compile because an explicit cast is required. Even though the object is stored in memory as a `Lemur` object, we need an explicit cast to assign it to `Lemur`. **Türkçe:** Bu örnekte önce bir `Lemur` object oluşturup onu implicit olarak `Primate` reference'a cast ederiz. `Lemur`, `Primate` subtype'ı olduğundan bu işlem cast operator olmadan yapılabilir. Ardından explicit cast kullanarak yeniden bir `Lemur` object'e cast eder ve `Lemur` class'ındaki bütün method ve field'lara erişiriz. Son satır explicit cast gerektiği için derlenmez. Object memory'de bir `Lemur` object olarak tutulsa bile onu `Lemur`'a atamak için explicit cast gerekir.

English: Casting objects is similar to casting primitives, as you saw in Chapter 2, “Operators.” When casting objects, you do not need a cast operator if casting to an inherited supertype. This is referred to as an implicit cast and applies to classes or interfaces the object inherits. Alternatively, if you want to access a subtype of the current reference, you need to perform an explicit cast with a compatible type. If the underlying object is not compatible with the type, then a `ClassCastException` will be thrown at runtime. **Türkçe:** Object casting, Chapter 2, “Operators” bölümünde gördüğünüz primitive casting'e benzer. Object'leri inherited bir supertype'a cast ederken cast operator gerekmez. Buna implicit cast denir ve object'in inherit ettiği class veya interface'ler için geçerlidir. Buna karşılık mevcut reference'ın bir subtype'ına erişmek istiyorsanız compatible bir type ile explicit cast yapmanız gerekir. Underlying object bu type ile compatible değilse runtime'da `ClassCastException` fırlatılır.

English: When reviewing a question on the exam that involves casting and polymorphism, be sure to remember what the instance of the object actually is. Then, focus on whether the compiler will allow the object to be referenced with or without explicit casts. **Türkçe:** Sınavda casting ve polymorphism içeren bir soruyu incelerken object'in gerçek instance'ının ne olduğunu mutlaka hatırlayın. Ardından compiler'ın object'e explicit cast ile ya da castsiz reference verilmesine izin verip vermediğine odaklanın.

English: We summarize these concepts into a set of rules for you to memorize for the exam: 1. Casting a reference from a subtype to a supertype doesn't require an explicit cast. 2. Casting a reference from a supertype to a subtype requires an explicit cast. 3. At runtime, an invalid cast of a reference to an incompatible type results in a `ClassCastException` being thrown. 4. The compiler disallows casts to unrelated types. **Türkçe:** Bu kavramları sınav için ezberlemeniz gereken kurallar halinde özetleyelim: 1. Bir reference'ı subtype'dan supertype'a cast etmek explicit cast gerektirmez. 2. Bir reference'ı supertype'dan subtype'a cast etmek explicit cast gerektirir. 3. Runtime'da bir reference'ın incompatible type'a geçersiz cast edilmesi `ClassCastException` fırlatılmasıyla sonuçlanır. 4. Compiler unrelated type'lar arasındaki cast'lere izin vermez.

Disallowed Casts

English: The first three rules are just a review of what we've said so far. The last rule is a bit more complicated. The exam may try to trick you with a cast that the compiler knows is not permitted (aka impossible). In the previous example, we were able to cast a `Primate` reference to a `Lemur` reference because `Lemur` is a subclass of `Primate` and therefore related. Consider this example instead: **Türkçe:** İlk üç kural yalnızca şimdiye kadar söylediklerimizin tekrarıdır. Son kural biraz daha karmaşıktır. Sınav, compiler'ın permitted olmadığını (başka bir deyişle impossible olduğunu) bildiği bir cast ile sizi yanıltmaya çalışabilir. Önceki örnekte bir `Primate` reference'ını `Lemur` reference'ına cast edebildik; çünkü `Lemur`, `Primate` subclass'ıdır ve bu nedenle type'lar ilişkilidir. Bunun yerine şu örneği inceleyin:

```
public class Bird {}
```

```
public class Fish {  
    public static void main(String[] args) {  
        Fish fish = new Fish();  
        Bird bird = (Bird) fish; // DOES NOT COMPILE  
    }  
}
```

English: In this example, the classes `Fish` and `Bird` are not related through any class hierarchy that the compiler is aware of; therefore, the code will not compile. While they both extend `Object` implicitly, they are considered unrelated types since one cannot be a subtype of the other. **Türkçe:** Bu örnekte `Fish` ve `Bird` class'ları compiler'ın bildiği herhangi bir class hierarchy üzerinden ilişkili değildir; bu nedenle kod derlenmez. Her ikisi de implicit olarak `Object` extend etse de biri diğerinin subtype'ı olamayacağı için unrelated type kabul edilirler.

Casting Interfaces

English: While the compiler can enforce rules about casting to unrelated types for classes, it cannot always do the same for interfaces. Remember, instances support multiple inheritance, which limits what the compiler can reason about them. While a given class may not implement an interface, it's possible that some subclass may implement the interface. When holding a reference to a particular class, the compiler doesn't know which specific subtype it is holding. **Türkçe:** Compiler, class'lar için unrelated type'lara cast kurallarını uygulayabilse de interface'ler için her zaman aynısını yapamaz. Instance'ların multiple inheritance desteklediğini unutmayın; bu durum compiler'ın onlar hakkında çıkarım yapabileceklerini sınırlar. Belirli bir class bir interface'i implement etmese bile bazı subclass'larının bu interface'i implement etmesi mümkündür. Belirli bir class'a reference tutulduğunda compiler, gerçekte hangi specific subtype'ın tutulduğunu bilmez.

English: Let's try an example. Do you think the following program compiles? **Türkçe:** Bir örnek deneyelim. Aşağıdaki program sizce derlenir mi?

```
interface Canine {}  
interface Dog {}  
class Wolf implements Canine {}  
  
public class BadCasts {  
    public static void main(String[] args) {  
        Wolf wolffy = new Wolf();  
        Dog badWolf = (Dog) wolffy;  
    }  
}
```

English: In this program, a `Wolf` object is created and then assigned to a `Wolf` reference type on line 7. With interfaces, the compiler has limited ability to enforce many rules because even though a reference type may not implement an interface, one of its subclasses could. Therefore, it allows the invalid cast to the `Dog` reference type on line 8, even though `Dog` and `Wolf` are not related. Fear not, even though the code compiles, it still throws a `ClassCastException` at runtime. **Türkçe:** Bu programda line 7'de bir `Wolf` object oluşturulur ve ardından `Wolf` reference type'a atanır. Interface'lerde compiler'ın birçok kuralı uygulama yeteneği sınırlıdır; çünkü bir reference type bir interface'i implement etmese bile subclass'larından biri implement edebilir. Bu nedenle `Dog` ve `Wolf` ilişkili olmadığı halde line 8'de `Dog` reference type'a yapılan geçersiz cast'e izin verir. Endişelenmeyin: Kod derlense de runtime'da yine `ClassCastException` fırlatır.

English: This limitation aside, the compiler can enforce one rule around interface casting. The compiler does not allow a cast from an interface reference to an object reference if the object type cannot possibly implement the interface, such as if the class is marked `final`. For example, if the `Wolf` interface is marked `final` on line 3, then line 8 no longer compiles. The compiler recognizes that there are no possible subclasses of `Wolf` capable of implementing the `Dog` interface. **Türkçe:** Bu sınırlama bir yana, compiler interface casting çevresinde bir kuralı uygulayabilir. Kaynak metne göre, object type'ın interface'i implement etmesi mümkün değilse—örneğin class `final` işaretlenmişse—compiler interface reference'tan object reference'a cast işlemine izin vermez. Örneğin kaynak metin, line 3'teki `Wolf` interface `final` işaretlenirse line 8'in artık derlenmeyeceğini söyler. Compiler, `Dog` interface'ini implement edebilecek olası bir `Wolf` subclass'ı bulunmadığını anlar.

OCP teknik düzeltme: Kaynak paragrafta iki ifade ters/yanlış yazılmıştır: `Wolf` bir interface değil class'tır; örnekteki cast de `Wolf` object reference'ından `Dog` interface reference'ına doğrudur. Doğru Java 17 kuralı şudur: `final Wolf, Dog` interface'ini implement etmiyorsa hiçbir subclass bunu sonradan sağlayamaz; bu nedenle `(Dog) wolfy` compile time'da reddedilir.

The instanceof Operator

English: In Chapter 3, we presented the `instanceof` operator with pattern matching. The `instanceof` operator can be used to check whether an object belongs to a particular class or interface and to prevent a `ClassCastException` at runtime. Consider the following example: **Türkçe:** Chapter 3'te `instanceof` operator'ünü pattern matching ile sunduk. `instanceof` operator'ü, bir object'in belirli bir class veya interface'e ait olup olmadığını kontrol etmek ve runtime'da `ClassCastException` oluşmasını önlemek için kullanılabilir. Aşağıdaki örneği inceleyin:

```
1: class Rodent {}
2:
3: public class Capybara extends Rodent {
4:     public static void main(String[] args) {
5:         Rodent rodent = new Rodent();
6:         var capybara = (Capybara) rodent; // ClassCastException
7:     }
8: }
```

English: This program throws an exception on line 6. We can replace line 6 with the following: **Türkçe:** Program line 6'da exception fırlatır. Line 6 şu kodla değiştirilebilir:

```
6: if (rodent instanceof Capybara c) {
7:     // Do stuff
8: }
```

English: Now the code snippet doesn't throw an exception at runtime and performs the cast only if the `instanceof` operator is successful. **Türkçe:** Artık code snippet runtime'da exception fırlatmaz ve cast işlemini yalnızca `instanceof` operator'ü başarılıysa gerçekleştirir.

English: Just as the compiler does not allow casting an object to unrelated types, it also does not allow `instanceof` to be used with unrelated types. We can demonstrate this with our unrelated `Bird` and `Fish` classes: **Türkçe:** Compiler bir object'in unrelated type'lara cast edilmesine izin vermediği gibi `instanceof` operator'ünün unrelated type'larla kullanılmasına da izin vermez. Bunu unrelated `Bird` ve `Fish` class'larımızla gösterebiliriz:

```
public class Bird {}

public class Fish {
    public static void main(String[] args) {
        Fish fish = new Fish();
        if (fish instanceof Bird b) { // DOES NOT COMPILE
            // Do stuff
        }
    }
}
```

Polymorphism and Method Overriding

English: In Java, polymorphism states that when you override a method, you replace all calls to it, even those defined in the parent class. As an example, what do you think the following code snippet outputs?

Türkçe: Java'da polymorphism, bir method'u override ettiğinizde parent class'ta tanımlananlar dahil ona yapılan bütün çağrılarının yerine yeni implementation'ın geçtiğini söyler. Örnek olarak aşağıdaki code snippet sizce ne yazdırır?

```
class Penguin {
    public int getHeight() { return 3; }
```

```
    public void printInfo() {
        System.out.print(this.getHeight());
    }
}

public class EmperorPenguin extends Penguin {
    public int getHeight() { return 8; }

    public static void main(String[] fish) {
        new EmperorPenguin().printInfo();
    }
}
```

English: If you said 8, then you are well on your way to understanding polymorphism. In this example, the object being operated on in memory is an `EmperorPenguin`. The `getHeight()` method is overridden in the subclass, meaning all calls to it are replaced at runtime. Despite `printInfo()` being defined in the `Penguin` class, calling `getHeight()` on the object calls the method associated with the precise object in memory, not the current reference type where it is called. Even using the `this` reference, which is optional in this example, does not call the parent version because the method has been replaced. **Türkçe:** Cevabınız 8 ise polymorphism'i anlama yolunda oldukça iyi ilerliyorsunuz. Bu örnekte memory'de üzerinde işlem yapılan object bir `EmperorPenguin`'dir. `getHeight()` method'u subclass'ta override edildiği için ona yapılan bütün çağrılarının hedefi runtime'da değiştirilir. `printInfo()` `Penguin` class'ında tanımlanmış olsa da object üzerinde `getHeight()` çağrısı, çağrının yapıldığı current reference type'a değil, memory'deki precise object ile ilişkili method'a gider. Bu örnekte optional olan `this` reference'ını kullanmak bile parent sürümü çağırmaz; çünkü method değiştirilmiştir.

English: Polymorphism's ability to replace methods at runtime via overriding is one of the most important properties of Java. It allows you to create complex inheritance models with subclasses that have their own custom implementation of overridden methods. It also means the parent class does not need to be updated to use the custom or overridden method. If the method is properly overridden, then the overridden version will be used in all places that it is called. **Türkçe:** Polymorphism'in overriding yoluyla method'ları runtime'da değiştirebilmesi Java'nın en önemli özelliklerinden biridir. Overridden method'lar için kendi custom implementation'larına sahip subclass'larla complex inheritance modelleri oluşturmanızı sağlar. Bu aynı zamanda custom veya overridden method'u kullanabilmesi için parent class'ın güncellenmesine gerek olmadığı anlamına gelir. Method doğru biçimde override edilmişse çağrıldığı her yerde overridden sürüm kullanılır.

English: Remember, you can choose to limit polymorphic behavior by marking methods `final`, which prevents them from being overridden by a subclass. **Türkçe:** Method'ları `final` işaretleyerek polymorphic davranışı sınırlamayı seçebileceğinizi unutmayın; bu, method'ların bir subclass tarafından override edilmesini önler.

Calling the Parent Version of an Overridden Method

English: Just because a method is overridden doesn't mean the parent method is completely inaccessible. We can use the `super` reference that you learned about in Chapter 6 to access it. How can you modify our previous example to print 3 instead of 8? You could try calling `super.getHeight()` in the parent `Penguin` class. **Türkçe:** Bir method'un override edilmesi parent method'un tamamen erişilemez olduğu anlamına gelmez. Ona erişmek için Chapter 6'da öğrendiğiniz `super` reference'ını kullanabiliriz. Önceki örneğimizi 8 yerine 3 yazdıracak şekilde nasıl değiştirebilirsiniz? Parent `Penguin` class'ında `super.getHeight()` çağırmaı deneyebilirsiniz:

```
class Penguin {
    public int getHeight() { return 3; }

    public void printInfo() {
        System.out.print(super.getHeight()); // DOES NOT COMPILE
    }
}
```

English: Unfortunately, this does not compile, as `super` refers to the superclass of `Penguin`; in this case, `Object`. The solution is to override `printInfo()` in the child `EmperorPenguin` class and use `super` there. **Türkçe:** Ne yazık ki bu kod derlenmez; çünkü `super`, `Penguin`'in superclass'ına, bu durumda `Object`'e referans verir. Çözüm, child `EmperorPenguin` class'ında `printInfo()` method'unu override edip `super`'i orada kullanmaktır.

```
public class EmperorPenguin extends Penguin {
    public int getHeight() { return 8; }

    public void printInfo() {
        System.out.print(super.getHeight());
    }

    public static void main(String[] fish) {
        new EmperorPenguin().printInfo(); // 3
    }
}
```

Overriding vs. Hiding Members

English: While method overriding replaces the method everywhere it is called, static method and variable hiding do not. Strictly speaking, hiding members is not a form of polymorphism since the methods and variables maintain their individual properties. Unlike method overriding, hiding members is very sensitive to the reference type and location where the member is being used. **Türkçe:** Method overriding çağrıldığı her yerde method'un yerine geçerken static method ve variable hiding aynı etkiyi göstermez. Kesin olarak konuşursak member hiding bir polymorphism biçimi değildir; çünkü method ve variable'lar kendi ayrı property'lerini korur. Method overriding'in aksine hiding, reference type'a ve member'ın kullanılmakta olduğu location'a çok duyarlıdır.

English: Let's take a look at an example: **Türkçe:** Bir örneğe bakalım:

```
class Penguin {
    public static int getHeight() { return 3; }

    public void printInfo() {
        System.out.println(this.getHeight());
    }
}

public class CrestedPenguin extends Penguin {
    public static int getHeight() { return 8; }

    public static void main(String... fish) {
        new CrestedPenguin().printInfo();
    }
}
```

English: The `CrestedPenguin` example is nearly identical to our previous `EmperorPenguin` example, although as you probably already guessed, it prints 3 instead of 8. The `getHeight()` method is static and is therefore hidden, not overridden. The result is that calling `getHeight()` in `CrestedPenguin` returns a different value than calling it in `Penguin`, even if the underlying object is the same. **Türkçe:** `CrestedPenguin` örneği önceki `EmperorPenguin` örneğimize neredeyse aynıdır; ancak muhtemelen tahmin ettiğiniz gibi 8 yerine 3 yazdırır. `getHeight()` method'u static'tir ve bu nedenle override edilmez, hide edilir. Sonuç olarak underlying object aynı olsa bile `CrestedPenguin` içinde `getHeight()` çağırmak, onu `Penguin` içinde çağırmaktan farklı bir değer döndürür.

English: Contrast this with overriding a method, where it returns the same value for an object regardless of which class it is called in. **Türkçe:** Bunu, hangi class içinde çağrılırsa çağrılсын aynı object için aynı değeri döndüren overridden method davranışıyla karşılaştırın.

English: What about the fact that we used `this` to access a static method in `this.getHeight()`? As discussed in Chapter 5, while you are permitted to use an instance reference to access a static variable or method, doing so is often discouraged. The compiler will warn you when you access static members in a non-static way. In this case, the `this` reference had no impact on the program output. **Türkçe:** Peki `this.getHeight()` içinde static method'a erişmek için `this` kullanmış olmamız ne olacak? Chapter 5'te anlatıldığı gibi static variable veya method'a instance reference üzerinden erişmenize izin verilir; fakat bunu yapmak çoğu zaman önerilmez. Static member'lara non-static biçimde eriştiğinizde compiler sizi uyarır. Bu durumda `this` reference'ının program çıktısı üzerinde hiçbir etkisi yoktur.

English: Besides the location, the reference type can also determine the value you get when you are working with hidden members. Ready? Let's try a more complex example: **Türkçe:** Location'ın yanı sıra reference type da hidden member'larla çalışırken elde edeceğiniz değeri belirleyebilir. Hazır mısınız? Daha karmaşık bir örnek deneyelim:

```
class Marsupial {
    protected int age = 2;

    public static boolean isBiped() {
        return false;
    }
}

public class Kangaroo extends Marsupial {
    protected int age = 6;

    public static boolean isBiped() {
        return true;
    }

    public static void main(String[] args) {
        Kangaroo joey = new Kangaroo();
        Marsupial moey = joey;

        System.out.println(joey.isBiped());
        System.out.println(moey.isBiped());
        System.out.println(joey.age);
        System.out.println(moey.age);
    }
}
```

English: The program prints the following: **Türkçe:** Program aşağıdakileri yazdırır:

```
true
false
6
2
```

English: In this example, only one object (of type `Kangaroo`) is created and stored in memory! Since static methods can only be hidden, not overridden, Java uses the reference type to determine which version of `isBiped()` should be called, resulting in `joey.isBiped()` printing true and `moey.isBiped()` printing false. **Türkçe:** Bu örnekte yalnızca tek bir object (`Kangaroo` type'ında) oluşturulur ve memory'de saklanır! Static method'lar override edilemeyip yalnızca hide edilebildiğinden Java, `isBiped()` method'unun hangi sürümünün çağırılması gerektiğini reference type'a göre belirler. Bunun sonucunda `joey.isBiped()` true, `moey.isBiped()` ise false yazdırır.

English: Likewise, the `age` variable is hidden, not overridden, so the reference type is used to determine which value to output. This results in `joey.age` returning 6 and `moey.age` returning 2. **Türkçe:** Benzer biçimde `age` variable'ı override edilmez, hide edilir; bu nedenle hangi değerin çıktıya verileceğini belirlemek için reference type kullanılır. Sonuç olarak `joey.age` 6, `moey.age` ise 2 döndürür.

English: For the exam, make sure you understand these examples, as they show how hidden and overridden methods are fundamentally different. In practice, overriding methods is the cornerstone of polymorphism and an extremely powerful feature. **Türkçe:** Hidden ve overridden method'ların temelde nasıl farklı olduğunu gösterdikleri için sınav açısından bu örnekleri anladığınızdan emin olun. Uygulamada method overriding, polymorphism'in temel taşı ve son derece güçlü bir özelliktir.

Don't Hide Members in Practice / Uygulamada Member'ları Hide Etmeyin

English: Although Java allows you to hide variables and static methods, it is considered an extremely poor coding practice. As you saw in the previous example, the value of the variable or method can change depending on what reference is used, making your code very confusing, difficult to follow, and challenging for others to maintain. This is further compounded when you start modifying the value of the variable in both the parent and child methods, since it may not be clear which variable you're updating. **Türkçe:** Java variable'ları ve static method'ları hide etmenize izin verse de bu, son derece kötü bir coding practice kabul edilir. Önceki örnekte gördüğünüz gibi variable veya method'un değeri kullanılan reference'a göre değişebilir; bu da kodunuzu çok kafa karıştırıcı, izlenmesi zor ve başkaları için bakımı güç hâle getirir. Hem parent hem child method'larda variable'ın değerini değiştirmeye başladığınızda hangi variable'ı güncellediğiniz açık olmayabileceği için sorun daha da büyür.

English: When you're defining a new variable or static method in a child class, it is considered good coding practice to select a name that is not already used by an inherited member. Redeclaring private methods and variables is considered less problematic, though, because the child class does not have access to the variable in the parent class to begin with. **Türkçe:** Bir child class'ta yeni bir variable veya static method tanımlarken inherited bir member tarafından zaten kullanılmayan bir ad seçmek iyi bir coding practice kabul edilir. Bununla birlikte private method ve variable'ları yeniden bildirmek daha az sorunlu sayılır; çünkü child class en başta parent class'taki variable'a erişemez.

Summary / Özet

English: In this chapter, we presented numerous topics in advanced object-oriented design, covering many top-level types beyond classes. We started with interfaces and described how they can support multiple inheritance. Remember, interfaces and their members can include a number of implicit modifiers inserted by the compiler automatically. We then covered all six types of interface members you need to know for the exam: abstract methods, static constants, default methods, static methods, private methods, and private static methods. **Türkçe:** Bu chapter'da ileri object-oriented design kapsamındaki pek çok konuyu sunduk ve class'ların ötesindeki birçok top-level type'ı ele aldık. Interface'lerle başlayıp bunların multiple inheritance'ı nasıl destekleyebildiğini açıkladık. Interface'lerin ve member'larının compiler tarafından otomatik eklenen çeşitli implicit modifier'lar içerebildiğini unutmayın. Ardından sınav için bilmeniz gereken altı interface member türünün tamamını ele aldık: abstract method'lar, static constant'lar, default method'lar, static method'lar, private method'lar ve private static method'lar.

English: We next moved on to enums, which are compile-time constant properties. Simple enums are composed of a list of values, while complex enums can include constructors, methods, and fields. Enums can also be used in `switch` statements and expressions. When an enum method is marked abstract, each enum value must provide an implementation. **Türkçe:** Sonra compile-time constant property'leri olan enum'lara geçtik. Basit enum'lar bir value listesinden oluşurken karmaşık enum'lar constructor, method ve field içerebilir. Enum'lar `switch` statement ve expression'larında da kullanılabilir. Bir enum method'u abstract olarak işaretlendiğinde her enum value bir implementation sağlamalıdır.

English: Moving on to new topics in Java, we covered sealed classes and how they allow classes to function like enumerated types in which only certain subclasses are permitted. For the exam, **Türkçe:** Java'daki yeni konulara geçerek sealed class'ları ve bunların, yalnızca belirli subclass'lara izin verilen enumerated type'lar gibi çalışmasını nasıl sağladığını ele aldık. Sınav açısından,

English: it's important to remember that the subclasses of a sealed class must be marked `final`, `sealed`, or `non-sealed`. If the subclasses of the sealed class are defined in the same file, then the `permits` clause may be omitted in the sealed class declaration. Finally, sealed interfaces may be used to limit which classes can implement an interface, which interfaces may extend an interface, or both. **Türkçe:** sealed class'ın subclass'larının `final`, `sealed` veya `non-sealed` olarak işaretlenmesi gerektiğini unutmamak önemlidir. Sealed class'ın subclass'ları aynı file'da tanımlanırsa sealed class declaration'ında `permits` clause'u atlanabilir. Son olarak sealed interface'ler, bir interface'i hangi class'ların implement edebileceğini, hangi interface'lerin extend edebileceğini veya her ikisini birden sınırlamak için kullanılabilir.

English: Records are another new feature available in Java. Records are a compact way of declaring an immutable and encapsulated POJO in which the compiler adds a lot of the boilerplate code for you. Remember, encapsulation is the practice of preventing external callers from accessing the internal components of an object. Records include automatic creation of the accessor methods, a long constructor, and useful implementations of `equals()`, `hashCode()`, and `toString()`. Records can include overloaded and compact constructors to support data validation and transformation. Records do not permit instance variables, since this could break immutability, but they do allow methods, static members, and nested types. **Türkçe:** Record'lar Java'da bulunan bir başka yeni özelliktir. Record'lar, compiler'ın boilerplate code'un büyük bölümünü sizin için eklediği immutable ve encapsulated bir POJO bildiriminin kısa yoludur. Encapsulation'ın, external caller'ların bir object'in internal component'lerine erişmesini engelleme uygulaması olduğunu unutmayın. Record'larda accessor method'ların, long constructor'ın ve yararlı `equals()`, `hashCode()` ve `toString()` implementation'larının otomatik oluşturulması bulunur. Record'lar data validation ve transformation'ı desteklemek için overloaded ve compact constructor'lar içerebilir. Immutability'yi bozabileceğinden instance variable'lara izin vermez; ancak method'lara, static member'lara ve nested type'lara izin verir.

English: We then moved on to nested types. For simplicity, we focused on nested classes and covered each of the four types. An inner class requires an instance of the outer class to use, while a static nested class does not. A local class is commonly defined within a method or block. Local classes can only access local variables that are final and effectively final. Anonymous classes are a special type of local class that does not have a name. Anonymous classes are required to extend exactly one class or implement one interface. Inner, local, and anonymous classes can access private members of the class in which they are defined, provided the latter two are used inside an instance method. **Türkçe:** Ardından nested type'lara geçtik. Basitlik için nested class'lara odaklanıp dört türün her birini ele aldık. Inner class'ın kullanılabilmesi için outer class'ın bir instance'ı gerekirken static nested class'ta bu gerekmez. Local class genellikle bir method veya block içinde tanımlanır. Local class'lar yalnız final ve effectively final local variable'lara erişebilir. Anonymous class'lar adı olmayan özel bir local class türüdür. Anonymous class'ların tam olarak bir class'ı extend etmesi veya bir interface'i implement etmesi gerekir. Inner, local ve anonymous class'lar, tanımlandıkları class'ın private member'larına erişebilir; son ikisinin bir instance method içinde kullanılması gerekir.

English: We concluded this chapter with a discussion of polymorphism, which is central to the Java language, and showed how objects can be accessed in a variety of forms. Make sure you understand when casts are needed for accessing objects, and be able to spot the difference between compile-time and runtime cast problems. **Türkçe:** Bu chapter'ı Java dilinin merkezinde yer alan polymorphism tartışmasıyla bitirdik ve object'lere çeşitli biçimlerde nasıl erişilebildiğini gösterdik. Object'lere erişirken cast'in ne zaman gerektiğini anladığınızdan ve compile-time ile runtime cast sorunları arasındaki farkı ayırt edebildiğinizden emin olun.

Exam Essentials / Sınav İçin Temel Noktalar

English: ****Be able to write code that creates, extends, and implements interfaces.**** Interfaces are specialized abstract types that focus on abstract methods and constant variables. An interface may extend any number of interfaces and, in doing so, inherits their abstract methods. An interface cannot extend a class, nor can a class extend an interface. A class may implement any number of interfaces. **Türkçe:** ****Interface oluşturan, extend ve implement eden kod yazabilin.**** Interface'ler abstract method'lara ve constant variable'lara odaklanan özelleşmiş abstract type'lardır. Bir interface herhangi sayıda interface'i extend edebilir ve bunu yaparken onların abstract method'larını inherit eder. Interface bir class'ı extend edemez; class da interface'i extend edemez. Bir class herhangi sayıda interface'i implement edebilir.

English: ****Know which interface methods an interface method can reference.**** Non-static private, default, and abstract interface methods are associated with an instance of an interface. Non-static private and default interface methods may reference any method within the interface declaration. Alternatively, static interface methods are associated with class membership and can only reference other static members. Finally, private methods can only be referenced within the interface declaration. **Türkçe:** ****Bir interface method'unun hangi interface method'larına reference verebileceğini bilin.**** Non-static private, default ve abstract interface method'ları bir interface instance'ıyla ilişkilidir. Non-static private ve default interface method'ları interface declaration'ındaki her method'a reference verebilir. Buna karşılık static interface method'ları class membership ile ilişkilidir ve yalnız diğer static member'lara reference verebilir. Son olarak private method'lara yalnız interface declaration'ı içinden reference verilebilir.

English: **Be able to create and use enum types.** An enum is a data structure that defines a list of values. If the enum does not contain any other elements, the semicolon (;) after the values is optional. An enum can be used in `switch` statements and contain instance variables, constructors, and methods. Enum constructors are implicitly private. Enums can include methods, both as members or within individual enum values. If the enum declares an abstract method, each enum value must implement it. **Türkçe:** **Enum type'ları oluşturup kullanabilin.** Enum, bir value listesi tanımlayan data structure'dır. Enum başka element içermiyorsa value'lardan sonraki semicolon (;) isteğe bağlıdır. Enum `switch` statement'larında kullanılabilir ve instance variable, constructor ve method içerebilir. Enum constructor'ları implicit olarak private'dır. Enum'lar hem member olarak hem de tek tek enum value'ların içinde method içerebilir. Enum abstract bir method bildirirse her enum value onu implement etmelidir.

English: ****Be able to recognize when sealed classes are being correctly used.**** A sealed class is one that defines a list of permitted subclasses that extend it. Be able to use the correct modifier (`final`, `sealed`, or `non-sealed`) with sealed classes. Understand when the `permits` clause may be excluded. **Türkçe:** ****Sealed class'ların ne zaman doğru kullanıldığını tanıyabilin.**** Sealed class, kendisini extend eden permitted subclass'ların listesini tanımlar. Sealed class'larla doğru modifier'ı (`final`, `sealed` veya `non-sealed`) kullanabilin. `permits` clause'unun ne zaman dışarıda bırakılabileceğini anlayın.

English: **Identify properly encapsulated classes.** Instance variables in encapsulated classes are private. All code that retrieves the value or updates it uses methods. Encapsulated classes may include accessor (getter) or mutator (setter) methods, although this is not required. **Türkçe:** **Doğru biçimde encapsulated class'ları belirleyin.** Encapsulated class'lardaki instance variable'lar private'dır. Değeri alan veya güncelleyen bütün kod method kullanır. Zorunlu olmamakla birlikte encapsulated class'lar accessor (getter) veya mutator (setter) method'ları içerebilir.

English: ****Understand records and know which members the compiler is adding automatically.**** Records are encapsulated and immutable types in which the compiler inserts a long constructor, accessor methods, and useful implementations of `equals()`, `hashCode()`, and `toString()`. Each of these elements may be overridden. Be able to recognize compact constructors and know that they are used only for validation and transformation of constructor parameters, not for accessing fields. Recognize that when a record is declared with an instance member, it does not compile. **Türkçe:** ****Record'ları anlayın ve compiler'ın hangi member'ları otomatik eklediğini bilin.**** Record'lar compiler'ın long constructor, accessor method'lar ve yararlı `equals()`, `hashCode()` ve `toString()` implementation'ları eklediği encapsulated ve immutable type'lardır. Bu element'lerin her biri override edilebilir. Compact constructor'ları tanıyabilin ve bunların field'lara erişmek için değil, yalnız constructor parameter'larını validation ve transformation amacıyla kullanıldığını bilin. Bir record instance member ile bildirildiğinde derlenmediğini fark edin.

English: **Be able to declare and use nested classes.** There are four types of nested types: inner classes, static classes, local classes, and anonymous classes. Instantiating an inner class requires an instance of the outer class. On the other hand, static nested classes can be created without a reference to the outer class. Local and anonymous classes cannot be declared with an access modifier. Anonymous classes are limited to extending a single class or implementing one interface. **Türkçe:** **Nested class'ları bildirip kullanabilin.** Dört nested type türü vardır: inner class'lar, static class'lar, local class'lar ve anonymous class'lar. Inner class oluşturmak outer class'ın bir instance'ını gerektirir. Buna karşılık static nested class'lar outer class'a bir reference olmadan oluşturulabilir. Local ve anonymous class'lar access modifier ile bildirilemez. Anonymous class'lar tek bir class'ı extend etmekle veya bir interface'i implement etmekle sınırlıdır.

English: **Understand polymorphism.** An object may take on a variety of forms, referred to as polymorphism. The object is viewed as existing in memory in one concrete form but is accessible in many forms through reference variables. Changing the reference type of an object may grant access to new members, but the members always exist in memory. **Türkçe:** **Polymorphism'i anlayın.** Bir object, polymorphism denen çeşitli biçimler alabilir. Object memory'de tek bir concrete form'da bulunur fakat reference variable'lar üzerinden birçok biçimde erişilebilir. Object'ın reference type'ını değiştirmek yeni member'lara erişim sağlayabilir; ancak member'lar memory'de her zaman mevcuttur.

Review Questions / Gözden Geçirme Soruları

English: The answers to the chapter review questions can be found in the Appendix. **Türkçe:** Chapter review questions'ın cevapları Appendix'te bulunabilir.

Question 1

English: Which of the following are valid record declarations? (Choose all that apply.) **Türkçe:** Aşağıdakilerden hangileri geçerli record declaration'larıdır? (Uygun olanların tümünü seçin.)

```

public record Iguana(int age) {
    private static final int age = 10;
}

public final record Gecko() {}

public abstract record Chameleon() {
    private static String name;
}

public record BeardedDragon(boolean fun) {
    @Override public boolean fun() { return false; }
}

public record Newt(long size) {
    @Override public boolean equals(Object obj) { return false; }
    public void setSize(long size) {
        this.size = size;
    }
}

```

English — A: `Iguana` **Türkçe — A:** `Iguana`

English — B: `Gecko` **Türkçe — B:** `Gecko`

English — C: `Chameleon` **Türkçe — C:** `Chameleon`

English — D: `BeardedDragon` **Türkçe — D:** `BeardedDragon`

English — E: `Newt` **Türkçe — E:** `Newt`

English — F: None of the above **Türkçe — F:** Yukarıdakilerin hiçbiri

Question 2

English: Which of the following statements can be inserted in the blank line so that the code will compile successfully? (Choose all that apply.) **Türkçe:** Kodun başarıyla derlenmesi için aşağıdaki statement'lardan hangileri boş satıra eklenebilir? (Uygun olanların tümünü seçin.)

```

interface CanHop {}

public class Frog implements CanHop {
    public static void main(String[] args) {
        _____ frog = new TurtleFrog();
    }
}

class BrazilianHornedFrog extends Frog {}
class TurtleFrog extends Frog {}

```

English — A: `Frog` **Türkçe — A:** `Frog`

English — B: `TurtleFrog` **Türkçe — B:** `TurtleFrog`

English — C: `BrazilianHornedFrog` **Türkçe — C:** `BrazilianHornedFrog`

English — D: `CanHop` **Türkçe — D:** `CanHop`

English — E: `var` **Türkçe — E:** `var`

English — F: `Long` **Türkçe — F:** `Long`

English — G: None of the above; the code contains a compilation error. **Türkçe — G:** Yukarıdakilerin hiçbiri; kod bir compilation error içerir.

Question 3

English: What is the result of the following program? **Türkçe:** Aşağıdaki programın sonucu nedir?

```
public class Favorites {
    enum Flavors {
        VANILLA, CHOCOLATE, STRAWBERRY
        static final Flavors DEFAULT = STRAWBERRY;
    }

    public static void main(String[] args) {
        for (final var e : Flavors.values())
            System.out.print(e.ordinal() + " ");
    }
}
```

English — A: `0 1 2` **Türkçe — A:** `0 1 2`

English — B: `1 2 3` **Türkçe — B:** `1 2 3`

English — C: Exactly one line of code does not compile. **Türkçe — C:** Kodun tam olarak bir satırı derlenmez.

English — D: More than one line of code does not compile. **Türkçe — D:** Kodun birden fazla satırı derlenmez.

English — E: The code compiles but produces an exception at runtime. **Türkçe — E:** Kod derlenir ancak runtime'da exception üretir.

English — F: None of the above **Türkçe — F:** Yukarıdakilerin hiçbiri

Question 4

English: What is the output of the following program? **Türkçe:** Aşağıdaki programın çıktısı nedir?

```
public sealed class ArmoredAnimal permits Armadillo {
    public ArmoredAnimal(int size) {}
    @Override public String toString() { return "Strong"; }

    public static void main(String[] a) {
        var c = new Armadillo(10, null);
        System.out.println(c);
    }
}

class Armadillo extends ArmoredAnimal {
    @Override public String toString() { return "Cute"; }
    public Armadillo(int size, String name) {
        super(size);
    }
}
```

English — A: Strong **Türkçe — A:** Strong

English — B: Cute **Türkçe — B:** Cute

English — C: The program does not compile. **Türkçe — C:** Program derlenmez.

English — D: The code compiles but produces an exception at runtime. **Türkçe — D:** Kod derlenir ancak runtime'da exception üretir.

English — E: None of the above **Türkçe — E:** Yukarıdakilerin hiçbiri

Question 5

English: Which statements about the following program are correct? (Choose all that apply.) **Türkçe:** Aşağıdaki programla ilgili hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

```
1: interface HasExoskeleton {
2:     double size = 2.0f;
3:     abstract int getNumberOfSections();
4: }
5: abstract class Insect implements HasExoskeleton {
6:     abstract int getNumberOfLegs();
7: }
8: public class Beetle extends Insect {
9:     int getNumberOfLegs() { return 6; }
10:    int getNumberOfSections(int count) { return 1; }
11: }
```

English — A: It compiles without issue. **Türkçe — A:** Sorunsuz derlenir.

English — B: The code will produce a `ClassCastException` if called at runtime. **Türkçe — B:** Kod runtime'da çağrılırsa `ClassCastException` üretir.

English — C: The code will not compile because of line 2. **Türkçe — C:** Kod line 2 nedeniyle derlenmez.

English — D: The code will not compile because of line 5. **Türkçe — D:** Kod line 5 nedeniyle derlenmez.

English — E: The code will not compile because of line 8. **Türkçe — E:** Kod line 8 nedeniyle derlenmez.

English — F: The code will not compile because of line 10. **Türkçe — F:** Kod line 10 nedeniyle derlenmez.

Question 6

English: Which statements about the following program are correct? (Choose all that apply.) **Türkçe:** Aşağıdaki programla ilgili hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

```
1: public abstract interface Herbivore {  
2:     int amount = 10;  
3:     public void eatGrass();  
4:     public abstract int chew() { return 13; }  
5: }  
6:  
7: abstract class IsAPlant extends Herbivore {  
8:     Object eatGrass(int season) { return null; }  
9: }
```

English — A: It compiles and runs without issue. **Türkçe — A:** Sorunsuz derlenir ve çalışır.

English — B: The code will not compile because of line 1. **Türkçe — B:** Kod line 1 nedeniyle derlenmez.

English — C: The code will not compile because of line 2. **Türkçe — C:** Kod line 2 nedeniyle derlenmez.

English — D: The code will not compile because of line 4. **Türkçe — D:** Kod line 4 nedeniyle derlenmez.

English — E: The code will not compile because of line 7. **Türkçe — E:** Kod line 7 nedeniyle derlenmez.

English — F: The code will not compile because line 8 contains an invalid method override. **Türkçe — F:** Line 8 geçersiz bir method override içerdiği için kod derlenmez.

Question 7

English: What is the output of the following program? **Türkçe:** Aşağıdaki programın çıktısı nedir?

```

1: interface Aquatic {
2:     int getNumOfGills(int p);
3: }
4: public class ClownFish implements Aquatic {
5:     String getNumOfGills() { return "14"; }
6:     int getNumOfGills(int input) { return 15; }
7:     public static void main(String[] args) {
8:         System.out.println(new ClownFish().getNumOfGills(-1));
9:     }
10: }

```

English — A: 14 **Türkçe — A:** 14

English — B: 15 **Türkçe — B:** 15

English — C: The code will not compile because of line 4. **Türkçe — C:** Kod line 4 nedeniyle derlenmez.

English — D: The code will not compile because of line 5. **Türkçe — D:** Kod line 5 nedeniyle derlenmez.

English — E: The code will not compile because of line 6. **Türkçe — E:** Kod line 6 nedeniyle derlenmez.

English — F: None of the above **Türkçe — F:** Yukarıdakilerin hiçbirisi

Question 8

English: When inserted in order, which modifiers can fill in the blanks to create a properly encapsulated class? (Choose all that apply.) **Türkçe:** Sırayla eklendiğinde hangi modifier'lar boşlukları doldurarak doğru biçimde encapsulated bir class oluşturabilir? (Uygun olanların tümünü seçin.)

```

public class Rabbits {
    _____ int numRabbits = 0;
    _____ void multiply() {
        numRabbits *= 6;
    }
    _____ int getNumberOfRabbits() {
        return numRabbits;
    }
}

```

English — A: private, public, and public **Türkçe — A:** private, public ve public

English — B: private, protected, and private **Türkçe — B:** private, protected ve private

English — C: private, private, and protected **Türkçe — C:** private, private ve protected

English — D: public, public, and public **Türkçe — D:** public, public ve public

English — E: The class cannot be properly encapsulated since `multiply()` does not begin with `set`.
Türkçe — E: `multiply()` adı `set` ile başlamadığı için class doğru biçimde encapsulated olamaz.

English — F: None of the above **Türkçe — F:** Yukarıdakilerin hiçbiri

Question 9

English: Which of the following statements can be inserted in the blank so that the code will compile successfully? (Choose all that apply.) **Türkçe:** Kodun başarıyla derlenmesi için aşağıdaki statement'lardan hangileri boşluğa eklenebilir? (Uygun olanların tümünü seçin.)

```
abstract class Snake {}
class Cobra extends Snake {}
class GardenSnake extends Cobra {}

public class SnakeHandler {
    private Snake snakey;
    public void setSnake(Snake mySnake) { this.snakey = mySnake; }
    public static void main(String[] args) {
        new SnakeHandler().setSnake(______);
    }
}
```

English — A: `new Cobra()` **Türkçe — A:** `new Cobra()`

English — B: `new Snake()` **Türkçe — B:** `new Snake()`

English — C: `new Object()` **Türkçe — C:** `new Object()`

English — D: `new String("Snake")` **Türkçe — D:** `new String("Snake")`

English — E: `new GardenSnake()` **Türkçe — E:** `new GardenSnake()`

English — F: `null` **Türkçe — F:** `null`

English — G: None of the above. The class does not compile, regardless of the value inserted in the blank. **Türkçe — G:** Yukarıdakilerin hiçbiri. Boşluğa eklenen değerden bağımsız olarak class derlenmez.

Question 10

English: What types can be inserted in the blanks on the lines marked X and Z that allow the code to compile? (Choose all that apply.) **Türkçe:** X ve Z ile işaretli satırlardaki boşluklara hangi type'lar eklendiğinde kod derlenir? (Uygun olanların tümünü seçin.)

```
interface Walk { private static List move() { return null; } }
interface Run extends Walk { public ArrayList move(); }

class Leopard implements Walk {
    public _____ move() { // X
        return null;
    }
}

class Panther implements Run {
    public _____ move() { // Z
        return null;
    }
}
```

English — A: `Integer` on the line marked X **Türkçe — A:** X ile işaretli satırda `Integer`

English — B: `ArrayList` on the line marked X **Türkçe — B:** X ile işaretli satırda `ArrayList`

English — C: `List` on the line marked X **Türkçe — C:** X ile işaretli satırda `List`

English — D: `List` on the line marked Z **Türkçe — D:** Z ile işaretli satırda `List`

English — E: `ArrayList` on the line marked Z **Türkçe — E:** Z ile işaretli satırda `ArrayList`

English — F: None of the above, since the `Run` interface does not compile. **Türkçe — F:** `Run` interface'i derlenmediği için yukarıdakilerin hiçbiri.

English — G: The code does not compile for a different reason. **Türkçe — G:** Kod başka bir nedenle derlenmez.

Question 11

English: What is the result of the following code? (Choose all that apply.) **Türkçe:** Aşağıdaki kodun sonucu nedir? (Uygun olanların tümünü seçin.)

```
1: public class Movie {
2:     private int butter = 5;
3:     private Movie() {}
4:     protected class Popcorn {
5:         private Popcorn() {}
6:         public static int butter = 10;
7:         public void startMovie() {
8:             System.out.println(butter);
9:         }
10:    }
11:    public static void main(String[] args) {
12:        var movie = new Movie();
13:        Movie.Popcorn in = new Movie().new Popcorn();
14:        in.startMovie();
15:    }
16: }
```

English — A: The output is 5. **Türkçe — A:** Çıktı 5 olur.

English — B: The output is 10. **Türkçe — B:** Çıktı 10 olur.

English — C: Line 6 generates a compiler error. **Türkçe — C:** Line 6 compiler error üretir.

English — D: Line 12 generates a compiler error. **Türkçe — D:** Line 12 compiler error üretir.

English — E: Line 13 generates a compiler error. **Türkçe — E:** Line 13 compiler error üretir.

English — F: The code compiles but produces an exception at runtime. **Türkçe — F:** Kod derlenir ancak runtime'da exception üretir.

Question 12

English: Which of the following are true about encapsulation? (Choose all that apply.) **Türkçe:** Aşağıdakilerden hangileri encapsulation hakkında doğrudur? (Uygun olanların tümünü seçin.)

English — A: It allows getters. **Türkçe — A:** Getter'lara izin verir.

English — B: It allows setters. **Türkçe — B:** Setter'lara izin verir.

English — C: It requires specific naming conventions. **Türkçe — C:** Belirli naming convention'ları gerektirir.

English — D: It requires public instance variables. **Türkçe — D:** Public instance variable'lar gerektirir.

English — E: It requires private instance variables. **Türkçe — E:** Private instance variable'lar gerektirir.

Question 13

English: What is the result of the following program? **Türkçe:** Aşağıdaki programın sonucu nedir?

```
public class Weather {  
    enum Seasons {  
        WINTER, SPRING, SUMMER, FALL  
    }  
}
```

```
public static void main(String[] args) {  
    Seasons v = null;  
    switch (v) {  
        case Seasons.SPRING -> System.out.print("s");  
        case Seasons.WINTER -> System.out.print("w");  
        case Seasons.SUMMER -> System.out.print("m");  
        default -> System.out.println("missing data");  
    }  
}
```

English — A: s **Türkçe — A:** s

English — B: w **Türkçe — B:** w

English — C: m **Türkçe — C:** m

English — D: missing data **Türkçe — D:** missing data

English — E: Exactly one line of code does not compile. **Türkçe — E:** Kodun tam olarak bir satırı derlenmez.

English — F: More than one line of code does not compile. **Türkçe — F:** Kodun birden fazla satırı derlenmez.

English — G: The code compiles but produces an exception at runtime. **Türkçe — G:** Kod derlenir ancak runtime'da exception üretir.

Question 14

English: Which statements about sealed classes are correct? (Choose all that apply.) **Türkçe:** Sealed class'larla ilgili hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

English — A: A sealed interface restricts which subinterfaces may extend it. **Türkçe — A:** Sealed interface, kendisini hangi subinterface'lerin extend edebileceğini kısıtlar.

English — B: A sealed class cannot be indirectly extended by a class that is not listed in its `permits` clause. **Türkçe — B:** Sealed class, `permits` clause'unda listelenmeyen bir class tarafından dolaylı olarak extend edilemez.

English — C: A sealed class can be extended by an abstract class. **Türkçe — C:** Sealed class abstract bir class tarafından extend edilebilir.

English — D: A sealed class can be extended by a subclass that uses the `non-sealed` modifier. **Türkçe — D:** Sealed class, `non-sealed` modifier'ını kullanan bir subclass tarafından extend edilebilir.

English — E: A sealed interface restricts which subclasses may implement it. **Türkçe — E:** Sealed interface, kendisini hangi subclass'ların implement edebileceğini kısıtlar.

English — F: A sealed class cannot contain any nested subclasses. **Türkçe — F:** Sealed class herhangi bir nested subclass içeremez.

English — G: None of the above **Türkçe — G:** Yukarıdakilerin hiçbirisi

Question 15

English: Which lines, when entered independently into the blank, allow the code to print `Not scared` at runtime? (Choose all that apply.) **Türkçe:** Hangi satırlar boşluğa birbirinden bağımsız olarak girildiğinde kodun runtime'da `Not scared` yazdırmasını sağlar? (Uygun olanların tümünü seçin.)

```
public class Ghost {
    public static void boo() {
        System.out.println("Not scared");
    }
    protected final class Spirit {
        public void boo() {
            System.out.println("Booo!!!");
        }
    }
}
```

```
public static void main(String... haunt) {
    var g = new Ghost().new Spirit() {};
    _____;
}
```

English — A: `g.boo()` **Türkçe — A:** `g.boo()`

English — B: `g.super.boo()` **Türkçe — B:** `g.super.boo()`

English — C: `new Ghost().boo()` **Türkçe — C:** `new Ghost().boo()`

English — D: `g.Ghost.boo()` **Türkçe — D:** `g.Ghost.boo()`

English — E: `new Spirit().boo()` **Türkçe — E:** `new Spirit().boo()`

English — F: `Ghost.boo()` **Türkçe — F:** `Ghost.boo()`

English — G: None of the above **Türkçe — G:** Yukarıdakilerin hiçbiri

Question 16

English: The following code appears in a file named `Ostrich.java`. What is the result of compiling the source file? **Türkçe:** Aşağıdaki kod `Ostrich.java` adlı bir file'da yer almaktadır. Source file derlendiğinde sonuç ne olur?

```
1: public class Ostrich {
2:     private int count;
3:     static class OstrichWrangler {
4:         public int stampede() {
5:             return count;
6:         }
7:     }
8: }
```

English — A: The code compiles successfully, and one bytecode file is generated: `Ostrich.class`.

Türkçe — A: Kod başarıyla derlenir ve tek bir bytecode file üretilir: `Ostrich.class`.

English — B: The code compiles successfully, and two bytecode files are generated: `Ostrich.class`

and `OstrichWrangler.class`. **Türkçe — B:** Kod başarıyla derlenir ve iki bytecode file üretilir:

`Ostrich.class` ve `OstrichWrangler.class`.

English — C: The code compiles successfully, and two bytecode files are generated: `Ostrich.class`

and `Ostrich$OstrichWrangler.class`. **Türkçe — C:** Kod başarıyla derlenir ve iki bytecode file

üretilir: `Ostrich.class` ve `Ostrich$OstrichWrangler.class`.

English — D: A compiler error occurs on line 3. **Türkçe — D:** Line 3'te compiler error oluşur.

English — E: A compiler error occurs on line 5. **Türkçe — E:** Line 5'te compiler error oluşur.

Question 17

English: Which lines of the following interface declarations do not compile? (Choose all that apply.)

Türkçe: Aşağıdaki interface declaration'larının hangi satırları derlenmez? (Uygun olanların tümünü seçin.)

```
1: public interface Omnivore {  
2:     int amount = 10;  
3:     static boolean gather = true;  
4:     static void eatGrass() {}  
5:     int findMore() { return 2; }  
6:     default float rest() { return 2; }  
7:     protected int chew() { return 13; }  
8:     private static void eatLeaves() {}  
9: }
```

English — A: Line 2 **Türkçe — A:** Line 2

English — B: All of the lines compile without issue. **Türkçe — B:** Bütün satırlar sorunsuz derlenir.

English — C: Line 3 **Türkçe — C:** Line 3

English — D: Line 4 **Türkçe — D:** Line 4

English — E: Line 5 **Türkçe — E:** Line 5

English — F: Line 6 **Türkçe — F:** Line 6

English — G: Line 7 **Türkçe — G:** Line 7

English — H: Line 8 **Türkçe — H:** Line 8

Question 18

English: What is printed by the following program? **Türkçe:** Aşağıdaki program ne yazdırır?

```
public class Deer {  
    enum Food { APPLES, BERRIES, GRASS }  
    protected class Diet {  
        private Food getFavorite() {  
            return Food.BERRIES;  
        }  
    }  
    public static void main(String[] seasons) {  
        System.out.print(switch (new Diet().getFavorite()) {  
            case APPLES -> "a";  
            case BERRIES -> "b";  
            default -> "c";  
        });  
    }  
}
```

English — A: a **Türkçe — A:** a

English — B: `b` **Türkçe — B:** `b`

English — C: `c` **Türkçe — C:** `c`

English — D: The code declaration of the `Diet` class does not compile. **Türkçe — D:** `Diet` class'ının code declaration'ı derlenmez.

English — E: The `main()` method does not compile. **Türkçe — E:** `main()` method'u derlenmez.

English — F: The code compiles but produces an exception at runtime. **Türkçe — F:** Kod derlenir ancak runtime'da exception üretir.

English — G: None of the above **Türkçe — G:** Yukarıdakilerin hiçbiri

Question 19

English: Which of the following are printed by the `Bear` program? (Choose all that apply.) **Türkçe:** `Bear` programı aşağıdakilerden hangilerini yazdırır? (Uygun olanların tümünü seçin.)

```
public class Bear {
    enum FOOD {
        BERRIES,
        INSECTS {
            public boolean isHealthy() { return true; }
        },
        FISH, ROOTS, COOKIES, HONEY;

        public abstract boolean isHealthy();
    }
}
```

```
public static void main(String[] args) {
    System.out.print(FOOD.INSECTS);
    System.out.print(FOOD.INSECTS.ordinal());
    System.out.print(FOOD.INSECTS.isHealthy());
    System.out.print(FOOD.COOKIES.isHealthy());
}
}
```

English — A: `insects` **Türkçe — A:** `insects`

English — B: `INSECTS` **Türkçe — B:** `INSECTS`

English — C: `0` **Türkçe — C:** `0`

English — D: `1` **Türkçe — D:** `1`

English — E: `false` **Türkçe — E:** `false`

English — F: `true` **Türkçe — F:** `true`

English — G: The code does not compile. **Türkçe — G:** Kod derlenmez.

Question 20

English: Which statements about polymorphism and method inheritance are correct? (Choose all that apply.) **Türkçe:** Polymorphism ve method inheritance hakkında hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

English — A: Given an arbitrary instance of a class, it cannot be determined until runtime which overridden method will be executed in a parent class. **Türkçe — A:** Bir class'ın herhangi bir instance'ı verildiğinde, parent class'ta hangi overridden method'un çalıştırılacağı runtime'a kadar belirlenemez.

English — B: It cannot be determined until runtime which hidden method will be executed in a parent class. **Türkçe — B:** Parent class'ta hangi hidden method'un çalıştırılacağı runtime'a kadar belirlenemez.

English — C: Marking a method `static` prevents it from being overridden or hidden. **Türkçe — C:** Bir method'u `static` olarak işaretlemek onun override veya hide edilmesini engeller.

English — D: Marking a method `final` prevents it from being overridden or hidden. **Türkçe — D:** Bir method'u `final` olarak işaretlemek onun override veya hide edilmesini engeller.

English — E: The reference type of the variable determines which overridden method will be called at runtime. **Türkçe — E:** Runtime'da hangi overridden method'un çağrılacağını variable'ın reference type'ı belirler.

English — F: The reference type of the variable determines which hidden method will be called at runtime. **Türkçe — F:** Runtime'da hangi hidden method'un çağrılacağını variable'ın reference type'ı belirler.

Question 21

English: Given the following record declaration, which lines of code can fill in the blank and allow the code to compile? (Choose all that apply.) **Türkçe:** Aşağıdaki record declaration verildiğinde hangi code satırları boşluğu doldurup kodun derlenmesini sağlayabilir? (Uygun olanların tümünü seçin.)

```
public record RabbitFood(int size, String brand, LocalDate expires) {
    public static int MAX_STORAGE = 100;
    public RabbitFood() {
        _____;
    }
}
```

English — A: `size = MAX_STORAGE` **Türkçe — A:** `size = MAX_STORAGE`

English — B: `this.size = 10` **Türkçe — B:** `this.size = 10`

English — C: ``if(expires.isAfter(LocalDate.now())) throw new RuntimeException()`` **Türkçe — C:** ``if(expires.isAfter(LocalDate.now())) throw new RuntimeException()``

English — D: `if(brand==null) super.brand = "Unknown"` **Türkçe — D:** `if(brand==null) super.brand = "Unknown"`

English — E: `throw new RuntimeException()` **Türkçe — E:** `throw new RuntimeException()`

English — F: None of the above **Türkçe — F:** Yukarıdakilerin hiçbiri

Question 22

English: Which of the following can be inserted in the `rest()` method? (Choose all that apply.) **Türkçe:** Aşağıdakilerden hangileri `rest()` method'una eklenebilir? (Uygun olanların tümünü seçin.)

```
public class Lion {
    class Cub {}
    static class Den {}
    static void rest() {
        _____;
    }
}
```

English — A: `Cub a = Lion.new Cub()` **Türkçe — A:** `Cub a = Lion.new Cub()`

English — B: `Lion.Cub b = new Lion().Cub()` **Türkçe — B:** `Lion.Cub b = new Lion().Cub()`

English — C: `Lion.Cub c = new Lion().new Cub()` **Türkçe — C:** `Lion.Cub c = new Lion().new Cub()`

English — D: `var d = new Den()` **Türkçe — D:** `var d = new Den()`

English — E: `var e = Lion.new Cub()` **Türkçe — E:** `var e = Lion.new Cub()`

English — F: `Lion.Den f = Lion.new Den()` **Türkçe — F:** `Lion.Den f = Lion.new Den()`

English — G: `Lion.Den g = new Lion.Den()` **Türkçe — G:** `Lion.Den g = new Lion.Den()`

English — H: `var h = new Cub()` **Türkçe — H:** `var h = new Cub()`

Question 23

English: Given the following program, what can be inserted into the blank line that would allow it to print `Swim!` at runtime? **Türkçe:** Aşağıdaki program verildiğinde runtime'da `Swim!` yazdırmasını sağlamak için boş satıra ne eklenebilir?

```
interface Swim {
    default void perform() { System.out.print("Swim!"); }
}

interface Dance {
    default void perform() { System.out.print("Dance!"); }
}

public class Penguin implements Swim, Dance {
    public void perform() { System.out.print("Smile!"); }
    private void doShow() {
        _____;
    }
    public static void main(String[] eggs) {
        new Penguin().doShow();
    }
}
```

English — A: `super.perform()` **Türkçe — A:** `super.perform()`

English — B: `Swim.perform()` **Türkçe — B:** `Swim.perform()`

English — C: `super.Swim.perform()` **Türkçe — C:** `super.Swim.perform()`

English — D: `Swim.super.perform()` **Türkçe — D:** `Swim.super.perform()`

English — E: The code does not compile regardless of what is inserted into the blank. **Türkçe — E:** Boşluğa ne eklenirse eklensin kod derlenmez.

English — F: The code compiles, but due to polymorphism, it is not possible to produce the requested output without creating a new object. **Türkçe — F:** Kod derlenir ancak polymorphism nedeniyle yeni bir object oluşturmadan istenen çıktıyı üretmek mümkün değildir.

Question 24

English: Which lines of the following interface do not compile? (Choose all that apply.) **Türkçe:** Aşağıdaki interface'in hangi satırları derlenmez? (Uygun olanların tümünü seçin.)

```
1: public interface BigCat {  
2:     abstract String getName();  
3:     static int hunt() { getName(); return 5; }  
4:     default void climb() { rest(); }  
5:     private void roar() { getName(); climb(); hunt(); }  
6:     private static boolean sneak() { roar(); return true; }  
7:     private int rest() { return 2; };  
8: }
```

English — A: Line 2 **Türkçe — A:** Line 2

English — B: Line 3 **Türkçe — B:** Line 3

English — C: Line 4 **Türkçe — C:** Line 4

English — D: Line 5 **Türkçe — D:** Line 5

English — E: Line 6 **Türkçe — E:** Line 6

English — F: Line 7 **Türkçe — F:** Line 7

English — G: None of the above **Türkçe — G:** Yukarıdakilerin hiçbiri

Question 25

English: What does the following program print? **Türkçe:** Aşağıdaki program ne yazdırır?

```
1: public class Zebra {  
2:     private int x = 24;  
3:     public int hunt() {  
4:         String message = "x is ";  
5:         abstract class Stripes {  
6:             private int x = 0;  
7:             public void print() {  
8:                 System.out.print(message + Zebra.this.x);  
9:             }  
10:        }  
11:        var s = new Stripes() {};  
12:        s.print();  
13:        return x;  
}
```

```

14:     }
15:     public static void main(String[] args) {
16:         new Zebra().hunt();
17:     }
18: }

```

English — A: x is 0 **Türkçe — A:** x is 0

English — B: x is 24 **Türkçe — B:** x is 24

English — C: Line 6 generates a compiler error. **Türkçe — C:** Line 6 compiler error üretir.

English — D: Line 8 generates a compiler error. **Türkçe — D:** Line 8 compiler error üretir.

English — E: Line 11 generates a compiler error. **Türkçe — E:** Line 11 compiler error üretir.

English — F: None of the above **Türkçe — F:** Yukarıdakilerin hiçbirisi

Question 26

English: Which statements about the following enum are true? (Choose all that apply.) **Türkçe:** Aşağıdaki enum hakkında hangi ifadeler doğrudur? (Uygun olanların tümünü seçin.)

```

1: public enum Animals {
2:     MAMMAL(true), INVERTEBRATE(Boolean.FALSE), BIRD(false),
3:     REPTILE(false), AMPHIBIAN(false), FISH(false) {
4:         public int swim() { return 4; }
5:     }
6:     final boolean hasHair;
7:     public Animals(boolean hasHair) {
8:         this.hasHair = hasHair;
9:     }
10:    public boolean hasHair() { return hasHair; }
11:    public int swim() { return 0; }
12: }

```

English — A: Compiler error on line 2 **Türkçe — A:** Line 2'de compiler error

English — B: Compiler error on line 3 **Türkçe — B:** Line 3'te compiler error

English — C: Compiler error on line 7 **Türkçe — C:** Line 7'de compiler error

English — D: Compiler error on line 8 **Türkçe — D:** Line 8'de compiler error

English — E: Compiler error on line 10 **Türkçe — E:** Line 10'da compiler error

English — F: Compiler error on another line **Türkçe — F:** Başka bir satırda compiler error

English — G: The code compiles successfully. **Türkçe — G:** Kod başarıyla derlenir.

Question 27

English: Assuming a record is defined with at least one field, which components does the compiler always insert, each of which may be overridden or redeclared? (Choose all that apply.) **Türkçe:** Bir record'un en az bir field ile tanımlandığını varsayarsak compiler, her biri override veya redeclare edilebilen hangi component'leri her zaman ekler? (Uygun olanların tümünü seçin.)

English — A: A no-argument constructor **Türkçe — A:** No-argument constructor

English — B: An accessor method for each field **Türkçe — B:** Her field için bir accessor method

English — C: The `toString()` method **Türkçe — C:** `toString()` method'u

English — D: The `equals()` method **Türkçe — D:** `equals()` method'u

English — E: A mutator method for each field **Türkçe — E:** Her field için bir mutator method

English — F: A sort method for each field **Türkçe — F:** Her field için bir sort method'u

English — G: The `hashCode()` method **Türkçe — G:** `hashCode()` method'u

Question 28

English: Which of the following classes and interfaces do not compile? (Choose all that apply.) **Türkçe:** Aşağıdaki class ve interface'lerden hangileri derlenmez? (Uygun olanların tümünü seçin.)

```
public abstract class Camel { void travel(); }
public interface EatsGrass { private abstract int chew(); }
public abstract class Elephant {
    abstract private class SleepsAlot {
        abstract int sleep();
    }
}
public class Eagle { abstract soar(); }
public interface Spider { default void crawl() {} }
```

English — A: `Camel` **Türkçe — A:** `Camel`

English — B: `EatsGrass` **Türkçe — B:** `EatsGrass`

English — C: Elephant **Türkçe — C:** Elephant

English — D: Eagle **Türkçe — D:** Eagle

English — E: Spider **Türkçe — E:** Spider

English — F: None of the classes or interfaces compile. **Türkçe — F:** Class ve interface'lerin hiçbiri derlenmez.

Question 29

English: How many lines of the following program contain a compilation error? **Türkçe:** Aşağıdaki programın kaç satırı compilation error içerir?

```
1: class Primate {
2:     protected int age = 2;
3:     { age = 1; }
4:     public Primate() {
5:         this().age = 3;
6:     }
7: }
8: public class Orangutan {
9:     protected int age = 4;
10:    { age = 5; }
11:    public Orangutan() {
12:        this().age = 6;
13:    }
14:    public static void main(String[] bananas) {

15:        final Primate x = (Primate)new Orangutan();
16:        System.out.println(x.age);
17:    }
18: }
```

English — A: 1 **Türkçe — A:** 1

English — B: None, and the program prints 1 at runtime. **Türkçe — B:** Hiçbiri; program runtime'da 1 yazdırır.

English — C: None, and the program prints 3 at runtime. **Türkçe — C:** Hiçbiri; program runtime'da 3 yazdırır.

English — D: None, but it causes a `ClassCastException` at runtime. **Türkçe — D:** Hiçbiri; ancak runtime'da `ClassCastException` oluşmasına neden olur.

English — E: 2 **Türkçe — E:** 2

English — F: 3 Türkçe — F: 3

English — G: 4 Türkçe — G: 4

Question 30

English: Assuming the following classes are declared as top-level types in the same file, which classes contain compiler errors? (Choose all that apply.) **Türkçe:** Aşağıdaki class'ların aynı file'da top-level type olarak bildirildiğini varsayarsak hangi class'lar compiler error içerir? (Uygun olanların tümünü seçin.)

```
sealed class Bird {
    public final class Flamingo extends Bird {}
}

sealed class Monkey {}
class EmperorTamarin extends Monkey {}
non-sealed class Mandrill extends Monkey {}
sealed class Friendly extends Mandrill permits Silly {}
final class Silly {}
```

English — A: Bird Türkçe — A: Bird

English — B: Monkey Türkçe — B: Monkey

English — C: EmperorTamarin Türkçe — C: EmperorTamarin

English — D: Mandrill Türkçe — D: Mandrill

English — E: Friendly Türkçe — E: Friendly

English — F: Silly Türkçe — F: Silly

English — G: All of the classes compile without issue. Türkçe — G: Class'ların tümü sorunsuz derlenir.

Coverage ledger / Kapsam kaydı

Kaynak aralığı	Marker aralığı	Kapsanan içerik	Durum
Chapter 7, PDF 345–418	0345–0418	Başlıklar, prose paragrafları, tablolar/figure metinleri, code, Summary, Exam Essentials ve Review Questions 1–30	Eksiksiz işlendi

Toplam beklenen ve bulunan source-page marker sayısı: **74 / 74**.

Appendix · Önceki review-question teknik analizleri

Chapter 7 Review Questions 15–29, English–Türkçe eşleşmeleriyle işlenmiştir. [Vocabulary](#) · [Grammar notes](#).

Editor note: Gönderilen metin Question 15'in ortasında başlayıp Question 29'un ortasında bitiyordu. Eksik kısımlar proje içindeki kaynak PDF'den tamamlandı. OCR kaynaklı - > dizileri Java 17 switch arrow'ı -> olarak düzeltildi.

1. Nested classes and interface members · Questions 15–18

Question 15 · Anonymous subclass of a `final` inner class

```
public class Ghost {
    public static void boo() {
        System.out.println("Not scared");
    }

    protected final class Spirit {
        public void boo() {
            System.out.println("Booo!!!");
        }
    }

    public static void main(String... haunt) {
        var g = new Ghost().new Spirit() {};
        // Which expression can print "Not scared"?
    }
}
```

English: Which independently inserted lines let the program print `Not scared` at runtime? **Türkçe:** Ayrı ayrı eklendiğinde hangi satırlar programın runtime'da `Not scared` yazdırmasını sağlar?

Cevap: G — None of the above. Sorudaki blank'e gelmeden önce compilation başarısızdır. `{}` bir anonymous subclass oluşturur; `Spirit` ise `final` olduğu için extend edilemez. Anonymous body kaldırılıp doğrudan `Spirit` oluşturulsaydı `Ghost.boo()` static method'u `Not scared`, `g.boo()` ise `Booo!!!` yazdırırdı.

OCP exam trap: `{}` yalnız object creation süsü değildir; burada yeni bir anonymous class bildirir.

Question 16 · Static nested class and outer instance state

English: A static nested `OstrichWrangler` tries to return the outer object's instance field `count`. What is the compilation result? **Türkçe:** Static nested `OstrichWrangler`, outer object'in instance field'ı `count` değerini döndürmeye çalışır. Derleme sonucu nedir?

Cevap: E — compiler error on line 5. Static nested class'ın enclosing `Ostrich` instance'ına ait implicit reference'ı yoktur. `count` ancak bir `Ostrich` object'i üzerinden okunabilir. Kod bu hâliyle bytecode üretmez; dolayısıyla class-file sayılarını veren seçenekler uygulanmaz.

Question 17 · Implicit interface modifiers

English: Which declarations fail when an interface contains fields, static methods, bodyless methods, default methods, protected methods, and private static methods? **Türkçe:** Bir interface field, static method, bodyless method, default method, protected method ve private static method içerdiğinde hangi declaration'lar başarısız olur?

Cevap: E ve G — lines 5 and 7. Body taşıyan non-static interface method `default` veya `private` olmalıdır; bu yüzden line 5 geçersizdir. Interface member'ları `protected` olamaz; line 7 de geçersizdir. Field'lar implicit `public static final`, bodyless method'lar implicit `public abstract` olur. `static`, `default` ve `private static` concrete interface method'ları geçerlidir.

Question 18 · Inner class construction from static context

```
System.out.print(switch (new Diet().getFavorite()) {
    case APPLES -> "a";
    case BERRIES -> "b";
    default -> "c";
});
```

English: `Diet` is a non-static inner class of `Deer`. What happens when `main()`, a static method, invokes `new Diet()`? **Türkçe:** `Diet`, `Deer`'in non-static inner class'ıdır. Static `main()` method'u `new Diet()` çağırdığında ne olur?

Cevap: E — the `main()` method does not compile. Inner class instance'ı bir outer instance'a bağlıdır. Static context'te implicit `Deer.this` yoktur. `new Deer().new Diet()` kullanılırsa kod derlenir ve `b` yazdırır.

2. Enums, polymorphism and records · Questions 19–21**Question 19 · Abstract enum methods**

English: `FOOD` declares abstract `isHealthy()`, but only the `INSECTS` constant supplies an implementation. Which listed values are printed? **Türkçe:** `FOOD`, abstract `isHealthy()` bildirir; ancak yalnız `INSECTS` constant'ı implementation sağlar. Listelenen değerlerden hangileri yazdırılır?

Cevap: G — Does not compile. Abstract enum method'u her enum constant implement etmelidir. Alternatif olarak method abstract olmaktan çıkarılıp enum gövdesinde concrete implementation verilebilir. `BERRIES`, `FISH`, `ROOTS`, `COOKIES` ve `HONEY` eksik kaldığından hiçbir çıktı oluşmaz.

Question 20 · Overriding versus hiding

English: Which statements correctly distinguish runtime selection of overridden methods from compile-time selection of hidden methods? **Türkçe:** Hangi ifadeler overridden method'ların runtime seçimini hidden method'ların compile-time seçiminden doğru biçimde ayırır?

Cevap: A, D ve F. Overridden instance method runtime object type'a göre seçilir; arbitrary bir object için kesin target runtime'a kadar bilinmeyebilir. `final` method override/hide edilemez. Hidden static method ise reference type ve call location kullanılarak compile time'da seçilir.

OCP exam trap: `static` method override edilmez ama hide edilebilir; dolayısıyla “static prevents hiding” yanlıştır.

Question 21 · Non-canonical record constructor

```
import java.time.LocalDate;

public record RabbitFood(int size, String brand, LocalDate expires) {
    public static int MAX_STORAGE = 100;

    public RabbitFood() {
        // blank
    }
}
```

English: Which individual statements can fill the no-argument record constructor and make the declaration compile? **Türkçe:** Hangi tekil statement'lar record'un no-argument constructor'ını doldurup declaration'ın derlenmesini sağlar?

Cevap: F — None of the above. Bu constructor canonical/compact değil, overloaded bir constructor'dır; ilk statement başka bir constructor'a delegate etmelidir: `this(100, "Unknown", LocalDate.now());`. Component field'larına doğrudan assignment yapmak ya da yalnız exception fırlatmak bu zorunluluğu karşılamaz.

3. Nested classes and interface dispatch · Questions 22–25

Question 22 · Constructing inner and static nested classes

English: Which declarations in static `rest()` correctly instantiate non-static inner class `Cub` or static nested class `Den`? **Türkçe:** Static `rest()` içinde hangi declaration'lar non-static inner class `Cub` veya static nested class `Den` instance'ını doğru oluşturur?

Cevap: C, D ve G. Inner class için outer instance gerekir: `new Lion().new Cub()`. Static nested class için outer object gerekmez; hem `new Den()` hem `new Lion.Den()` geçerlidir. `Lion.new ...` Java sözdizimi değildir.

Question 23 · Selecting an inherited default method

English: `Penguin` implements two interfaces with the same default `perform()` method and overrides it. What can `doShow()` use to print `Swim!`? **Türkçe:** `Penguin`, aynı default `perform()` method'una sahip iki interface'i implement eder ve method'u override eder. `doShow()`, `Swim!` yazdırmak için ne kullanabilir?

Cevap: D — `Swim.super.perform()` ; . Conflicting default methods class tarafından override edilmiştir. Belirli bir interface'in inherited default implementation'ı `InterfaceName.super.method()` ile çağrılır.

Question 24 · Static and instance access inside interfaces

English: Which lines fail when static and instance interface methods call abstract, default, and private helpers? **Türkçe:** Static ve instance interface method'ları abstract, default ve private helper'ları çağırdığında hangi satırlar başarısız olur?

Cevap: B ve E — lines 3 and 6. Static `hunt()` bir instance olmadan `getName()` çağırılmaz. Private static `sneak()` de instance method `roar()` ı doğrudan çağırılmaz. Buna karşılık instance `roar()`, abstract/default/static method'lara erişebilir; default `climb()` da private instance `rest()` i çağırabilir.

Question 25 · Local and anonymous classes

English: A local abstract class declares its own `x`, while its method prints `message + Zebra.this.x`. An anonymous concrete subclass is then instantiated. What is printed? **Türkçe:** Local abstract class kendi `x` field'ını bildirir; method'u ise `message + Zebra.this.x` yazdırır. Ardından anonymous concrete subclass oluşturulur. Ne yazdırılır?

Cevap: B — `x` is 24. `Zebra.this.x` explicitly outer Zebra instance field'ını seçer; local class'taki `x = 0` seçilmez. `message` effectively final olduğu için local class tarafından capture edilebilir. Anonymous subclass, abstract method kalmadığından concrete olabilir.

4. Enums, records and abstract declarations · Questions 26–29

Question 26 · Enum constructor access and constant-list terminator

English: Which statements are true when an enum gives `FISH` a constant-specific class body, declares a field, and uses a public constructor? **Türkçe:** Bir enum `FISH` için constant-specific class body sağladığında, field bildirdiğinde ve public constructor kullandığında hangi ifadeler doğrudur?

Cevap: C ve F. Enum constructor'ı implicit `private`dir; explicit `public` olamaz, bu nedenle line 7 hatalıdır. Constant list'ten sonra field/method gibi member'lar geldiği için line 5'teki closing brace'in ardından semicolon gerekir: `};`. Diğer enum yapıları geçerlidir.

Question 27 · Compiler-generated record members

English: For a record with at least one component, which generated members can always be overridden or redeclared? **Türkçe:** En az bir component'i olan record için compiler'ın ürettiği hangi member'lar her zaman override veya redeclare edilebilir?

Cevap: B, C, D ve G. Compiler her component için accessor ile birlikte `toString()`, `equals()` ve `hashCode()` sağlar. Record mutator veya sort method üretmez. En az bir component bulunduğu için generated constructor no-argument değil, tüm component'leri declaration sırasıyla alan canonical constructor'dır.

Question 28 · Invalid abstract declarations

English: Which of the five class/interface declarations fail because of missing bodies, illegal modifier combinations, or abstract methods in a concrete class? **Türkçe:** Beş class/interface declaration'ından hangileri eksik body, geçersiz modifier birleşimi veya concrete class'taki abstract method nedeniyle başarısız olur?

Cevap: A, B ve D. `Camel.travel()` body veya `abstract` modifier ister. `EatsGrass.chew()` için `private abstract` birleşimi geçersizdir. Concrete `Eagle` abstract method bildiremez; `Elephant` ve `Spider` derlenir.

Question 29 · `this` versus `this()` and unrelated casts

English: How many lines fail when constructors use `this().age` and an `Orangutan` is cast to unrelated class `Primate`? **Türkçe:** Constructor'lar `this().age` kullandığında ve `Orangutan` unrelated `Primate` class'ına cast edildiğinde kaç satır başarısız olur?

Cevap: F — three lines. Lines 5 ve 12'de constructor call olan `this()`, current instance reference `this` yerine kullanılıp `.age` ile zincirlenemez. Line 15'te unrelated `Orangutan` → `Primate` cast'i compile time'da reddedilir; runtime output veya `ClassCastException` oluşmaz.

Kısa tekrar özeti

- Inner class outer instance ister; static nested class istemez.
- Interface field'i `public static final`, bodyless method `public abstract`dir.
- Enum abstract method'unu her constant implement etmelidir.
- Override runtime type'a, static hiding reference/context'e bağlıdır.
- Record constructor delegation, `this(...)`; current object reference, `this`.

OCP tarzı özgün mini quiz

- 1 `record Point(int x, int y)` içinde `Point() { x=0; y=0; }` derlenir mi?
- 2 Static nested class, outer class'ın private static field'ına erişebilir mi?
- 3 İki interface aynı default method'u sağlarsa implementing class ne yapmalıdır?
- 4 `final` inner class'tan anonymous subclass oluşturulabilir mi?

Cevaplar ve açıklamalar

- 1 **Does not compile;** ilk statement `this(0, 0);` olmalıdır.
- 2 **Evet.** Sorun private access değil, instance receiver eksikliğidir; static member için outer object gerekmez.
- 3 Signature conflict'i çözmek için method'u override etmelidir.
- 4 **Hayır.** Anonymous class da inheritance kullandığı için `final` engeline tabidir.

5. Teknik pekiştirme · Beyond Classes master map

Interface member modifier tablosu

Declaration türü	Implicit/izin verilen modifier	Body
Field	<code>public static final</code>	Initializer zorunlu
Abstract method	<code>public abstract</code>	Yok
Default method	<code>public default</code>	Var
Static method	<code>public static</code>	Var
Private instance helper	<code>private</code>	Var
Private static helper	<code>private static</code>	Var

Interface method `protected` olamaz. Private method inherited değildir ve abstract olamaz. Static interface method da implementing class'a inherited olmaz; `InterfaceName.method()` ile çağrılır. Interface birden fazla interface'i `extends` ile inherit edebilir.

İki inherited default method aynı signature ile conflict oluştursa class veya subinterface bunu override ederek çözmelidir. Belirli parent default'u seçmek için `InterfaceName.super.method()` kullanılır.

Enum: class gibi, instance listesi sabit

- Enum constant'lar `public static final` instance'lardır.
- `values()` declaration order'da yeni array döndürür.
- `valueOf("NAME")` exact ve case-sensitive name ister; bulunamazsa `IllegalArgumentException` fırlatır.
- `name()` declared constant adını; `ordinal()` zero-based position'ı döndürür.
- Enum constructor implicit private'dır ve yalnız constant creation sırasında çalışır; `new` ile enum oluşturulamaz.
- Constant list'ten sonra field/method/constructor varsa semicolon zorunludur.
- Abstract enum method varsa her constant implementation sağlamalıdır; concrete method constant-specific body'de optionally override edilebilir.

Memory tip: `ordinal()` business id değildir; declaration sırası değişirse değer de değişir.

Sealed hierarchy kontrol kartı

```
sealed class Shape permits Circle, Polygon {}
final class Circle extends Shape {}
non-sealed class Polygon extends Shape {}
```

- `permits` yalnız **direct** subtype'ları listeler.
- Her permitted direct subclass `final`, `sealed` veya `non-sealed` olmalıdır.

- 3 Permitted subtype sealed parent'ı gerçekten directly extend/implement etmelidir.
- 4 Named module içinde aynı module'da; unnamed module'da aynı package'da olma kuralı uygulanır.
- 5 Direct subtype'lar aynı source file'da veya nested ise `permits` clause compiler tarafından çıkarılabilir.
- 6 `non-sealed` branch'ten sonraki indirect subtype'lar permits listesinde olmak zorunda değildir.

Sealed type abstract olabilir. `sealed`, “hiç extend edilemez” değil, “doğrudan kimlerin extend edebileceği kontrollüdür” anlamına gelir.

Record: generated API ve constructor kuralları

```
record Range(int min, int max) {
    Range {
        if (min > max) throw new IllegalArgumentException();
    }
}
```

Compiler her component için `private final` field, same-name accessor, canonical constructor ve `equals()`, `hashCode()`, `toString()` implementation'ları sağlar.

- Record implicit `final`dır ve `java.lang.Record`u extend eder; başka class extend edemez ama interface implement edebilir.
- Additional instance field ve instance initializer bildiremez; static member bildirebilir.
- Canonical constructor'ın parameter listesi component listesiyle eşleşir.
- Canonical constructor'ın access'i record declaration'dan daha restrictive olamaz; public record'un canonical constructor'ı public olmalıdır.
- Compact constructor parameter listesi yazmaz; validation/normalization body'de yapılır ve implicit field assignment body sonunda gerçekleşir.
- Non-canonical constructor ilk statement'ta `this(...)` ile başka record constructor'ına delegate etmelidir.
- Explicit accessor component type ile aynı return type'a sahip olmalı ve parameter almamalıdır.

Record **shallowly immutable**dir: component reference'ları finaldir, fakat bir component mutable object gösteriyorsa o object'in içeriği kendiliğinden immutable olmaz. Gerçek immutability için defensive copy gerekir.

Nested type karşılaştırması

Tür	Outer instance	Static context erişimi	Local capture
Inner member class	Gerekir	Outer instance üzerinden	Uygulanmaz
Static nested class	Gerekmez	Doğrudan	Uygulanmaz
Local class	Context'e bağlı	Declaration scope'u	Final/effectively final
Anonymous class	Context'e bağlı	Tek expression	Final/effectively final

Inner class oluşturma syntax'ı `outer.new Inner()` dir. Local class yalnız declaration'dan sonra ve enclosing block içinde görünür. Anonymous class'ın adı ve explicit constructor'ı yoktur; class extend

eder veya interface implement eder. Java 16'dan itibaren inner/local class'lar static member bildirebilir; outer instance gereksinimi yine instance access için geçerlidir.

Reference kapıyı açar, object method'u seçer

```
Animal animal = new Dog();
animal.move(); // accessible olup olmadığı Animal'a, çalışan override Dog'a bağlı
```

- Compile time reference type hangi field ve method signature'larının accessible olduğunu belirler.
- Runtime object type overridden instance method implementation'ını belirler.
- Field ve static method selection reference/call-site type'a bağlıdır.
- Cast object'i değiştirmez; yalnız reference view'ını değiştirir.

Cast kontrolü iki aşamalıdır:

- 1 Type'lar arasında mümkün bir inheritance/interface ilişkisi yoksa **Does not compile**.
- 2 Cast compile-time mümkün fakat object target type'ın instance'ı değilse runtime'da `ClassCastException`.

`null instanceof AnyType` false'tur. Pattern matching kullanıldığında cast ancak match true path'inde güvenli hâle gelir.

Memory tip — R/O: Reference erişim kapısını açar; Object overridden method'u çalıştırır.

60 saniyelik active recall

- 1 Interface static method neden implementing class üzerinden inherited olmaz?
- 2 Permitted subclass hangi üç modifier'dan birini taşır?
- 3 Compact record constructor field assignment'ı ne zaman gerçekleşir?
- 4 Record neden deep değil shallow immutability sağlar?
- 5 Compile-time cast error ile `ClassCastException` farkını anlat.

Hızlı kontrol

- 1 Static method interface'e aittir ve inheritance contract'ına katılmaz.
- 2 `final`, `sealed` veya `non-sealed`.
- 3 Compact constructor body başarıyla tamamlandığında implicit olarak.
- 4 Final component reference mutable bir target object gösterebilir.
- 5 İmkânsız type ilişkisi derlenmez; mümkün görünen fakat object'e uymayan cast runtime'da `ClassCastException` fırlatır.